UNIT - 2

Faculty: Dr. Jisha P

Syllabus

Gate Level Modelling: Gate Types, Gate Delays, Examples.

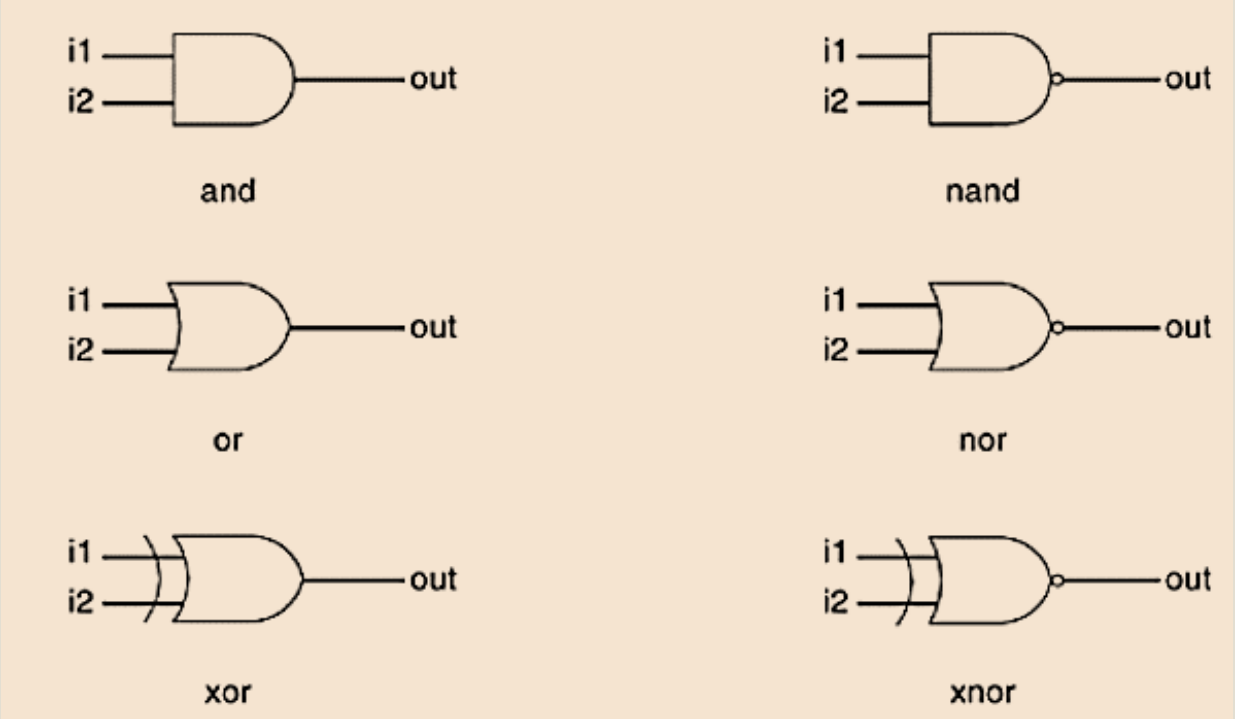
Dataflow Modelling: Continuous assignment, Delays, Expressions, Operators, Operands, Operator Types, and Examples.

Gate Types

- A logic circuit can be designed by use of logic gates.
- Verilog supports basic logic gates as predefined primitives.
- These primitives are instantiated like modules except that they are predefined in Verilog and do not need a module definition.
- All logic circuits can be designed by using basic gates.
- There are two classes of basic gates: **and/or gates** and **buf/not gates**.

And/Or Gates

- And/or gates have one scalar output and multiple scalar inputs.
- The first terminal in the list of gate terminals is an output and the other terminals are inputs.
- The output of a gate is evaluated as soon as one of the inputs changes.
- The and/or gates available in Verilog are shown below.
  - `and` `or` `xor` `nand` `nor` `xnor`
- **These gates are instantiated to build logic circuits in Verilog.**



Example - Gate Instantiation of And/Or Gates

```
wire OUT, IN1, IN2;
// basic gate instantiations.
and a1(OUT, IN1, IN2);
nand na1(OUT, IN1, IN2);
or or1(OUT, IN1, IN2);
nor nor1(OUT, IN1, IN2);
xor x1(OUT, IN1, IN2);
xnor nx1(OUT, IN1, IN2);
nand na1_3inp(OUT, IN1, IN2, IN3);// More than two inputs; 3 input nand gate
and (OUT, IN1, IN2); // gate instantiation without instance name
                     // legal gate instantiation
```

Truth Tables for And/Or

and	i1: 0	1	x	z
i2: 0	0	0	0	0
1	0	1	x	x
x	0	x	x	x
z	0	x	x	x

nand	i1: 0	1	x	z
i2: 0	1	1	1	1
1	1	0	x	x
x	1	x	x	x
z	1	x	x	x

or	i1: 0	1	x	z
i2: 0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

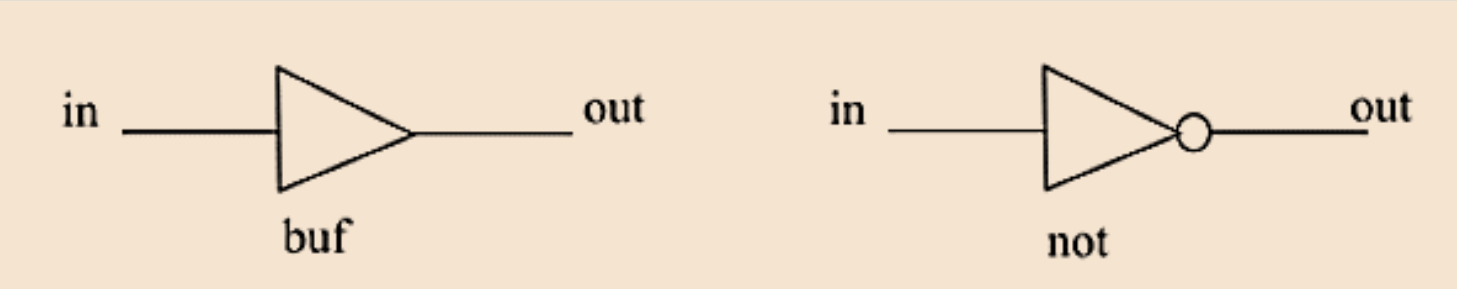
nor	i1: 0	1	x	z
i2: 0	1	0	x	x
1	0	0	0	0
x	x	0	x	x
z	x	0	x	x

xor	i1: 0	1	x	z
i2: 0	0	1	x	x
1	1	0	x	x
x	x	x	x	x
z	x	x	x	x

xnor	i1: 0	1	x	z
i2: 0	1	0	x	x
1	0	1	x	x
x	x	x	x	x
z	x	x	x	x

Buf/Not Gates

- Buf/not gates have one scalar input and one or more scalar outputs.
- The last terminal in the port list is connected to the input. Other terminals are connected to the outputs.
- We will discuss gates that have one input and one output.
- Two basic buf/not gate primitives are provided in Verilog.
- ***buf*** ***not***
- The symbols for these logic gates are



Example - Gate Instantiations of Buf/Not Gates

- `// basic gate instantiations.`
- `buf b1(OUT1, IN);`
- `not n1(OUT1, IN);`
- `// More than two outputs`
- `buf b1_2out(OUT1, OUT2, IN);`
- `// gate instantiation without instance name`
- `not (OUT1, IN); // legal gate instantiation`

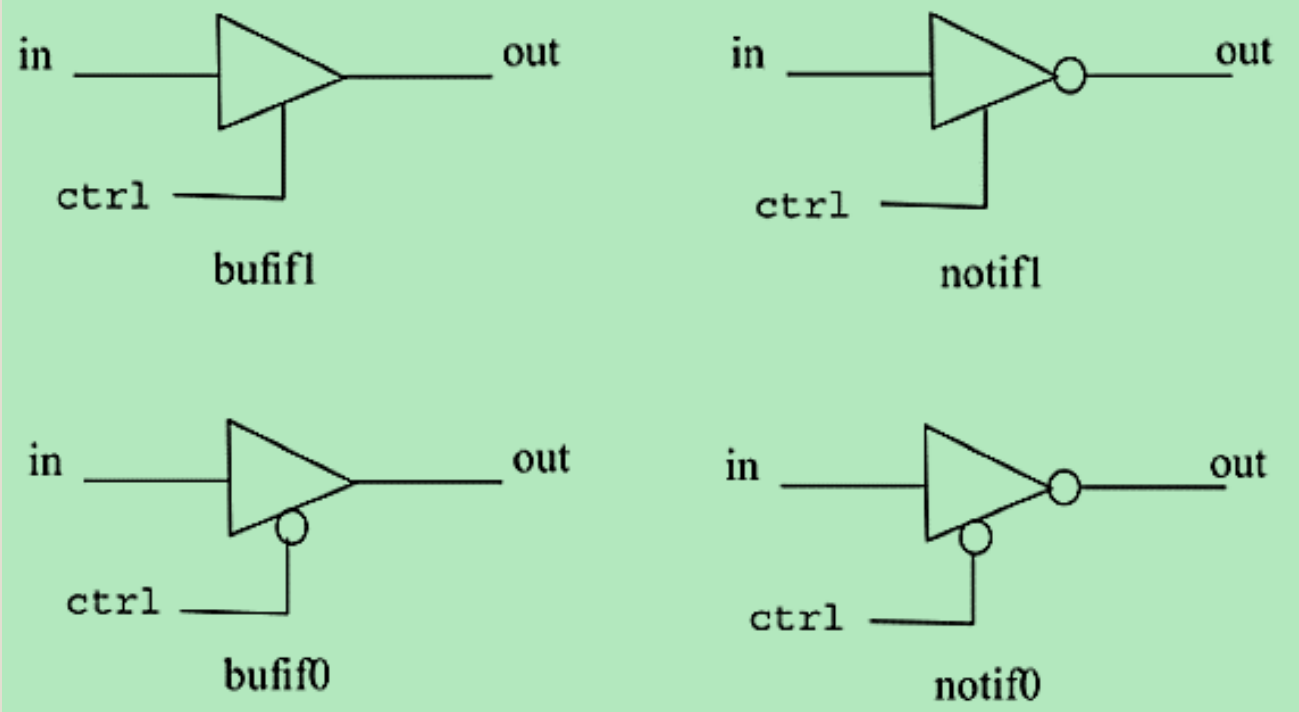
Truth Tables for Buf/Not Gates

buf	in	out
	0	0
	1	1
	x	x
	z	x

not	in	out
	0	1
	1	0
	x	x
	z	x

Bufif/notif

- Gates with an additional control signal on buf and not gates are also available.
- **bufif1 notif1**
- **bufif0 notif0**
- These gates propagate only if their control signal is asserted.
- They propagate z if their control signal is deasserted.
- Symbols for bufif/notif are:



Truth Tables for Bufif/Notif Gates

- These gates are used when a signal is to be driven only when the control signal is asserted.
- Such a situation is applicable when multiple drivers drive the signal.
- These drivers are designed to drive the signal on mutually exclusive control signals.

bufif1	ctrl 0	1	x	z
in 0	z	0	L	L
1	z	1	H	H
x	z	x	x	x
z	z	x	x	x

bufif0	ctrl 0	1	x	z
in 0	0	z	L	L
1	1	z	H	H
x	x	z	x	x
z	x	z	x	x

notif1	ctrl 0	1	x	z
in 0	z	1	H	H
1	z	0	L	L
x	z	x	x	x
z	z	x	x	x

notif0	ctrl 0	1	x	z
in 0	1	z	H	H
1	0	z	L	L
x	x	z	x	x
z	x	z	x	x

Gate Instantiations of Bufif/Notif Gates

```
//Instantiation of bufif gates.
bufif1 b1 (out, in, ctrl);
bufif0 b0 (out, in, ctrl);
//Instantiation of notif gates
notif1 n1 (out, in, ctrl);
notif0 n0 (out, in, ctrl);
```

Array of Instances

- **Simple Array of Primitive Instances**

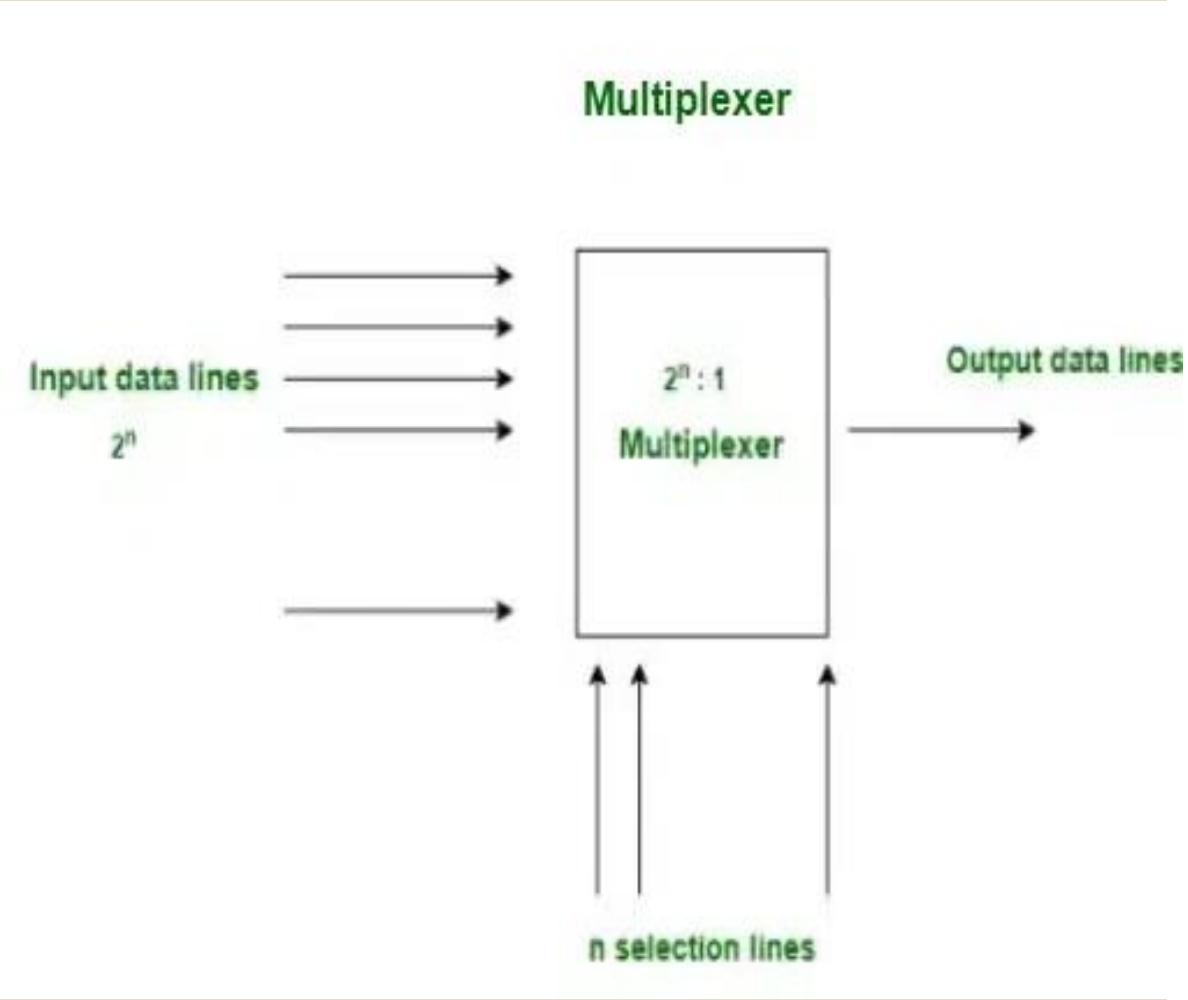
```
wire [7:0] OUT, IN1, IN2;
// basic gate instantiations.
nand n_gate[7:0](OUT, IN1, IN2);
// This is equivalent to the following 8 instantiations
nand n_gate0(OUT[0], IN1[0], IN2[0]);
nand n_gate1(OUT[1], IN1[1], IN2[1]);
nand n_gate2(OUT[2], IN1[2], IN2[2]);
nand n_gate3(OUT[3], IN1[3], IN2[3]);
nand n_gate4(OUT[4], IN1[4], IN2[4]);
nand n_gate5(OUT[5], IN1[5], IN2[5]);
nand n_gate6(OUT[6], IN1[6], IN2[6]);
nand n_gate7(OUT[7], IN1[7], IN2[7]);
```

Design of Gate-level Digital Circuits: **Multiplexer**

- A multiplexer is a combinational circuit that has many data inputs and a single output, depending on control or select inputs.
- **For N input lines, log2(N) selection lines are required, or equivalently, for 2^n input lines, n selection lines are needed.**
- Multiplexers are also known as “N-to-1 selectors,” parallel-to-serial converters, many-to-one circuits, and universal logic circuits.
- They are mainly used to increase the amount of data that can be sent over a network within a certain amount of time and bandwidth .

Multiplexer

- Multiplexer is an combinational circuit which has 2,4,8 and so on inputs with select line 1,2,3 and so on.
- Output of the multiplexer is connected to the particular input based on the select line.
- For example in 2X1 MUX, if select line is 0, first input is selected else second input is selected.
- There are different ways to describe MUX in Verilog.
- We can use gate level modelling, we can use data flow modelling, we can use ternary operator, we can use if else statement or we can use case statement.



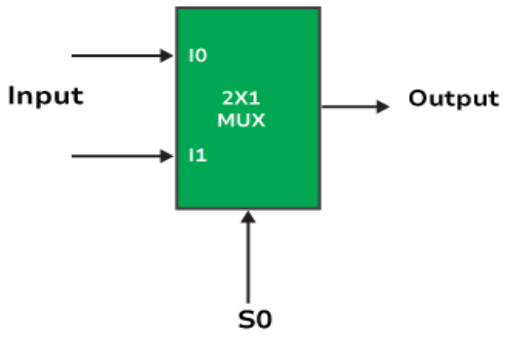
Types of Mux: 2X1 Multiplexer

- The Mux can be of different types based on input but in this article we will go through two major types of mux which are
  - 2×1 Mux
  - 4×1 Mux
- **2×1 Multiplexer**
- The 2×1 is a fundamental circuit which is also known 2-to-1 multiplexer that are used to choose one signal from two inputs and transmits it to the output.
- The 2×1 mux has two input lines, one output line, and a single selection line.
- It has various applications in digital systems such as in microprocessor it is used to select between two different data sources or between two different instructions.

2X1 Multiplexer

- In this Block Diagram where I0 and I1 are the input lines ,Y is the output line and S0 is a single select line.
- The output of the 2×1 Mux will depend on the selection line S0,
  - When S is 0(low), the I0 is selected
  - when S0 is 1(High), I1 is selected
- Logical Expression of 2×1 Mux Using the Truth Table , the Logical Expression for Mux can be determined as:

$$Y = \overline{S_0}.I_0 + S_0.I_1$$

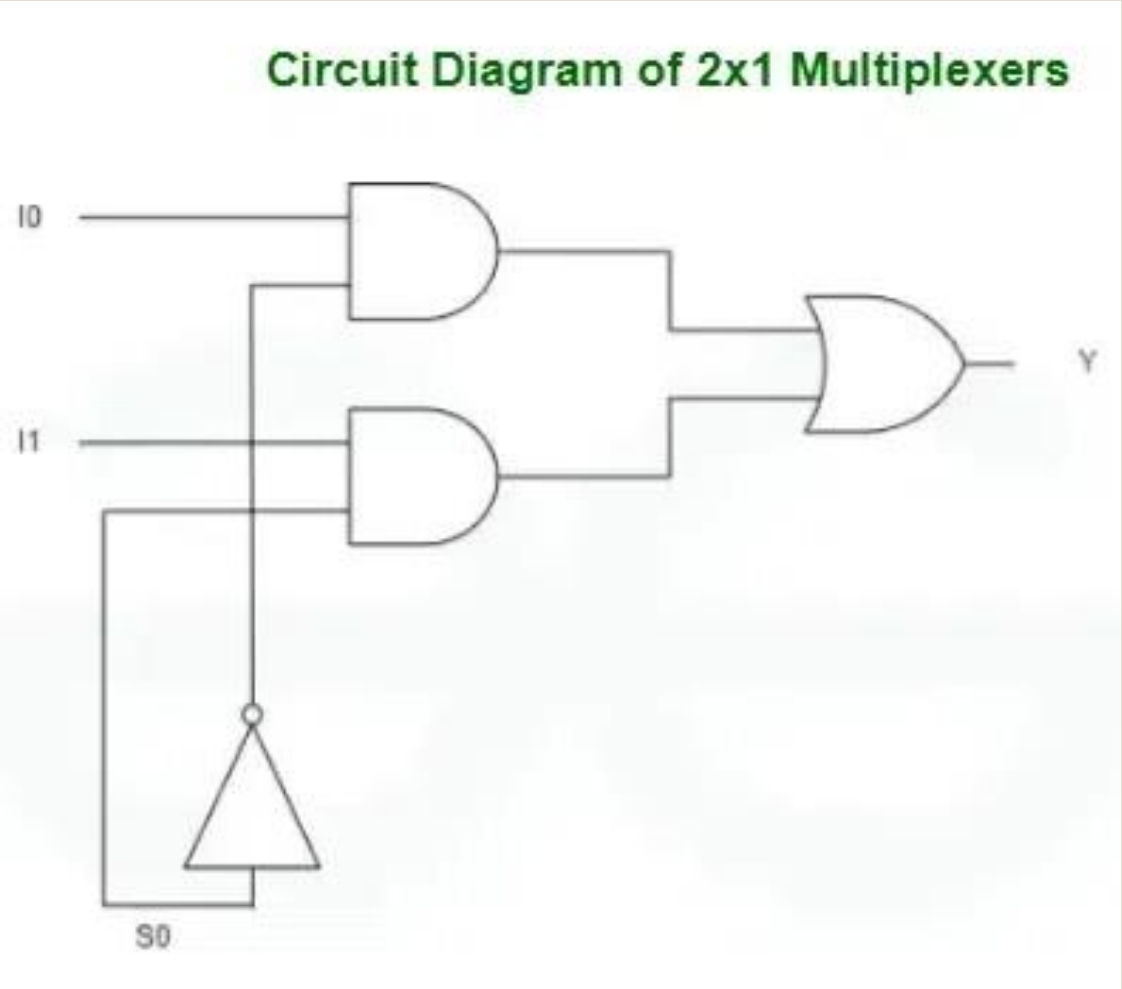


Truth Table

S_0	I_0	I_1	Y
0	0	X	0
0	1	X	1
1	X	0	0
1	X	1	1

2X1 Multiplexer

- Circuit Diagram of 2×1 Multiplexers:



2X1 Multiplexer Verilog Code

- Basic 2X1 MUX in gate level modelling.
- It has two inputs, one select line and one output.
- The boolean expression for 2X1 is given as

$$Y = \overline{S}.I0 + S.I2$$

- For this, we require two AND gate, one OR gate and one NOT gate.

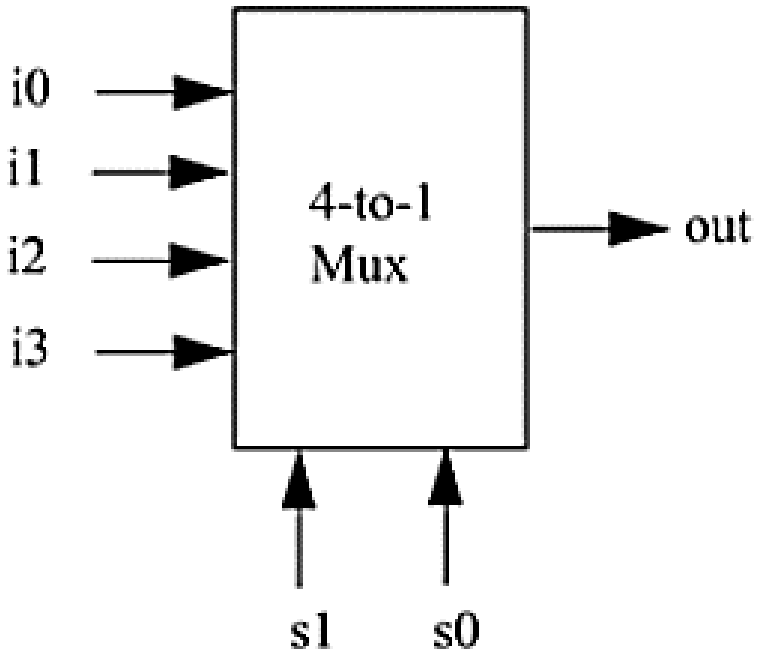
```
//2X1 MUX in gate level modeling
module mux_2x1_gl(
input I0,I1,S,
output Y);
wire sb,a,b;

not(sb,S);
and(a,sb,I0);
and(b,S,I1);
or(Y,a,b);

endmodule
```

Examples: **Design of Gate-level Digital Circuits**

- **Gate-level multiplexer**
- Design a 4-to-1 multiplexer with 2 select signals.
- Multiplexers serve a useful purpose in logic design.
- They can connect two or more sources to a single destination.
- They can also be used to implement Boolean functions.
- Assume for this example that signals s1 and s0 do not get the value x or z.
- The I/O diagram and the truth table for the multiplexer

Design of Gate-level Digital Circuits

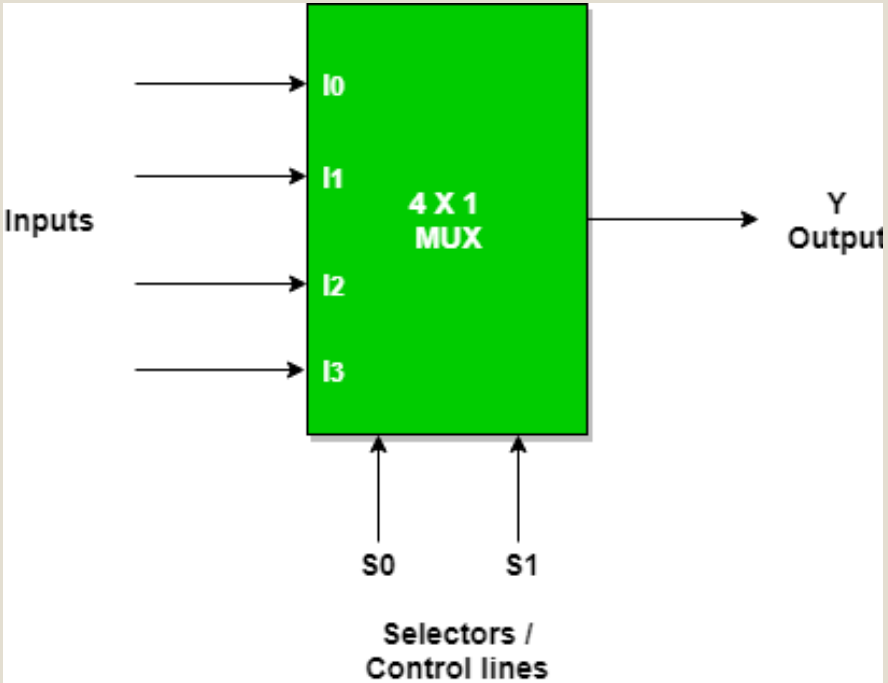
- The I/O diagram will be useful in setting up the port list for the **4-to-1 Multiplexer**.



s1	s0	out
0	0	I0
0	1	I1
1	0	I2
1	1	I3

4×1 Multiplexer

- The 4×1 Multiplexer which is also known as the 4-to-1 multiplexer.
- It is a multiplexer that has 4 inputs and a single output.
- The Output is selected as one of the 4 inputs which is based on the selection inputs.
- The number of the Selection lines will depend on the number of the input which is determined by the equation $\log_2 n$, In 4×1 Mux the selection lines can be determined as $\log_2 4 = 2$. so two selections linputs are needed.
- In the Given Block Diagram I0, I1, I2, and I3 are the 4 inputs and Y is the Single output which is based on Select lines S0 and S1.
- Multiplexer can act as universal combinational circuit. All the standard logic gates can be implemented with multiplexers.



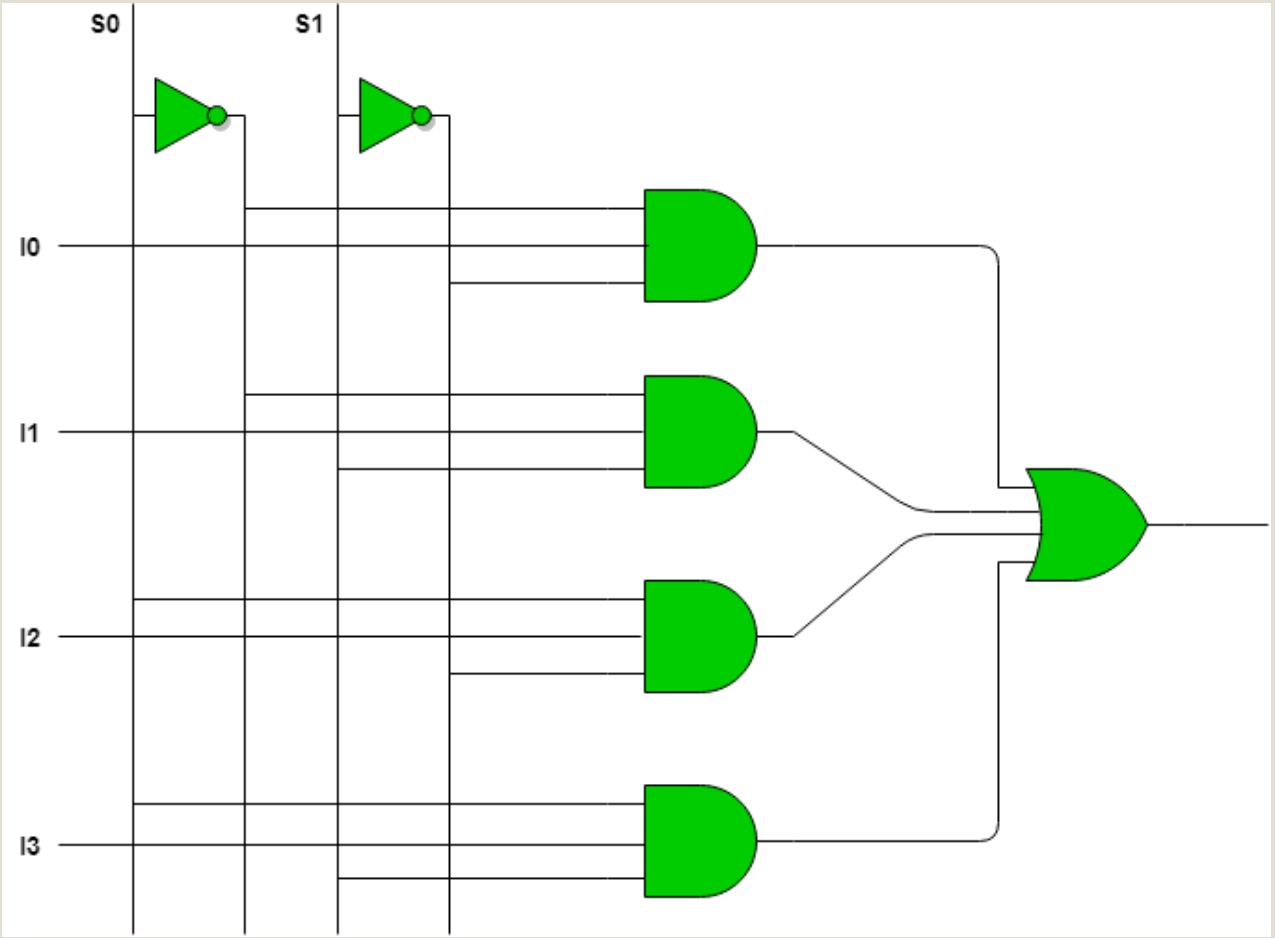
Truth Table and Circuit Diagram of 4×1 Multiplexer

Truth Table

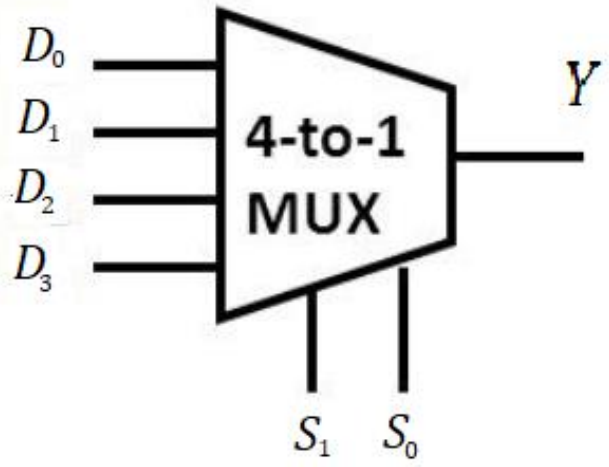
S0	S1	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

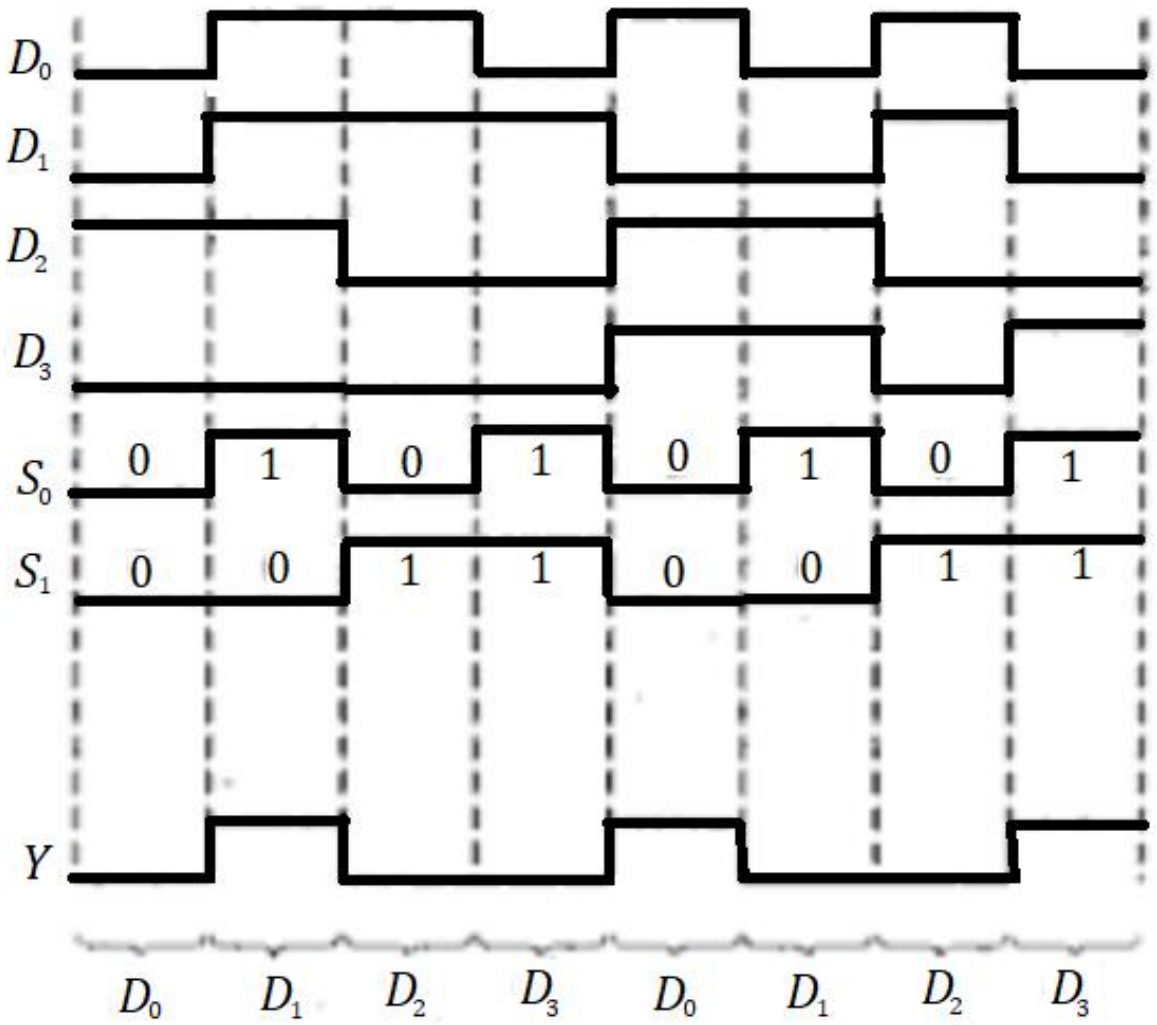
So, final equation,

$$Y = S0'.S1'.I0 + S0'.S1.I1 + S0.S1'.I2 + S0.S1.I3$$



4-to-1 Multiplexer





$S_1S_0 = 00 : Y = D_0$

$S_1S_0 = 01 : Y = D_1$

$S_1S_0 = 10 : Y = D_2$

$S_1S_0 = 11 : Y = D_3$

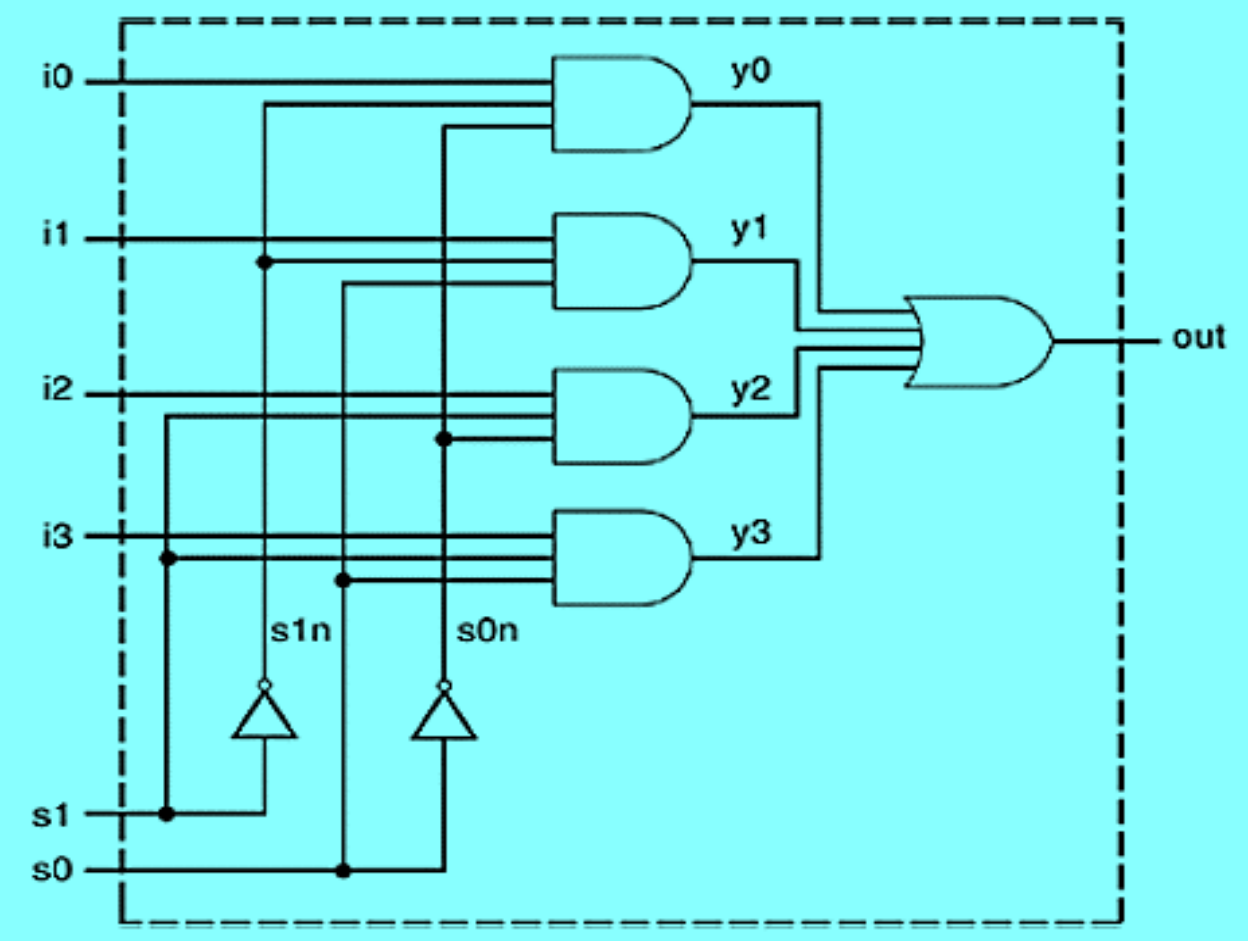
Design of Gate-level Digital Circuits

- The gate-level modeling is virtually the lowest abstract level of modeling.
- This style of modeling will include primitive gates that are predefined in Verilog HDL.
- The designer should know the basic logic circuit and the logic gates that are employed in that circuit for a particular system.
- There is another abstraction layer below gate-level: switch level modeling, which deals with the transistor technologies.

Design of Gate-level Digital Circuits- 4-to-1 Multiplexer

- The logic diagram for the multiplexer using basic logic gates :

- The logic diagram has a one-to-one correspondence with the Verilog description.
- Two intermediate nets, s0n and s1n, are created; they are complements of input signals **s1 and s0**.
- Internal nets y0, y1, y2, y3 are also required.
- Note that instance names are not specified for primitive gates, not, and, and or.
- Instance names are optional for Verilog primitives but are mandatory for instances of user-defined modules.



Verilog Description of 4:1Multiplexer

```
// Module 4-to-1 multiplexer. Port
list //is taken exactly from the I/O
diagram.
module mux4_to_1 (out, i0, i1, i2,
i3, s1, s0);
// Port declarations from the I/O
diagram
output out;
input i0, i1, i2, i3;
input s1, s0;
// Internal wire declarations
wire s1n, s0n;
wire y0, y1, y2, y3;
```

```
// Gate instantiations
// Create s1n and s0n signals.
not (s1n, s1);
not (s0n, s0);
// 3-input and gates instantiated
and (y0, i0, s1n, s0n);
and (y1, i1, s1n, s0);
and (y2, i2, s1, s0n);
and (y3, i3, s1, s0);
// 4-input or gate instantiated
or (out, y0, y1, y2, y3);
endmodule
```

Verilog Description of Stimulus for Multiplexer

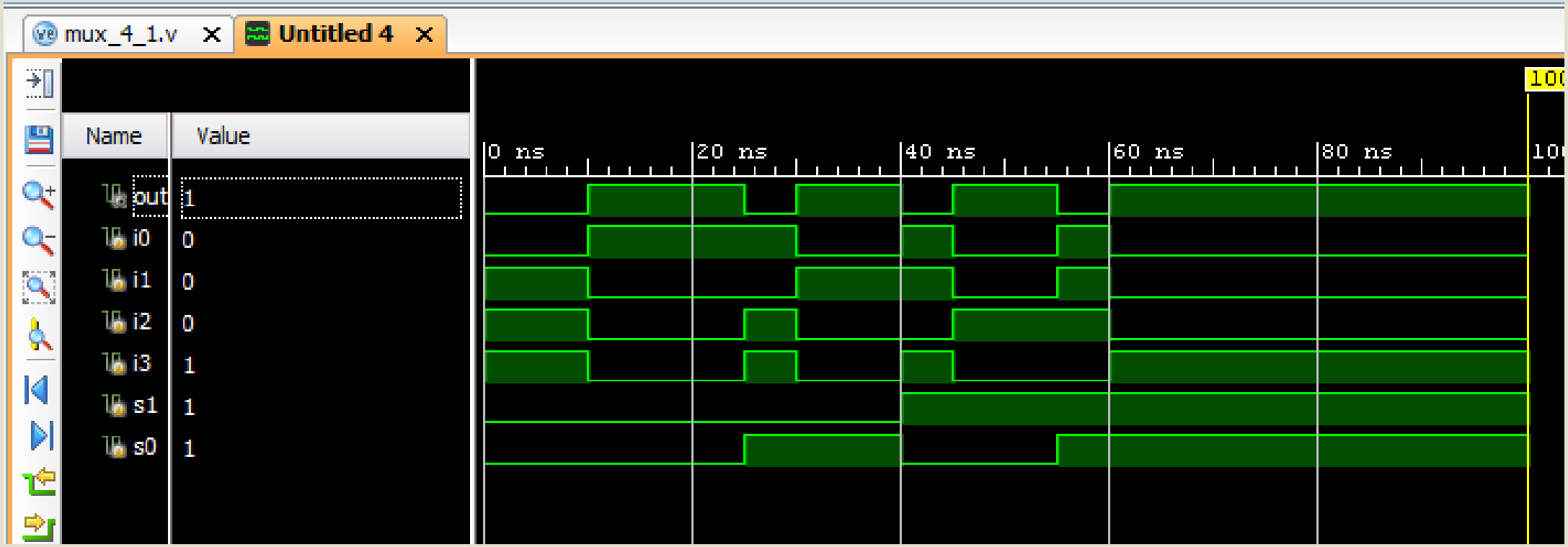
```
// Define the stimulus module (no ports)
module stimulus;
// Declare variables to be connected
// to inputs
reg IN0, IN1, IN2, IN3;
reg S1, S0;
// Declare output wire
wire OUTPUT;
// Instantiate the multiplexer
mux4_to_1 mymux(OUTPUT, IN0, IN1, IN2, IN3, S1, S0);
// Stimulate the inputs
// Define the stimulus module (no ports)
initial
begin
IN0 = 1; IN1 = 0; IN2 = 1; IN3 = 0; // set input lines
#1 $display("IN0= %b, IN1= %b, IN2= %b, IN3= %b\n",IN0,IN1,IN2,IN3);
S1 = 0; S0 = 0; // choose IN0
#1 $display("S1 = %b, S0 = %b, OUTPUT = %b \n", S1, S0, OUTPUT);
S1 = 0; S0 = 1; // choose IN1
#1 $display("S1 = %b, S0 = %b, OUTPUT = %b \n", S1, S0, OUTPUT);
S1 = 1; S0 = 0; // choose IN2
#1 $display("S1 = %b, S0 = %b, OUTPUT = %b \n", S1, S0, OUTPUT);
S1 = 1; S0 = 1; // choose IN3
#1 $display("S1 = %b, S0 = %b, OUTPUT = %b \n", S1, S0, OUTPUT);
end
endmodule
```

Output

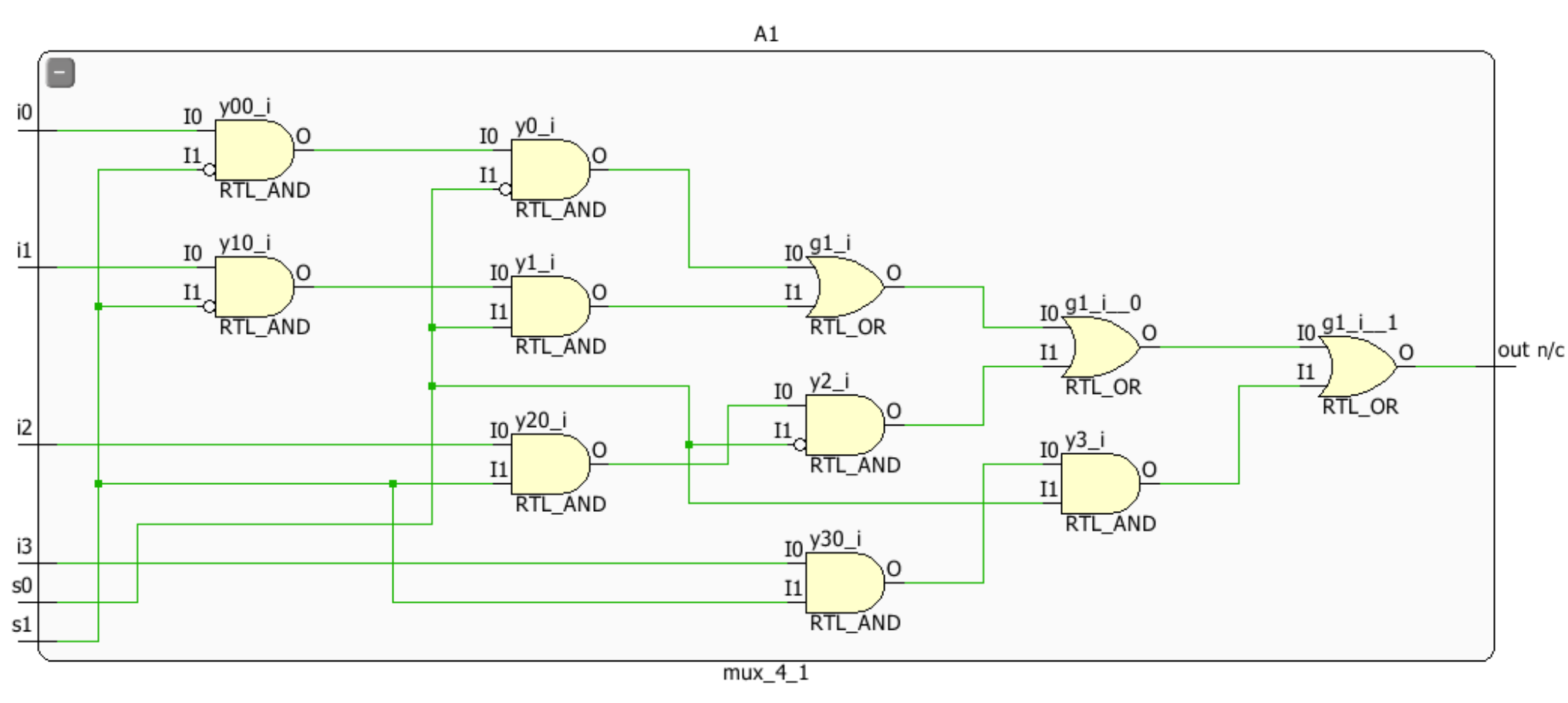
The output of the simulation is shown below. Each combination of the select signals is tested.

```
IN0= 1, IN1= 0, IN2= 1, IN3= 0

S1 = 0, S0 = 0, OUTPUT = 1

S1 = 0, S0 = 1, OUTPUT = 0

S1 = 1, S0 = 0, OUTPUT = 1

S1 = 1, S0 = 1, OUTPUT = 0
```

The output of the simulation is shown below. Each combination of the select signals is tested.

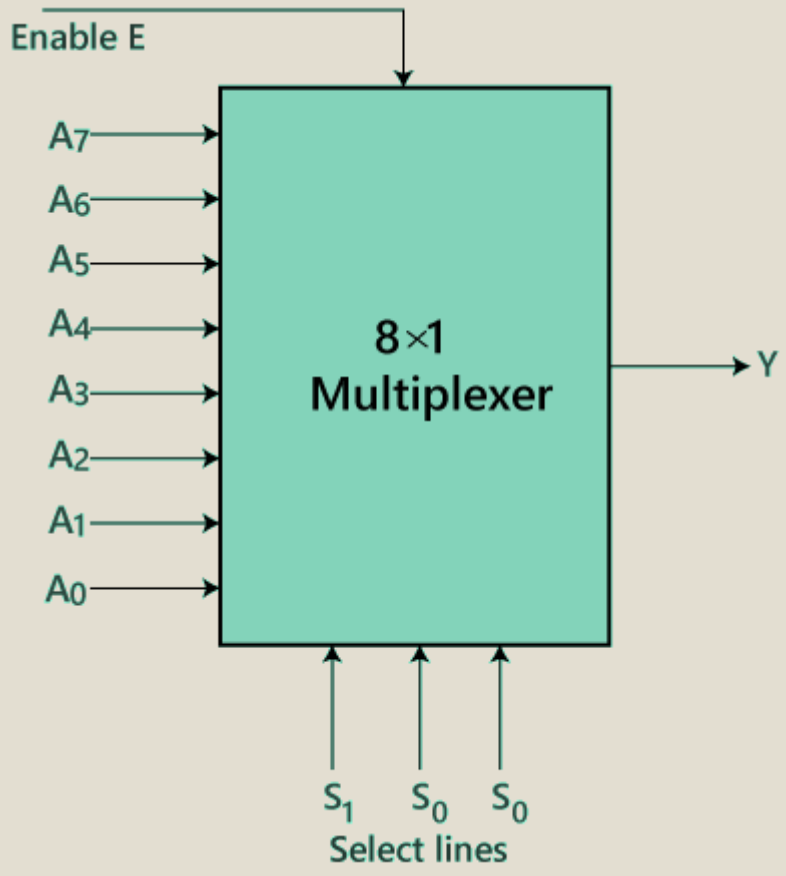


Schematic


A1
i0
i1
i2
i3
s0
s1
y00_i
y10_i
y0_i
y1_i
y20_i
y30_i
y2_i
y3_i
g1_i
g1_i__0
g1_i__1
I0
I1
O
RTL_AND
RTL_OR
out n/c
mux_4_1

Design of Gate-level Digital Circuits-8×1 multiplexer

An **8-to-1 multiplexer** is a digital device that selects **one** of the **eight inputs lines** to the output line by using **three-bit selection line**.



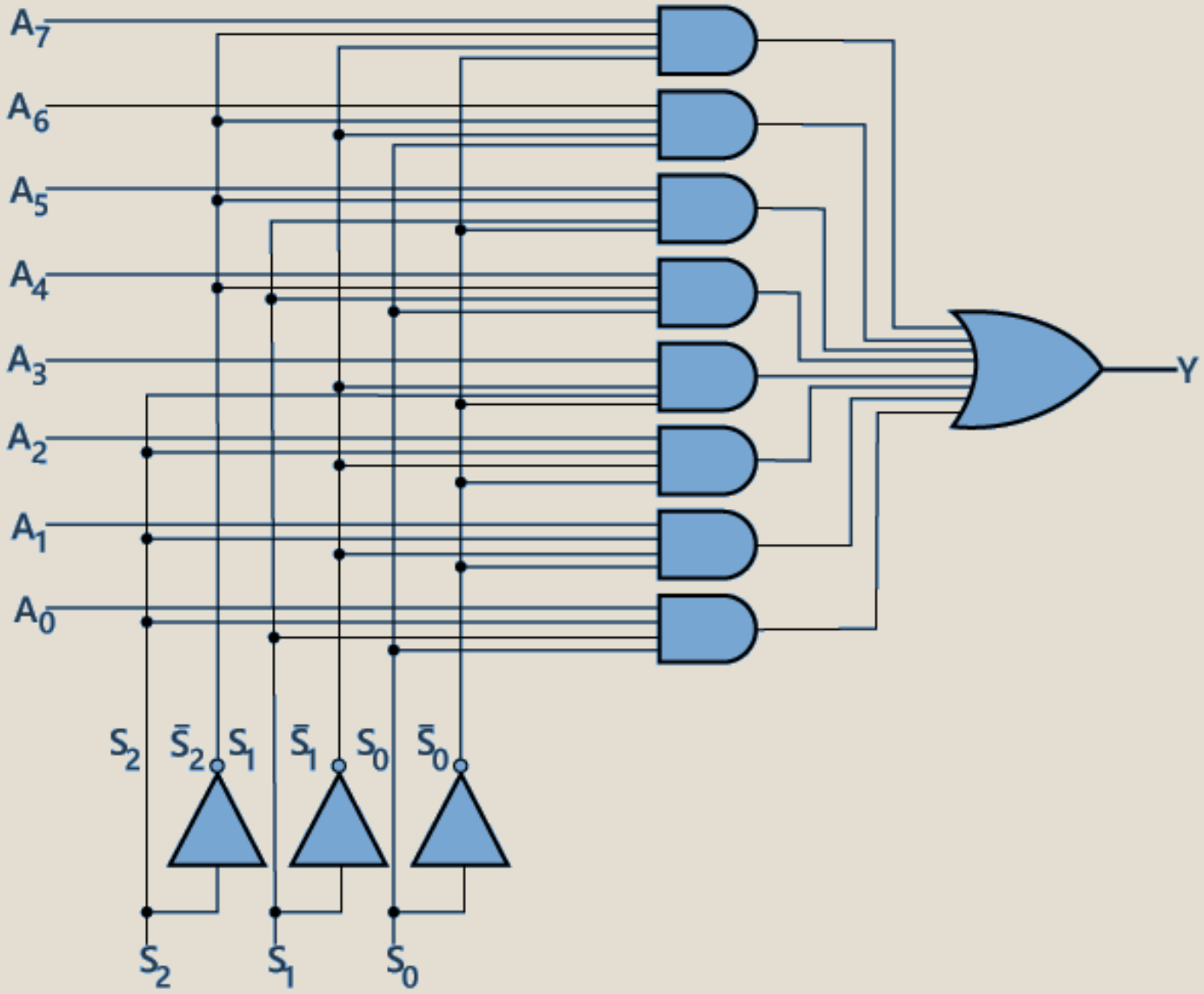
INPUTS			Output
S_2	S_1	S_0	Y
0	0	0	A_0
0	0	1	A_1
0	1	0	A_2
0	1	1	A_3
1	0	0	A_4
1	0	1	A_5
1	1	0	A_6
1	1	1	A_7

The logical expression of the term Y is as follows:

Y=S0'.S1'.S2'.A0+S0.S1'.S2'.A1+S0'.S1.S2'.A2+S0.S1.S2'.A3+S0'.S1'.S2 A4+S0.S1'.S2 A5+ S0'.S1.S2.A6+S0.S1.S3.A7

Design of Gate-level Digital Circuits-8×1 multiplexer

- Logic diagram for 8×1 MUX:
- The input signals are A0, A1, A2, A3, A4, A5, A6, A7, S0, S1, S2 and the output signal is **Y.**



Verilog Description of 8:1Multiplexer

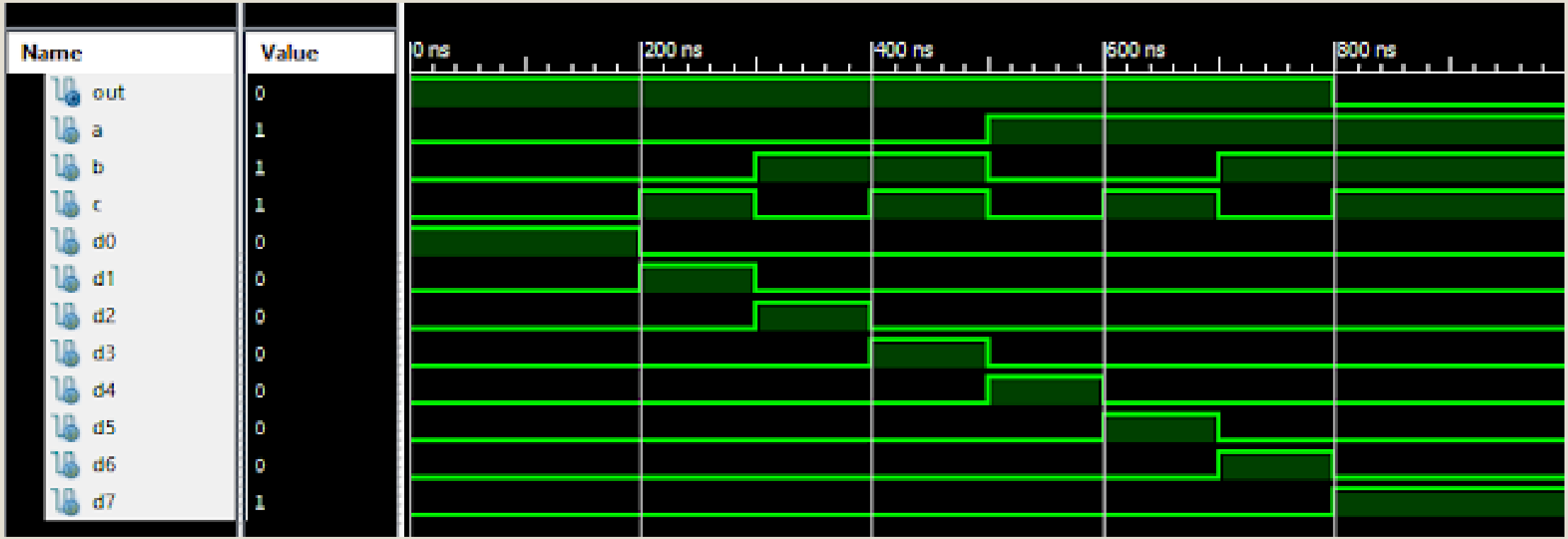
```
`timescale 1ns/1ps
module m81(input A0, A1, A2, A3, A4, A5, A6, A7, S0, S1, S2, output Y);
wire T1, T2, T3, T4, T5, T6, T7, T8, T9, T10, T11;
not(T1, S0);
not(T2, S1);
not(T3, S2);
and(T4, A0, T1, T2, T3), (T5, A1, S0, T2, T3);
and(T6, A2, T1, S1, T3), (T7, A3, S0, S1, T3);
and(T8, A4, T1, T2, S2), (T9, A5, S0, T2, S2);
and(T10, A6, T1, S1, S2), (T11, A7, S0, S1, S2);
or(out, T4, T5, T6, T7, T8, T9, T10, T11);
endmodule
```

If there exist more than two same gates, we can concatenate the expression into one single statement !
Eg: not(T1, S0), (T2, S1), (T3, S2);

Verilog Description of 8:1Multiplexer- TB

```
initial begin   // Initialize Inputs
S0 = 0;S1 = 0;S2 = 0;A0 = 1;A1 = 0;A2 = 0; A3 = 0;A4 = 0;A5 = 0;A6 = 0;A7 = 0;
// Wait 100 ns for global reset to finish
#100; // Add stimulus here
#100; S0 = 0;S1 = 0;S2 = 1;A0 = 0;A1 = 1;A2 = 0;A3 = 0;A4 = 0;A5 = 0;A6 = 0;A7 = 0;
#100; S0 = 0;S1 = 1;S2 = 0;A0 = 0;A1 = 1;A2 = 0;A3 = 0;A4 = 0;A5 = 0;A6 = 0;A7 = 0;
#100; S0 = 0;S1 = 1;S2 = 1;A0 = 0;A1 = 1;A2 = 0;A3 = 0;A4 = 0;A5 = 0;A6 = 0;A7 = 0;
#100; S0 = 1;S1 = 0;S2 = 0;A0 = 0;A1 = 1;A2 = 0;A3 = 0;A4 = 0;A5 = 0;A6 = 0;A7 = 0;
#100; S0 = 1;S1 = 0;S2 = 1;A0 = 0;A1 = 1;A2 = 0;A3 = 0;A4 = 0;A5 = 0;A6 = 0;A7 = 0;
#100; S0 = 1;S1 = 1;S2 = 0;A0 = 0;A1 = 1;A2 = 0;A3 = 0;A4 = 0;A5 = 0;A6 = 0;A7 = 0;
#100; S0 = 1;S1 = 1;S2 = 1;A0 = 0;A1 = 1;A2 = 0;A3 = 0;A4 = 0;A5 = 0;A6 = 0;A7 = 0;
end
endmodule
```

Simulation Output



Design Methodology - example

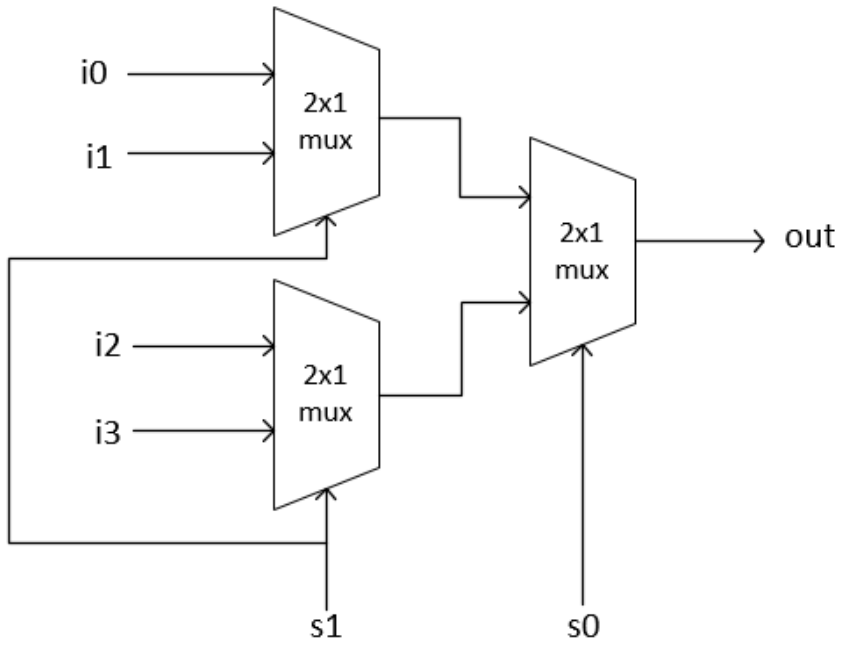
- **Top-Down Design Approach**

  1. **8x1 Multiplexer**: The main module will have eight inputs (D0 to D7), three selection lines (S2, S1, S0), and one output (Y).
  2. **4x1 Multiplexer**: This will be used as an intermediate block to select between four inputs, controlled by two selection lines (S1, S0).
  3. **2x1 Multiplexer**: This will serve as the base building block. Each 2x1 multiplexer will have two inputs, one select line, and one output.

- **Bottom up Design Approach**

  1. **Step 1**: Start with the smallest module, the 2x1 multiplexer.
  2. **Step 2**: Use multiple 2x1 multiplexers to build a 4x1 multiplexer.
  3. **Step 3**: Use the 4x1 multiplexers to construct the 8x1 multiplexer.

4 : 1 MUX using 2 : 1 MUX



i0	i1	i2	i3	s0	s1	out
1	0	1	1	0	1	0
0	1	0	0	0	1	1
0	0	1	0	1	0	1
0	0	0	1	1	1	1
1	0	0	0	0	0	1

Verilog Code - **Bottom up Design Approach**

- **Verilog Code for 2×1 Mux**

```
module mux2x1(out,a,b,s);
input a,b,s;
wire and_1,and_2,s_c;
output out;
not (s_c,s);
and (and_1,a,s_c);
and (and_2,b,s);
or (out,and_1,and_2);
endmodule
```

- **Verilog Code for 4×1 Mux**

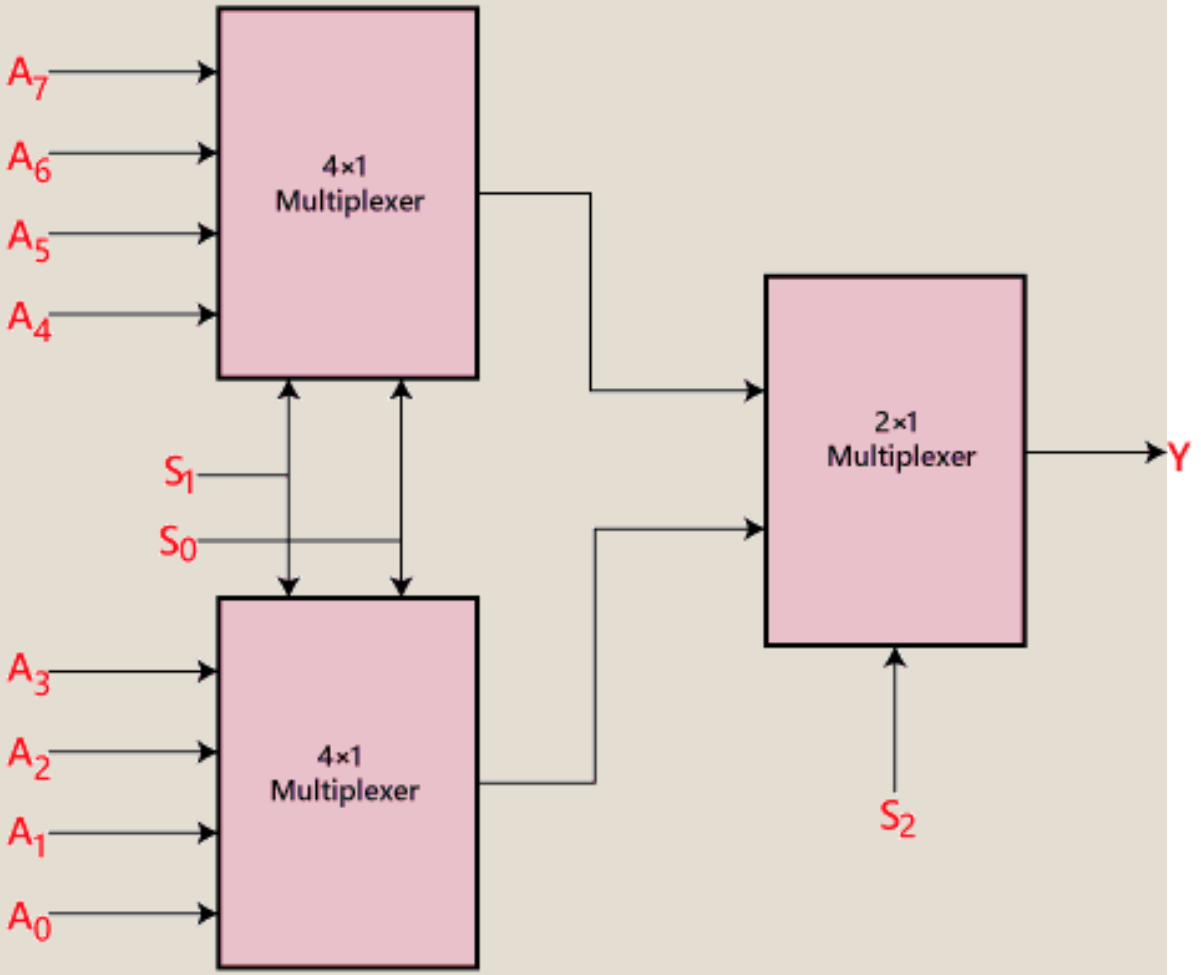
```
module mux4x2(out,i0,i1,i2,i3,s1,s0);
input i0,i1,i2,i3,s1,s0;
output out;
wire mux1,mux2;
mux2x1 mux_1(mux1,i0,i1,s1);
mux2x1 mux_2(mux2,i2,i3,s1);
mux2x1 mux_3(out,mux1,mux2,s0);
endmodule
```

Test Bench code for 4×1 Mux

```
module mux4x2_tb;
wire t_out;
reg t_a, t_b, t_c, t_d, t_s1, t_s0;
mux4x2 my_4x2_mux( .i0(t_a), .i1(t_b), .i2(t_c), .i3(t_d),
.s1(t_s1), .s0(t_s0), .out(t_out) );
initial
begin
// 1
t_a = 1'b1;t_b = 1'b0;t_c = 1'b1;t_d = 1’b1;
t_s0 = 1'b0;t_s1 = 1'b1;
#5 //2
t_a = 1'b0;t_b = 1'b1;t_c = 1'b0;t_d = 1’b0;
t_s0 = 1'b0;t_s1 = 1'b1;
#5 //3
t_a = 1'b0;t_b = 1'b0;t_c = 1'b1;t_d = 1’b0;
t_s0 = 1'b1;t_s1 = 1'b0;
#5 //4
t_a = 1'b0;t_b = 1'b0;t_c = 1'b0;t_d = 1’b1;
t_s0 = 1'b1;t_s1 = 1'b1;
#5 //5
t_a = 1'b1;t_b = 1'b0;t_c = 1'b0;t_d = 1’b0;
t_s0 = 1'b0;t_s1 = 1'b0;
end
endmodule
```

8 : 1 MUX using 4 : 1 MUX

We can implement the 8×1 multiplexer using a lower order multiplexer.
To implement the 8×1 multiplexer, we need two 4×1 multiplexers and one 2×1 multiplexer. The 4×1 multiplexer has 2 selection lines, 4 inputs, and 1 output. The 2×1 multiplexer has only 1 selection line.
For getting 8 data inputs, we need two 4×1 multiplexers. The 4×1 multiplexer produces one output. So, in order to get the final output, we need a 2×1 multiplexer.



Verilog Code – **Top-Down Design Approach**

```
module mux_8_1(i0, i1, i2, i3, i4, i5, i6, i7, s0, s1, s2, y);
 input i0, i1, i2, i3, i4, i5, i6, i7, s0, s1, s2;
 output y;
wire y1, y2;
// Instantiate two 4:1 multiplexers for lower and upper 4 bits
mux_4_1 mux_lower (i0, i1, i2, i3, s0, s1, y1);
mux_4_1 mux_upper (i4, i5, i6, i7, s0, s1, y2);
// Use a 2:1 MUX to choose between y1 and y2
mux_2_1 mux_final (y1, y2, s2, y);

endmodule
```

```
module mux_4_1(i0, i1, i2, i3, s0, s1, y);
    input i0, i1, i2, i3, s0, s1;
    output y;

    wire not_s0, not_s1;
    wire and0, and1, and2, and3;
    not (not_s0, s0);
    not (not_s1, s1);
    and (and0, i0, not_s0, not_s1
    and (and1, i1, s0, not_s1);
    and (and2, i2, not_s0, s1);
    and (and3, i3, s0, s1);
    or (y, and0, and1, and2, and3);
endmodule
```

```
module mux_2_1(i0, i1, s0, y);
    input i0, i1, s0;
    output y;

    wire not_s0, and0, and1;

not (not_s0, s0);
and (and0, i0, not_s0);
and (and1, i1, s0);

or (y, and0, and1);
endmodule
```

Verilog Code - behavioural

```
 module
mux_8_1(i0,i1,i2,i3,i4,i5,i6,i7,s0,s1,s2,y)
;
  input i0,i1,i2,i3,i4,i5,i6,i7,s0,s1,s2;
  output reg y;

  wire y1,y2;

      //  always @(*)
        //   begin

         mux_4_1 g1(i0,i1,i2,i3,s0,s1,y1);
         mux_4_1 g2(i4,i5,i6,i7,s0,s1,y2);
         mux_2_1 g3(y1,y2,s2,y);
          // end
       endmodule
```

```
module
mux_4_1(i0,i1,i2,i3,s0,s1,y);
  input i0,i1,i2,i3,s0,s1;
  output reg y;
  always @(*)
    begin
      case ({s0,s1})

  2'b00: y = i0;
  2'b01: y = i1;
  2'b10: y = i2;
  2'b11: y = i3;
    default: y = 0;

  endcase
    end
    endmodule
```

```
module
mux_2_1(i0,i1,s0,y);
      input i0,i1,s0;
      output reg y;
      always @(*)
        begin
          case (s0)

        1'b0:y=i0;
        1'b1:y=i1;
        default:y=0;

      endcase
        end
        endmodule
```

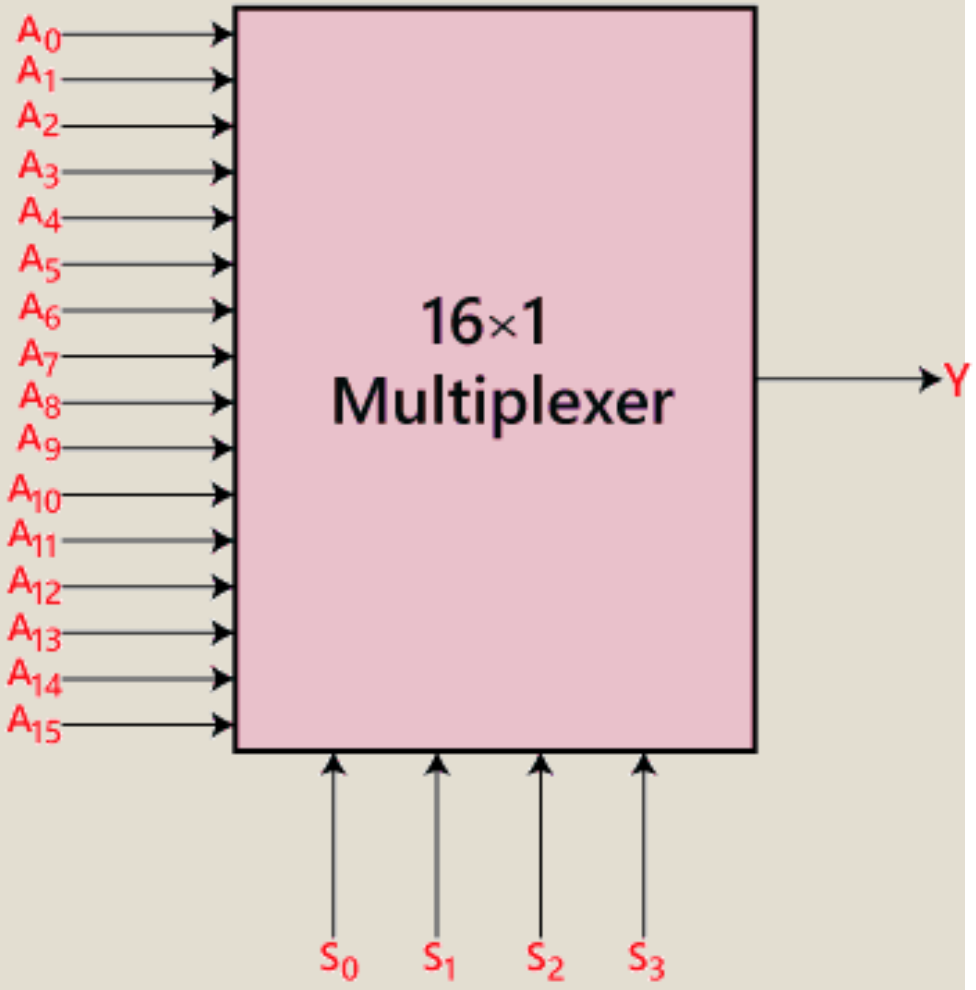
16 x 1 Multiplexer

The logical expression of the term Y is as follows:

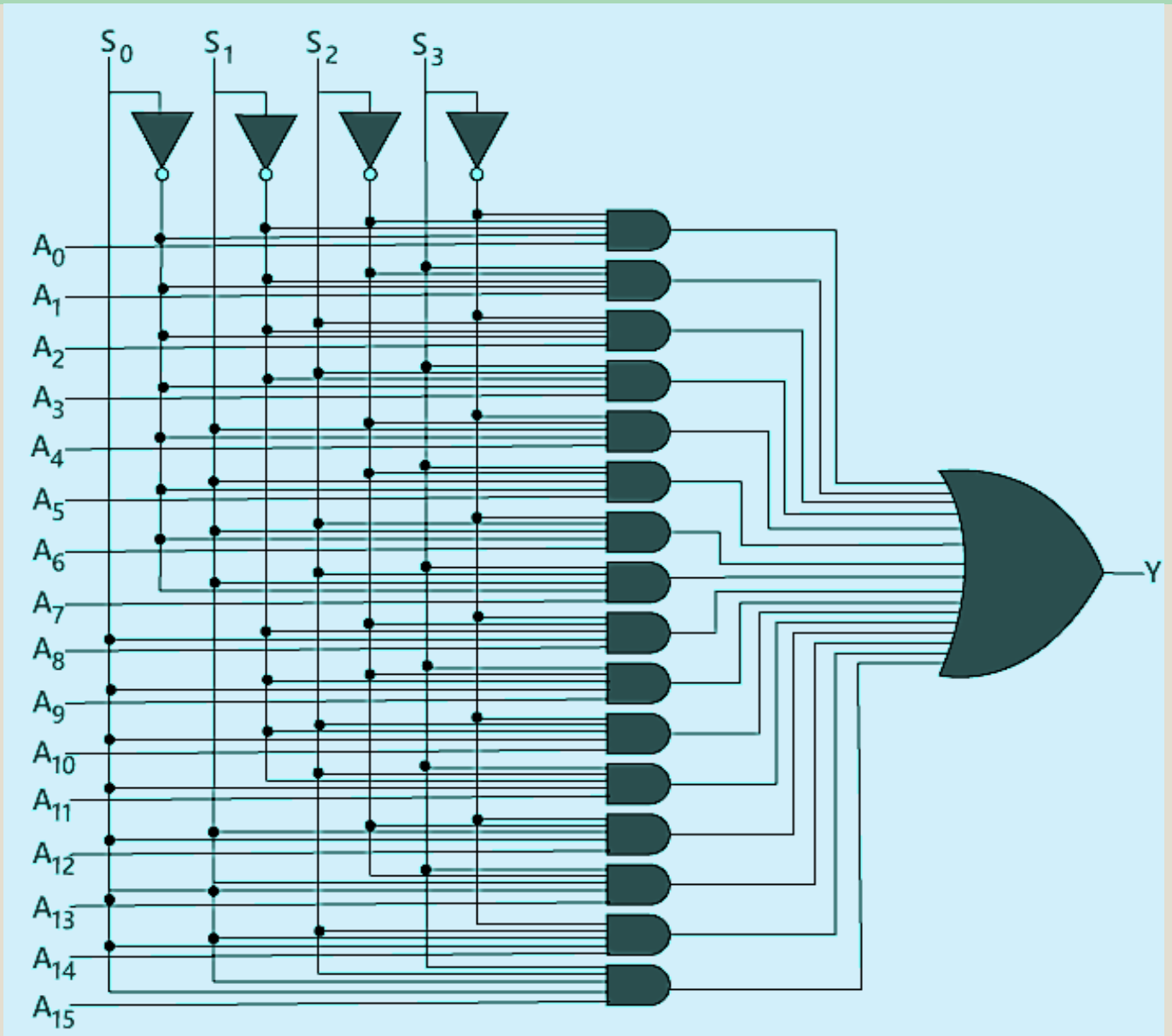
$$Y=A_0.S_0'.S_1'.S_2'.S_3'+A_1.S_0'.S_1'.S_2'.S_3+A_2.S_0'.S_1'.S_2.S_3'+A_3.S_0'.S_1'.S_2.S_3+A_4.S_0'.S_1.S_2'.S_3'+A_5.S_0'.S_1.S_2'.S_3+A_6.S_1.S_2.S_3'+A_7.S_0'.S_1.S_2.S_3+A_8.S_0.S_1'.S_2'.S_3'+A_9.S_0.S_1'.S_2'.S_3+Y_10.S_0.S_1'.S_2.S_3'+A_12S_0.S_1.S_2'.S_3'+A_13.S_0.S_1.S_2'.S_3+A_14.S_0.S_1.S_2.S_3'+A_15.S_0.S_1.S_2'.S_3$$

Truth Table:

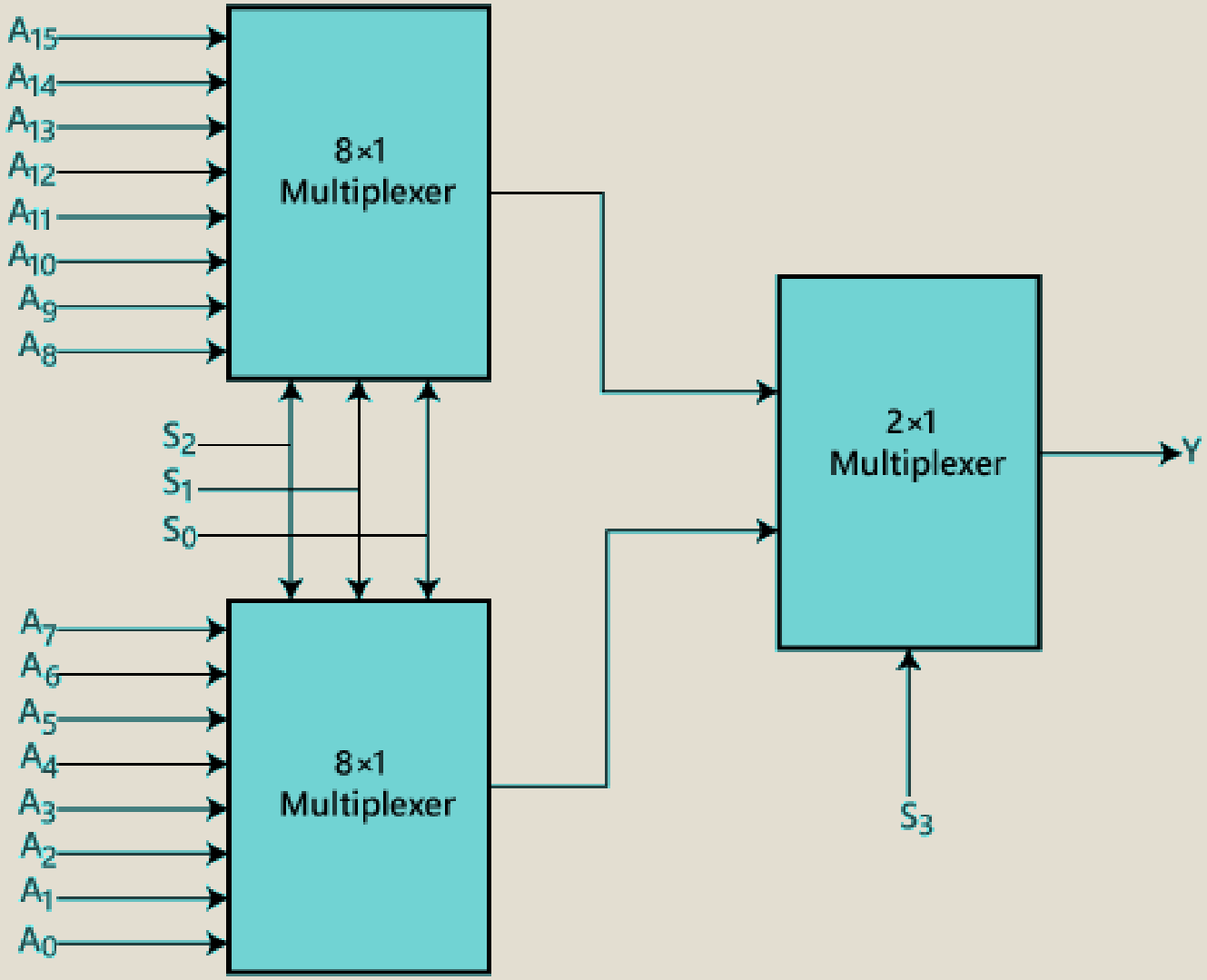
INPUTS				Output
S_0	S_1	S_2	S_3	Y
0	0	0	0	A_0
0	0	0	1	A_1
0	0	1	0	A_2
0	0	1	1	A_3
0	1	0	0	A_4
0	1	0	1	A_5
0	1	1	0	A_6
0	1	1	1	A_7
1	0	0	0	A_8
1	0	0	1	A_9
1	0	1	0	A_{10}
1	0	1	1	A_{11}
1	1	0	0	A_{12}
1	1	0	1	A_{13}
1	1	1	0	A_{14}
1	1	1	1	A_{15}



Logical circuit of the above expression is given below:



16×1 multiplexer using 8×1 and 2×1 multiplexer



// Verilog code - Gate-level modeling of 8:1 multiplexer

```
module mux_8_1(i0, i1, i2, i3, i4, i5, i6, i7, s0, s1, s2, y);
    input i0, i1, i2, i3, i4, i5, i6, i7, s0, s1, s2;
    output y;
    wire not_s0, not_s1, not_s2;
    wire and0, and1, and2, and3, and4, and5, and6, and7;
    // Inverting select lines
    not (not_s0, s0);
    not (not_s1, s1);
    not (not_s2, s2);
    // AND gates to select the correct input based on select lines
    and (and0, i0, not_s0, not_s1, not_s2); // when s2s1s0 = 000, select i0
    and (and1, i1, s0, not_s1, not_s2);     // when s2s1s0 = 001, select i1
    and (and2, i2, not_s0, s1, not_s2);     // when s2s1s0 = 010, select i2
    and (and3, i3, s0, s1, not_s2);         // when s2s1s0 = 011, select i3
    and (and4, i4, not_s0, not_s1, s2);     // when s2s1s0 = 100, select i4
    and (and5, i5, s0, not_s1, s2);         // when s2s1s0 = 101, select i5
    and (and6, i6, not_s0, s1, s2);         // when s2s1s0 = 110, select i6
    and (and7, i7, s0, s1, s2);             // when s2s1s0 = 111, select i7
    // OR gate to combine the results
    or (y, and0, and1, and2, and3, and4, and5, and6, and7);
endmodule
```

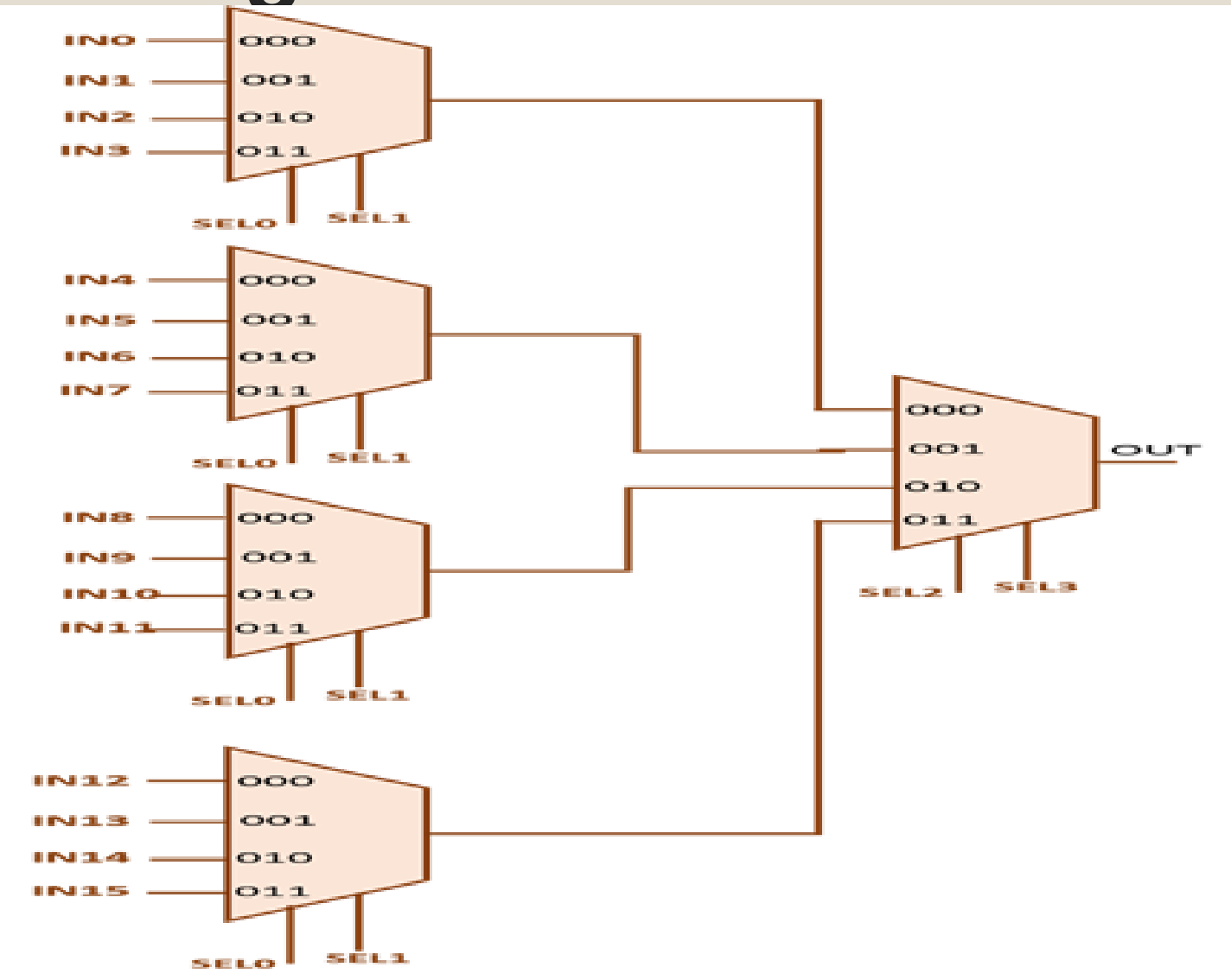
Verilog Code – Gate Level

```
// Gate-level modeling of 2:1 multiplexer
module mux_2_1(i0, i1, s0, y);
    input i0, i1, s0;
    output y;

    wire not_s0, and0, and1;

    not (not_s0, s0);

    and (and0, i0, not_s0);
// when s0 = 0, select i0
    and (and1, i1, s0);
 // when s0 = 1, select i1

    // OR gate to produce final output
    or (y, and0, and1);
endmodule
```

```
// Top-level module: 16:1 multiplexer using two 8:1 multiplexers and one 2:1 multiplexer
module mux_16_1( input i0, i1, i2, i3, i4, i5, i6, i7,
    input i8, i9, i10, i11, i12, i13, i14, i15,
    input s0, s1, s2, s3, output y);

    wire y1, y2;  // Outputs of the two 8:1 multiplexers

    // Instantiate two 8:1 multiplexers

mux_8_1 mux1 (i0, i1, i2, i3, i4, i5, i6, i7, s0, s1, s2, y1);

mux_8_1 mux2 (i8, i9, i10, i11, i12, i13, i14, i15, s0, s1, s2, y2);

// Instantiate a 2:1 multiplexer to select between the outputs of

the two 8:1 MUXes

    mux_2_1 mux3 (y1, y2, s3, y);

endmodule
```

16 x 1 Multiplexer using 4 x 1 MUX

- You need a combinational logic with 16 input pins, 4 select lines, and one output.
- In a 4:1 mux, you have 4 input pins, two select lines, and one output.
- At least you have to use 4 4:1 MUX, to obtain 16 input lines.
- But you then have a logic with 4 output pins.
- We can use another 4:1 MUX, to multiplex only one of those 4 outputs at a time.



Verilog Code – Gate Level

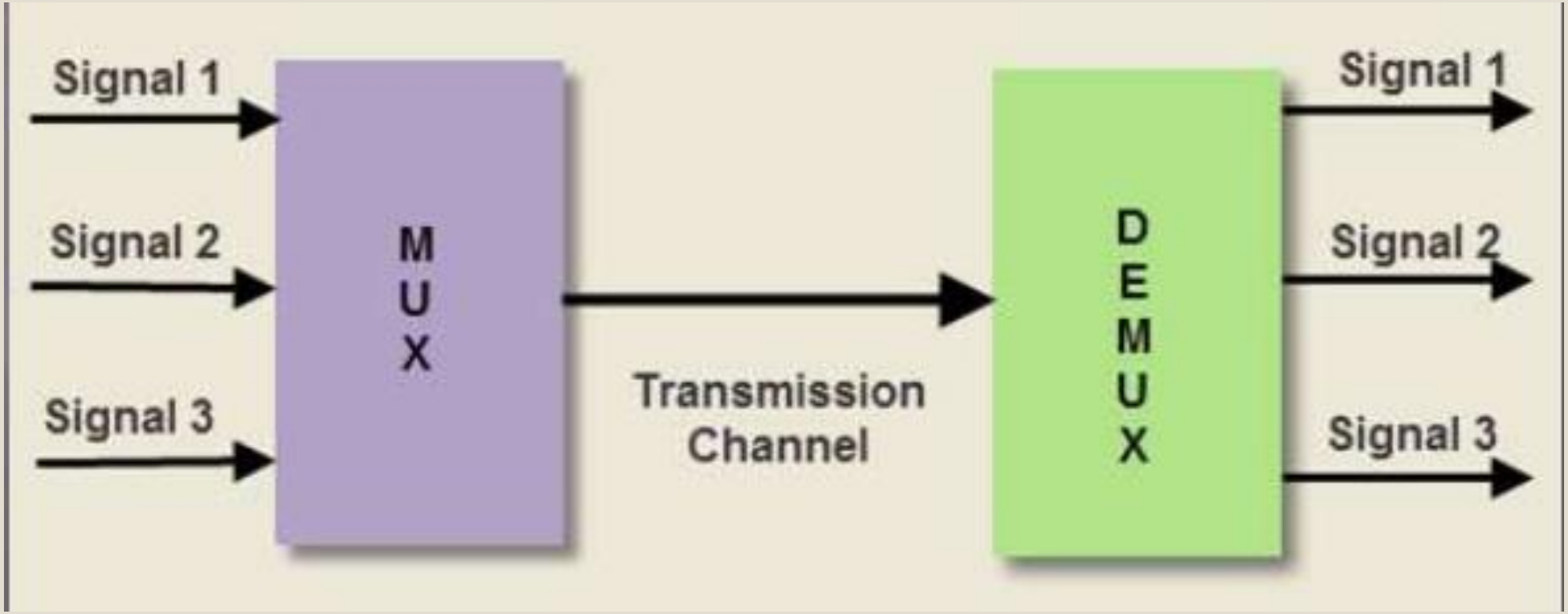
```
module mux4to1_gate(out,in,sel);

  input [0:3] in;
  input [0:1] sel;
  output out;
  wire a,b,c,d,n1,n2,a1,a2,a3,a4;
  not n(n1,sel[1]);
  not nn(n2,sel[0]);

  and (a1,in[0],n1,n2);
  and (a2,in[1],n2,sel[1]);
  and (a3,in[2],sel[0],n1);
  and (a4,in[3],sel[0],sel[1]);

  or or1(out,a1,a2,a3,a4);

 endmodule
```

```
module mux16to1(out,in,sel);

 input [0:15] in;
 input [0:3] sel;
 output out;
 wire [0:3] ma;

 mux4to1_gate mux1(ma[0],in[0:3],sel[2:3]);

 mux4to1_gate mux2(ma[1],in[4:7],sel[2:3]);

 mux4to1_gate mux3(ma[2],in[8:11],sel[2:3]);

 mux4to1_gate mux4(ma[3],in[12:15],sel[2:3]);

 mux4to1_gate mux5(out,ma,sel[0:1]);
endmodule
```

Test Bench File for 16:1 MUX

```
module testmux_16;
reg [0:15] in;
reg [0:3] sel;
wire out;
mux16to1 mux(out,in,sel);
initial
begin
$monitor("in=%b | sel=%b | out=%b",
in,sel,out);
end
initial
begin
in=16'b1000000000000000; sel=4'b0000;
#30 in=16'b0100000000000000; sel=4'b0001;
#30 in=16'b0010000000000000; sel=4'b0010;
#30 in=16'b0001000000000000; sel=4'b0011;
#30 in=16'b0000100000000000; sel=4'b0100;
#30 in=16'b0000010000000000; sel=4'b0101;
#30 in=16'b0000001000000000; sel=4'b0110;
#30 in=16'b0000000100000000; sel=4'b0111;
#30 in=16'b0000000010000000; sel=4'b1000;
#30 in=16'b0000000001000000; sel=4'b1001;
#30 in=16'b0000000000100000; sel=4'b1010;
#30 in=16'b0000000000010000; sel=4'b1011;
#30 in=16'b0000000000001000; sel=4'b1100;
#30 in=16'b0000000000000100; sel=4'b1101;
#30 in=16'b0000000000000010; sel=4'b1110;
#30 in=16'b0000000000000001; sel=4'b1111;
end
endmodule
```

De-multiplexer

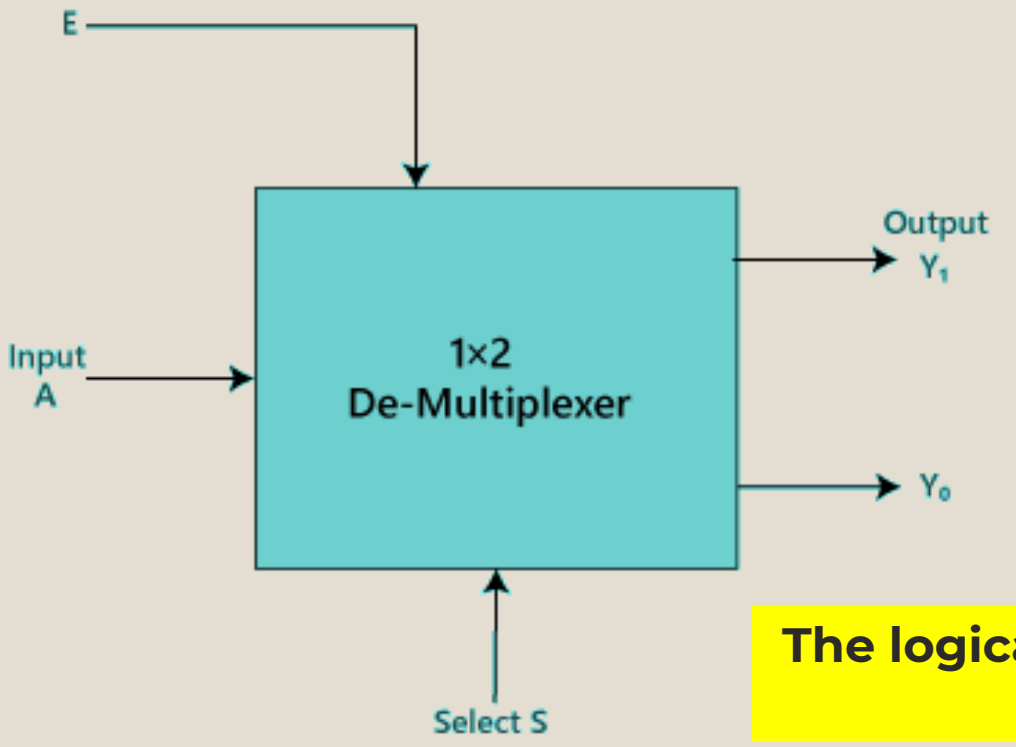


De-multiplexer

- A De-multiplexer is a combinational circuit that has only 1 input line and 2N output lines.
- Simply, the multiplexer is a single-output and multi-input combinational circuit.
- De-multiplexer is opposite to the multiplexer.
- The information is received from the single input lines and directed to the output line. On the basis of the values of the selection lines, the input will be connected to one of these outputs.
- Unlike encoder and decoder, **there are n selection lines and 2^n outputs**.
- So, there is a total of 2^n possible combinations of inputs.
- There are various types of De-multiplexer which are as follows: 1×2, 1×4, 1×8..etc

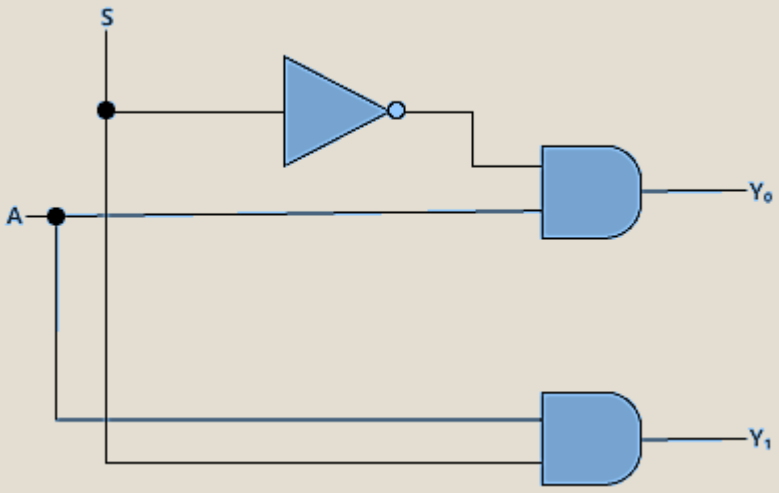
1×2 De-multiplexer:

- In the 1 to 2 De-multiplexer, there are only two outputs, i.e., Y0, and Y1, 1 selection lines, i.e., S0, and single input, i.e., A.
- On the basis of the selection value, the input will be connected to one of the outputs. The block diagram and the truth table of the 1×2 multiplexer are given below.



INPUTS	Output	
S_0	Y_1	Y_0
0	0	A
1	A	0

Logical circuit



The logical expression of the term Y is as follows

$Y_0=S_0'.A$ $\quad$ $Y_1=S_0.A$

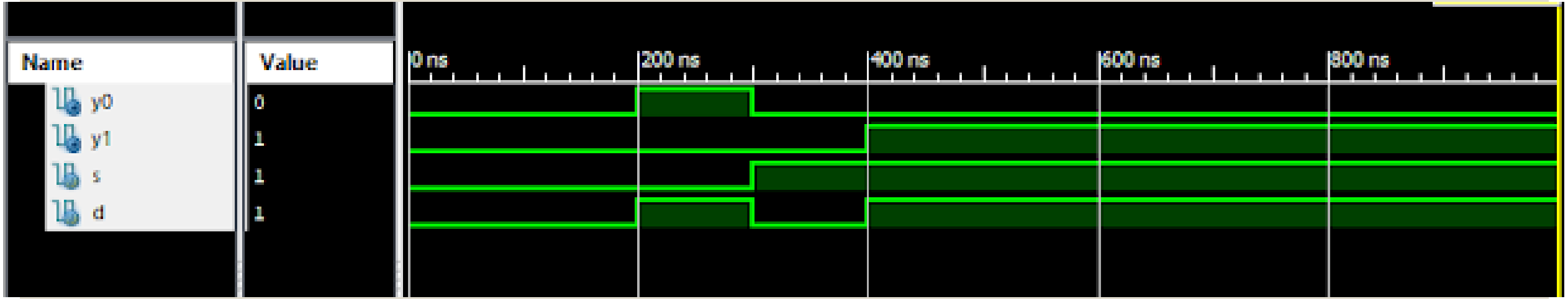
Gate level modelling: 1×2 De-multiplexer:

```
module DEMUX_1_to_2(
    input s,
    input d,
    output y0,
    output y1
    );
not(sn,s);
and(y0,sn,d);
and(y1,s,d);
endmodule
```

//Testbench code for 1-2 DEMUX Structural/Gate Level Modelling

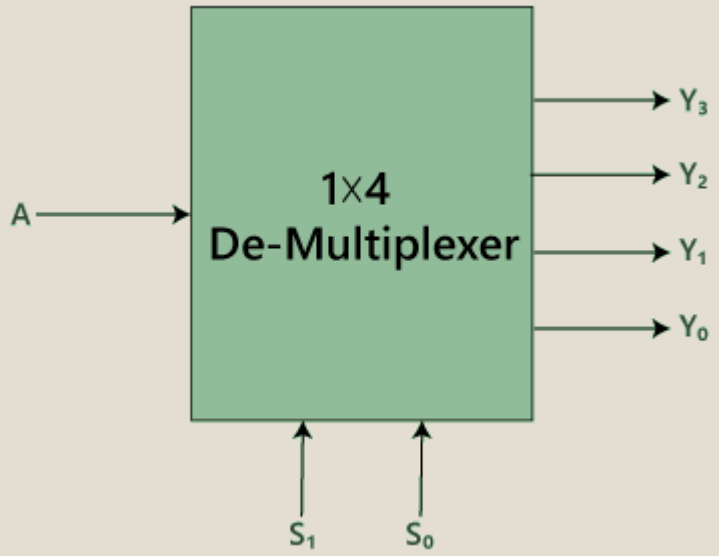
```
initial begin

// Initialize Inputs
        s = 0;
        d = 0;
// Wait 100 ns for global reset to finish
        #100;
// Add stimulus here
#100; s=0;d=1;
#100; s=1;d=0;
#100; s=1;d=1;
end
```

Simulation Output



1×4 De-multiplexer:

- In 1 to 4 De-multiplexer, there are total of four outputs, i.e., Y0, Y1, Y2, and Y3, 2 selection lines, i.e., S0 and S1 and single input, i.e., A.
- On the basis of the combination of inputs which are present at the selection lines S0 and S1, the input be connected to one of the outputs.
- The block diagram and the truth table of the 1×4 multiplexer are given below.



Truth Table:

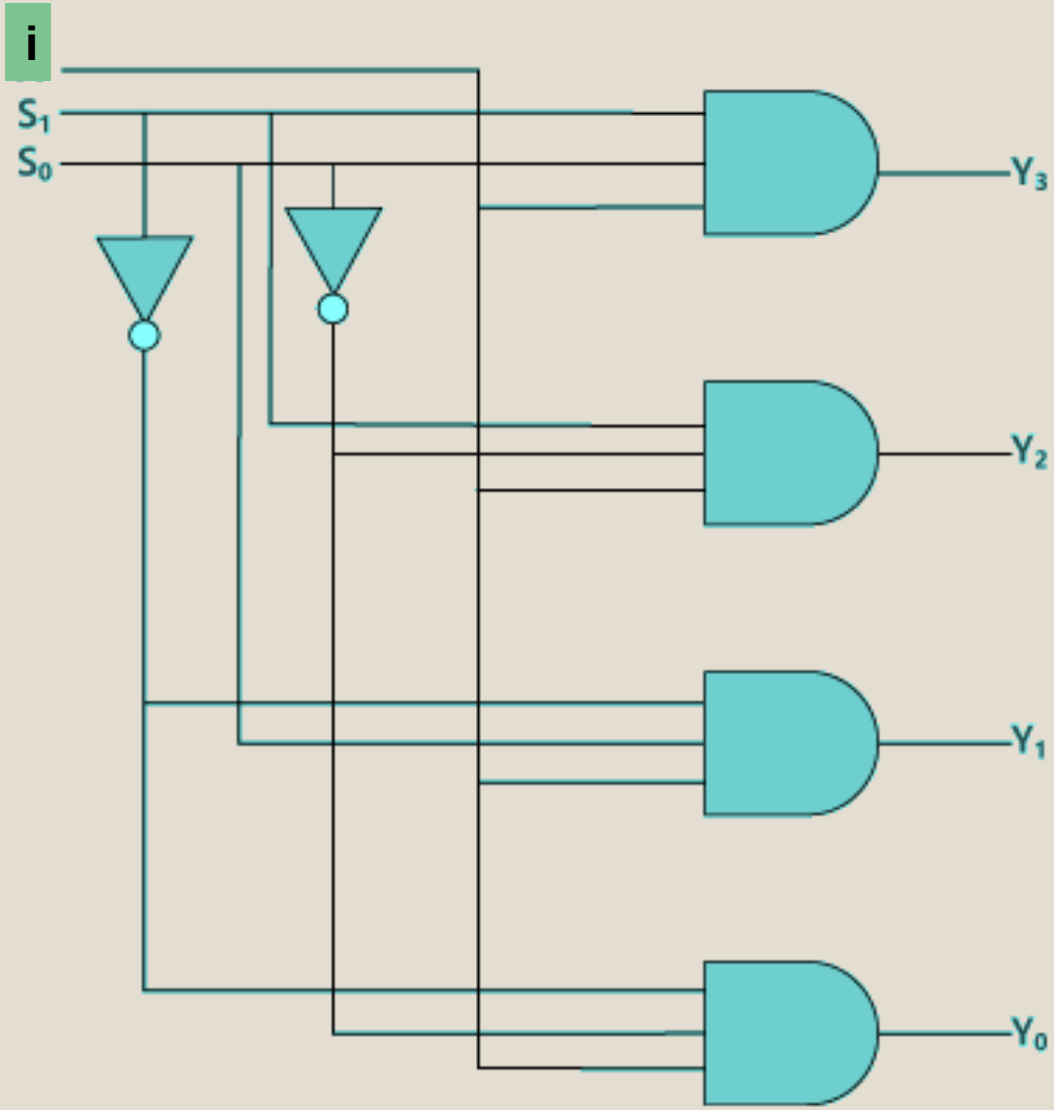
INPUTS		Output			
S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	A
0	1	0	0	A	0
1	0	0	A	0	0
1	1	A	0	0	0

1×4 De-multiplexer:

- The logical expression of the term Y is as follows:

- O0=S1' S0’ i
- O1=S1' S0 i
- O2=S1 S0’ i
- O3=S1 S0 i

Logical circuit



Gate level modelling: 1×4 De-multiplexer:

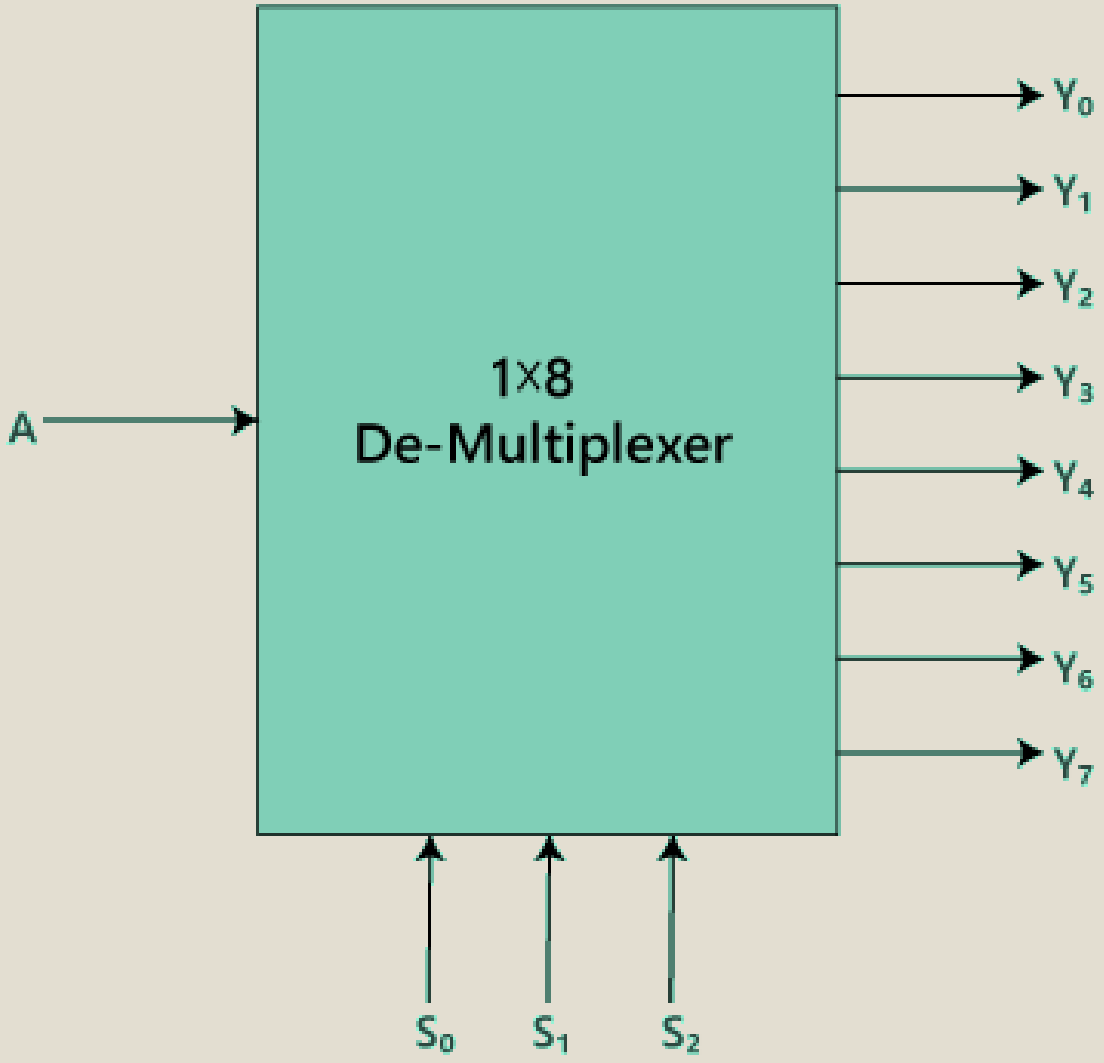
```
module demux ( input i,s1,s0, output o0,o1,o2,o3 );
wire s1n, s0n, o0,o1,o2,o3;
not(s1n,s1);
not(s0n,s0);
and(o0,s1n,s0n,i);
and (o1,s1n,s0,i);
and(o2,s1,s0n,i);
and(o3,s1,s0,i);
endmodule
```

Gate level modelling: 1×4 De-multiplexer-TB

```
module demux_tb;

reg I, s1, s0; // Inputs

wire o0,o1, o2,o3; // Outputs

demux uut (i,s1,s0, o0,o1,o2,o3 );

initial begin    // Testbench logic


    i = 0; s1 = 0; s0 = 0; 10; // Initialize Inputs
    i = 1; s1 = 0; s0 = 0; // Test case 1: s1=0, s0=0, i=1 -> o0 should be 1
   #10;    i = 1; s1 = 0; s0 = 1; // Test case 2: s1=0, s0=1, i=1 -> o1 should be 1
   #10;    i = 1; s1 = 1; s0 = 0; // Test case 3: s1=1, s0=0, i=1 -> o2 should be 1
   #10;    i = 1; s1 = 1; s0 = 1; // Test case 4: s1=1, s0=1, i=1 -> o3 should be 1
   #10;   i = 0; s1 = 1; s0 = 1; // Test case 5: i=0, all outputs should be 0
   #10;    // End simulation
   $stop;

 end
endmodule
```

1×8 De-multiplexer:

- In 1 to 8 De-multiplexer, there are total of eight outputs, i.e., Y_0, Y_1, Y_2, Y_3, Y_4, Y_5, Y_6, and Y_7, 3 selection lines, i.e., S_0, $S_1$and S_2 and single input, i.e., A.
- On the basis of the combination of inputs which are present at the selection lines S^0, S^1 and S_2, the input will be connected to one of these outputs.



1×8 De-multiplexer:

Truth Table:

INPUTS			Output							
S_2	S_1	S_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	A
0	0	1	0	0	0	0	0	0	A	0
0	1	0	0	0	0	0	0	A	0	0
0	1	1	0	0	0	0	A	0	0	0
1	0	0	0	0	0	A	0	0	0	0
1	0	1	0	0	A	0	0	0	0	0
1	1	0	0	A	0	0	0	0	0	0
1	1	1	A	0	0	0	0	0	0	0

The logical expression of the term Y is as follows:

$$Y_0=S_0'.S_1'.S_2'.A$$

$$Y_1=S_0.S_1'.S_2'.A$$

$$Y_2=S_0'.S_1.S_2'.A$$

$$Y_3=S_0.S_1.S_2'.A$$

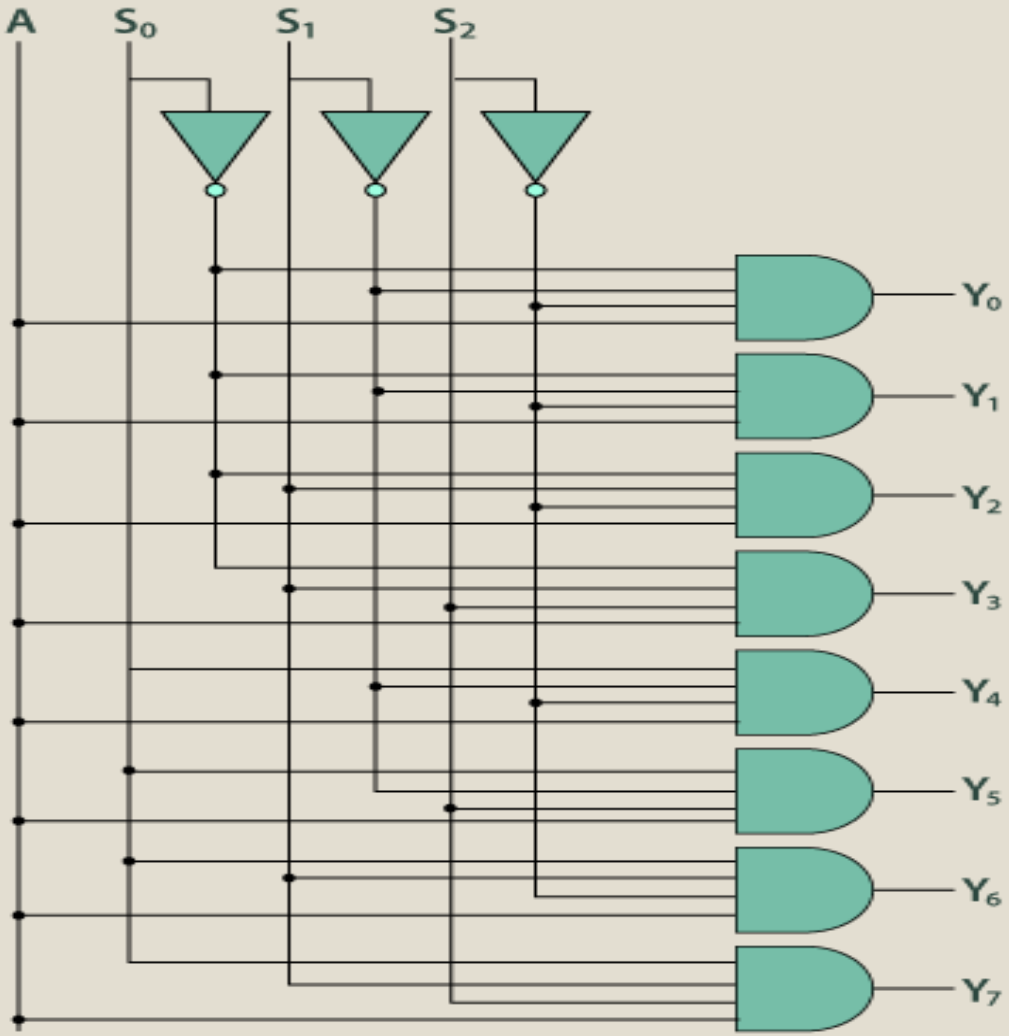
$$Y_4=S_0'.S_1'.S_2A$$

$$Y_5=S_0.S_1'.S_2A$$

$$Y_6=S_0'.S_1.S_2A$$

$$Y_7=S_0.S_1.S_3.A$$

Logical circuit of the above expressions is given below:



Verilog code

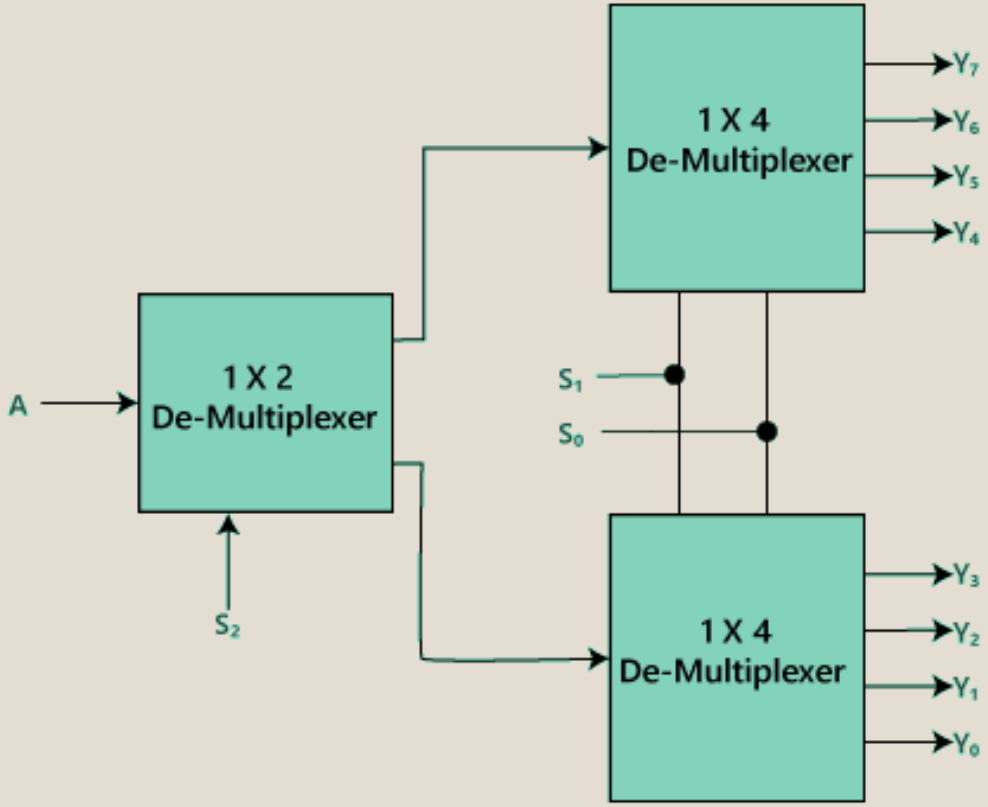
```
module demux_1_to_8( input d,s0,s1,s2,output y0, y1,y2,y3, y4, y5, y6, y7   );

not (s0n,s0),(s1n,s1),(s2n,s2);

and g1 (y0,d,s0n,s1n,s2n);
and g2 (y1,d,s0,s1n,s2n);
and g3 (y2,d,s0n,s1,s2n);
and g4 (y3,d,s0,s1,s2n);
and g5 (y4,d,s0n,s1n,s2);
and g6 (y5,d,s0,s1n,s2);
and g7 (y6,d,s0n,s1,s2);
and g8 (y7,d,s0,s1,s2);
endmodule
```

//Testbench code for 1-8 DEMUX Structural/Gate Level Modelling

```
initial begin
 // Initialize Inputs

 d = 0;s0 = 0;s1 = 0;s2 = 0;

 // Wait 100 ns for global reset to finish

#100;
// Add stimulus here
 #100; d = 1;s0 = 0;s1 = 0;s2 = 0;
 #100; d = 1;s0 = 1;s1 = 0;s2 = 0;
 #100; d = 1;s0 = 0;s1 = 1;s2 = 0;
 #100; d = 1;s0 = 1;s1 = 1;s2 = 0;
 #100; d = 1;s0 = 0;s1 = 0;s2 = 1;
 #100; d = 1;s0 = 1;s1 = 0;s2 = 1;
 #100; d = 1;s0 = 0;s1 = 1;s2 = 1;
 #100; d = 1;s0 = 1;s1 = 1;s2 = 1;
end
```

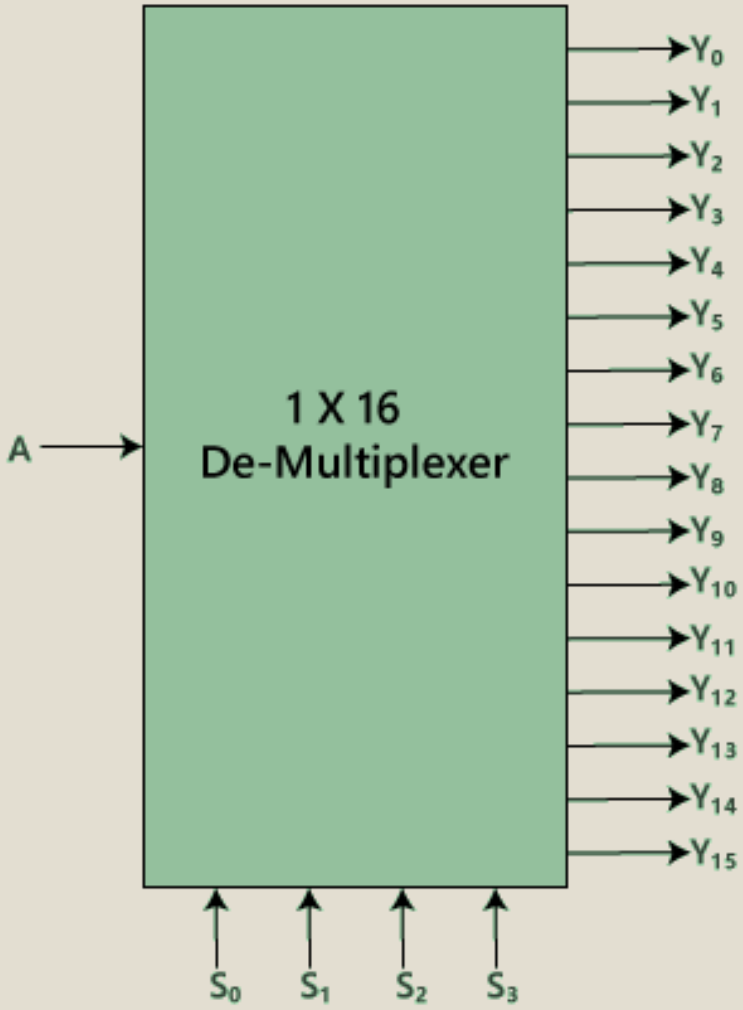
1×8 De-multiplexer using 1×4 and 1×2 de-multiplexer

- We can implement the 1×8 de-multiplexer using a lower order de-multiplexer.
- To implement the 1×8 de-multiplexer, we need two 1×4 de-multiplexer and one 1×2 de-multiplexer.
- The 1×4 multiplexer has 2 selection lines, 4 outputs, and 1 input.
- The 1×2 de-multiplexer has only 1 selection line.
- For getting 8 data outputs, we need two 1×4 de-multiplexer.
- The 1×2 de-multiplexer produces two outputs.
- So, in order to get the final output, we have to pass the outputs of 1×2 de-multiplexer as an input of both the 1×4 de-multiplexer.



1 x 16 De-multiplexer

- In 1×16 de-multiplexer, there are total of 16 outputs, i.e., Y0, Y1, …, Y16, 4 selection lines, i.e., S0, S1, S2, and S3 and single input, i.e., A. On the basis of the combination of inputs which are present at the selection lines S0, S1, and S2, the input will be connected to one of these outputs.

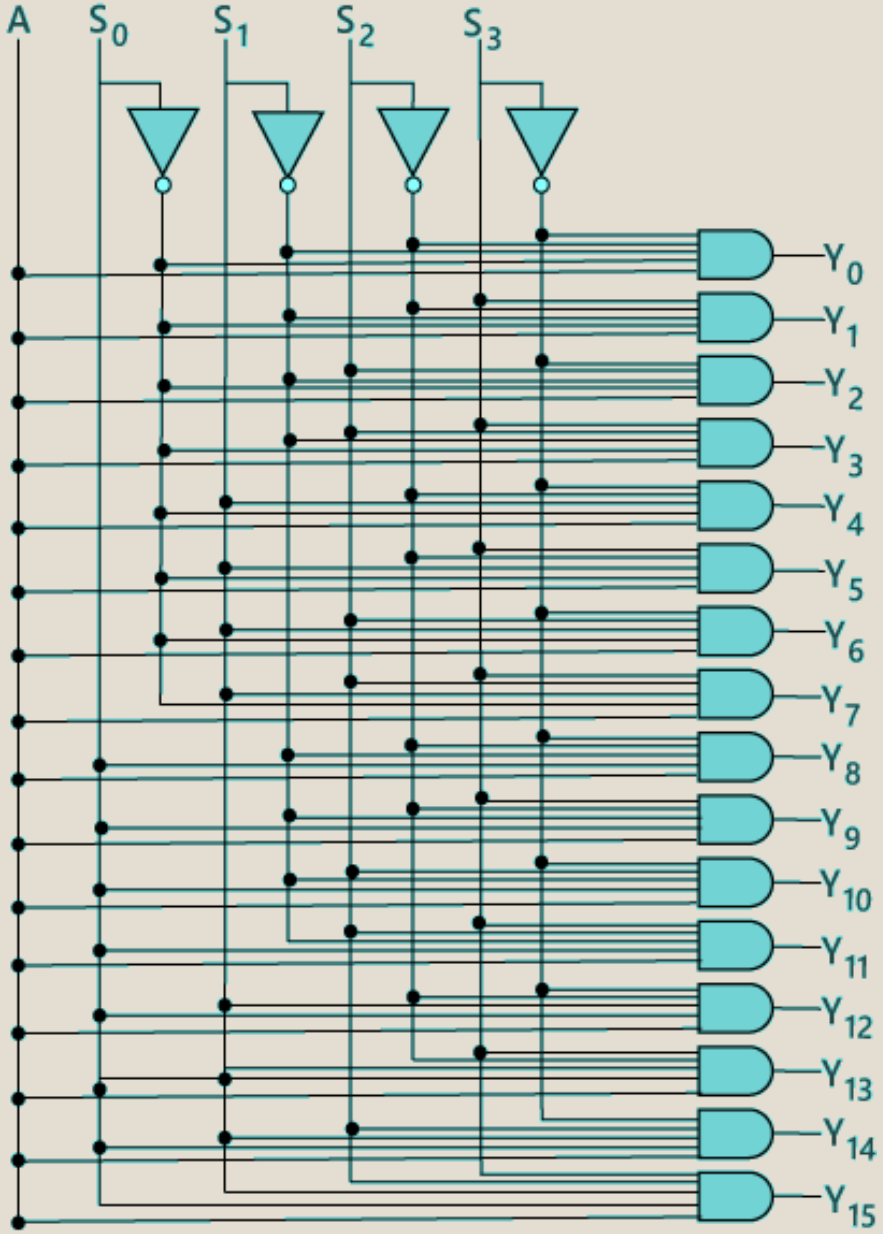


Truth Table:

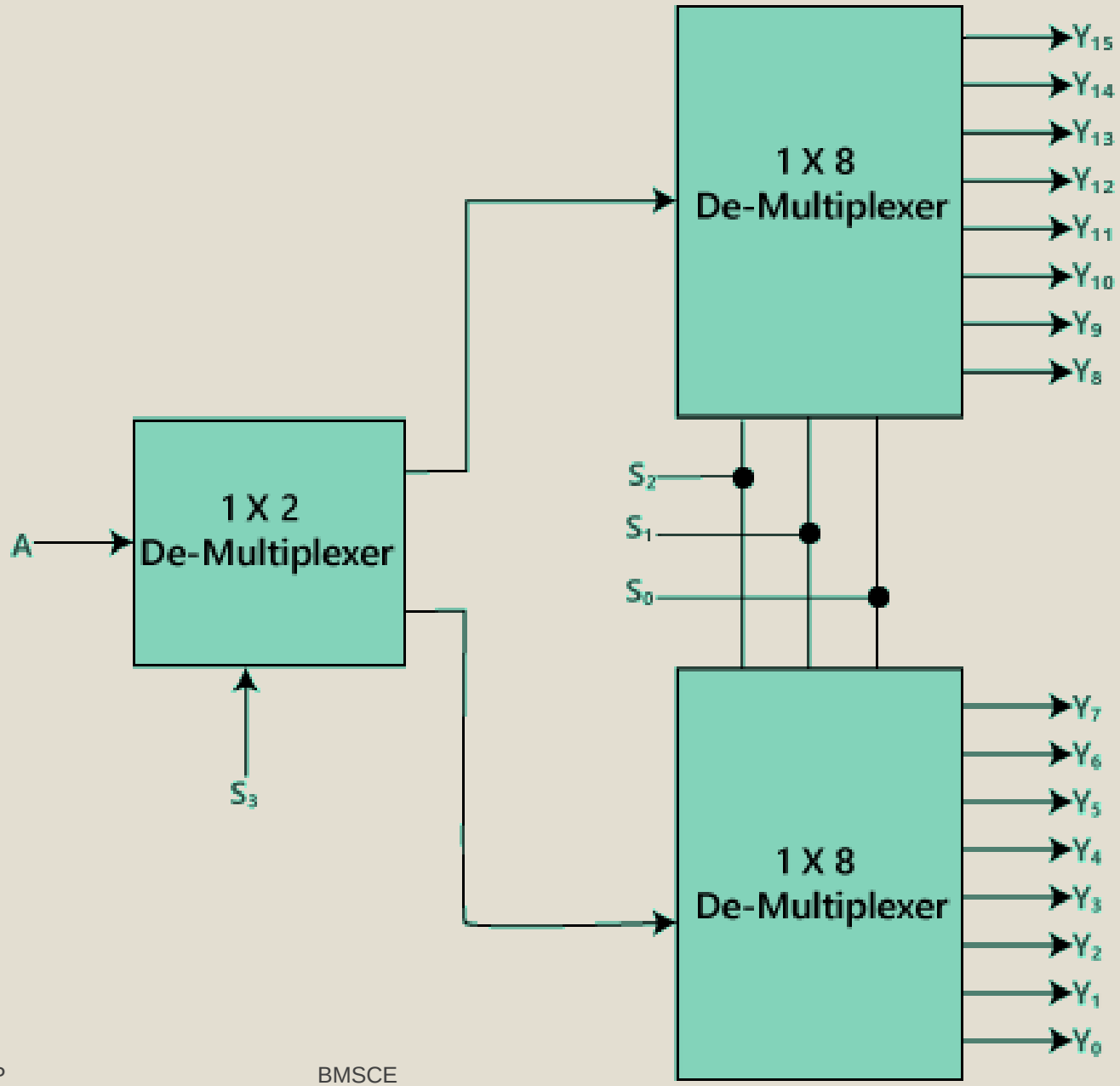
INPUTS				OUTPUTS															
S_3	S_2	S_1	S_0	Y_{15}	Y_{14}	Y_{13}	Y_{12}	Y_{11}	Y_{10}	Y_9	Y_8	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	A
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	A	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	A	0	0
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	A	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	A	0	0	0	0
0	1	0	1	0	0	0	0	0	0	0	0	0	0	A	0	0	0	0	0
0	1	1	0	0	0	0	0	0	0	0	0	0	A	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	0	A	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	A	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	A	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	A	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	A	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	A	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	A	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	A	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	A	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

- The logical expression of the term Y is as follows:
  - $Y_0=A.S_0'.S_1'.S_2'.S_3'$
    $Y_1=A.S_0'.S_1'.S_2'.S_3$
    $Y_2=A.S_0'.S_1'.S_2.S_3'$
    $Y_3=A.S_0'.S_1'.S_2.S_3$
    $Y_4=A.S_0'.S_1.S_2'.S_3'$
    $Y_5=A.S_0'.S_1.S_2'.S_3$
    $Y_6=A.S_0'.S_1.S_2.S_3'$
    $Y_7=A.S_0'.S_1.S_2.S_3$
    $Y_8=A.S_0.S_1'.S_2'.S_3'$
    $Y_9=A.S_0.S_1'.S_2'.S_3$
    $Y_{10}=A.S_0.S_1'.S_2.S_3'$
    $Y_{11}=A.S_0.S_1'.S_2.S_3$
    $Y_{12}=A.S_0.S_1.S_2'.S_3'$
    $Y_{13}=A.S_0.S_1.S_2'.S_3$
    $Y_{14}=A.S_0.S_1.S_2.S_3'$
    $Y_{15}=A.S_0.S_1.S_2'.S_3$

Logical circuit:

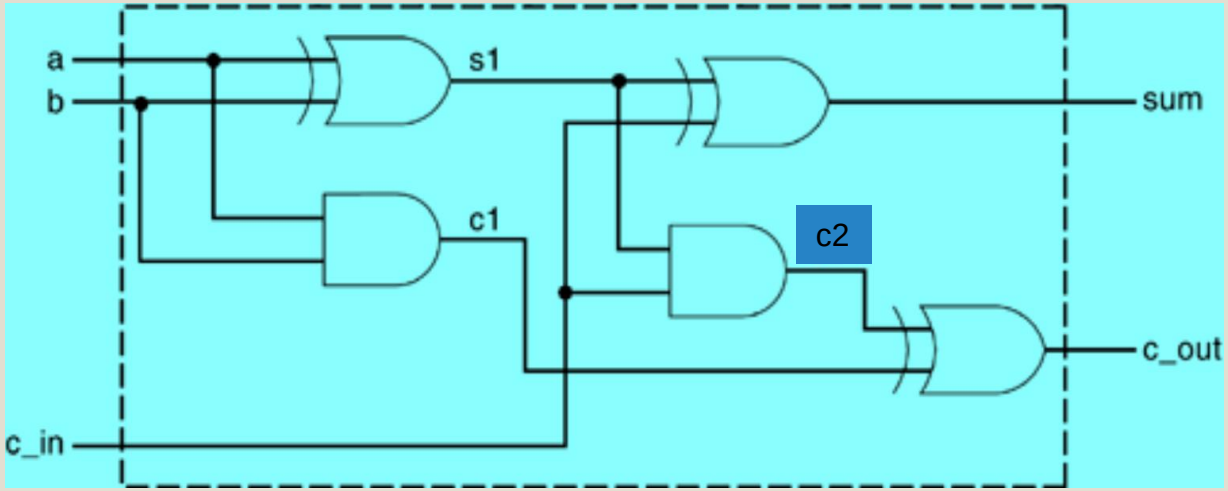


The block diagram of 1×16 de-multiplexer using 1×8 and 1×2 de-multiplexer



Eg: 4-bit Ripple Carry Full Adder

- The basic building block is a 1-bit full adder.
- The mathematical equations for a 1-bit full adder are shown below.
  - sum = (a $\oplus$ b $\oplus$ cin)
  - cout = (a . b) + cin . (a $\oplus$ b)
- The logic diagram for a 1-bit full adder is shown in Figure.



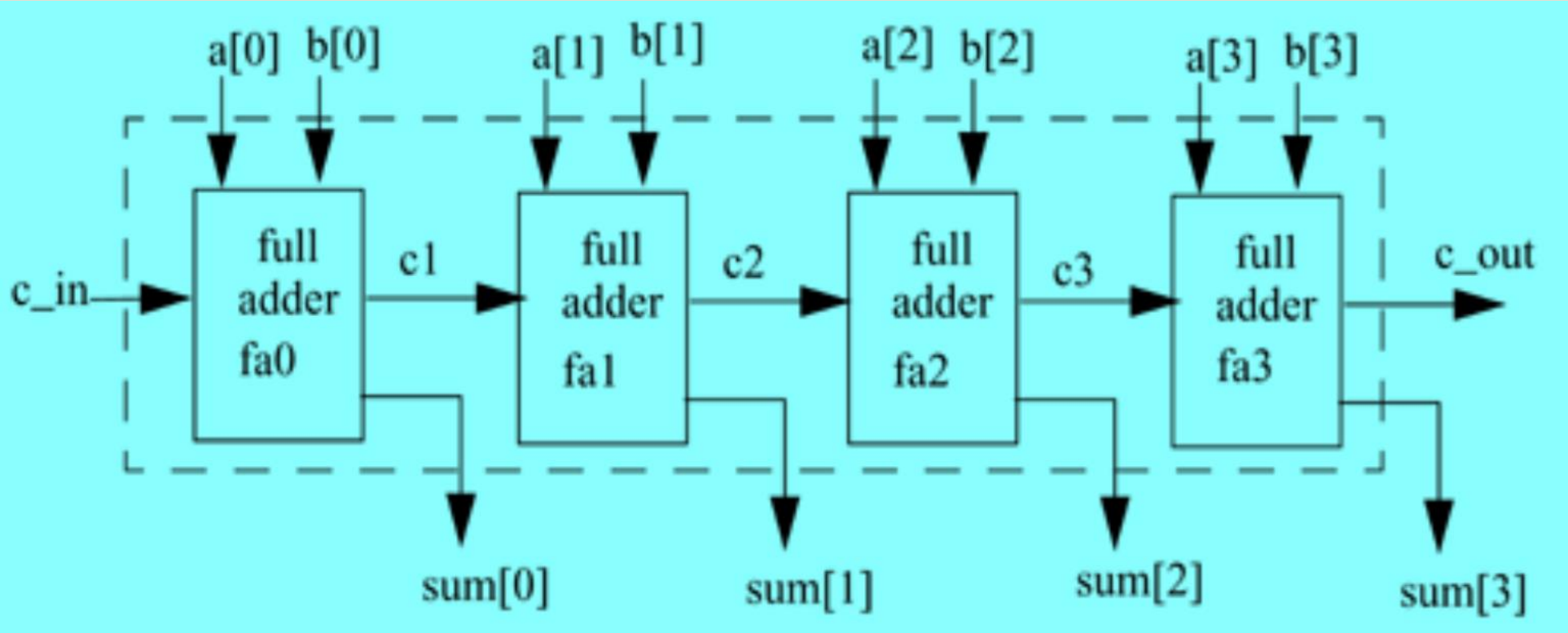
Eg: 4-bit Ripple Carry Full Adder

- **Verilog Description for 1-bit Full Adder**

```
// Define a 1-bit full adder
module fulladd(sum, c_out, a, b, c_in);
// I/O port declarations
output sum, c_out;
input a, b, c_in;
// Internal nets
wire s1, c1, c2;
// Instantiate logic gate primitives
xor (s1, a, b);
and (c1, a, b);
xor (sum, s1, c_in);
and (c2, s1, c_in);
xor (c_out, c2, c1);
endmodule
```

Eg: 4-bit Ripple Carry Full Adder

- A 4-bit ripple carry full adder can be constructed from four 1-bit full adders, as shown in Figure.
- Note that **fa0, fa1, fa2, and fa3 are instances** of the **module fulladd (1-bitfull adder).**



Eg: 4-bit Ripple Carry Full Adder

- Note that the port names used in a 1-bit full adder and a 4-bit full adder are the same but they represent different elements.
- The element sum in a 1-bit adder is a scalar quantity and the element sum in the 4-bit full adder is a 4-bit vector quantity.
- Verilog keeps names local to a module.
- Names are not visible outside the module unless hierarchical name referencing is used.
- Also note that instance names must be specified when defined modules are instantiated, but when instantiating Verilog primitives, the instance names are optional.

Verilog Description for 4-bit Ripple Carry Full Adder

```
// Define a 4-bit full adder
module fulladd4(sum, c_out, a, b, c_in);
// I/O port declarations
output [3:0] sum;
output c_out;
input[3:0] a, b;
input c_in;
// Internal nets
wire c1, c2, c3;
// Instantiate four 1-bit full adders.
fulladd fa0(sum[0], c1, a[0], b[0], c_in);
fulladd fa1(sum[1], c2, a[1], b[1], c1);
fulladd fa2(sum[2], c3, a[2], b[2], c2);
fulladd fa3(sum[3], c_out, a[3], b[3], c3);
endmodule
```

```
module stimulus;
reg [3:0] A, B; // Set up variables
reg C_IN;
wire [3:0] SUM;
wire C_OUT;
// Instantiate the 4-bit full adder. call it FA1_4

fulladd4 FA1_4(SUM, C_OUT, A, B, C_IN);
Initial    begin
$monitor($time," A= %b, B=%b, C_IN= %b, --- C_OUT= %b,
SUM= %b\n",A, B, C_IN, C_OUT, SUM);
end
Initial   begin                    // Stimulate inputs
A = 4'd0; B = 4'd0; C_IN = 1'b0;
#5 A = 4'd3; B = 4'd4;
#5 A = 4'd2; B = 4'd5;
#5 A = 4'd9; B = 4'd9;
#5 A = 4'd10; B = 4'd15;
#5 A = 4'd10; B = 4'd5; C_IN = 1'b1;
end
endmodule
```

Verilog Description for 4-bit Ripple Carry Full Adder

The output of the simulation is shown below:

0 A= 0000, B=0000, C_IN= 0, --- C_OUT= 0, SUM= 0000

5 A= 0011, B=0100, C_IN= 0, --- C_OUT= 0, SUM= 0111

10 A= 0010, B=0101, C_IN= 0, --- C_OUT= 0, SUM= 0111

15 A= 1001, B=1001, C_IN= 0, --- C_OUT= 1, SUM= 0010

20 A= 1010, B=1111, C_IN= 0, --- C_OUT= 1, SUM= 1001

25 A= 1010, B=0101, C_IN= 1,, C_OUT= 1, SUM= 0000

Gate Delays

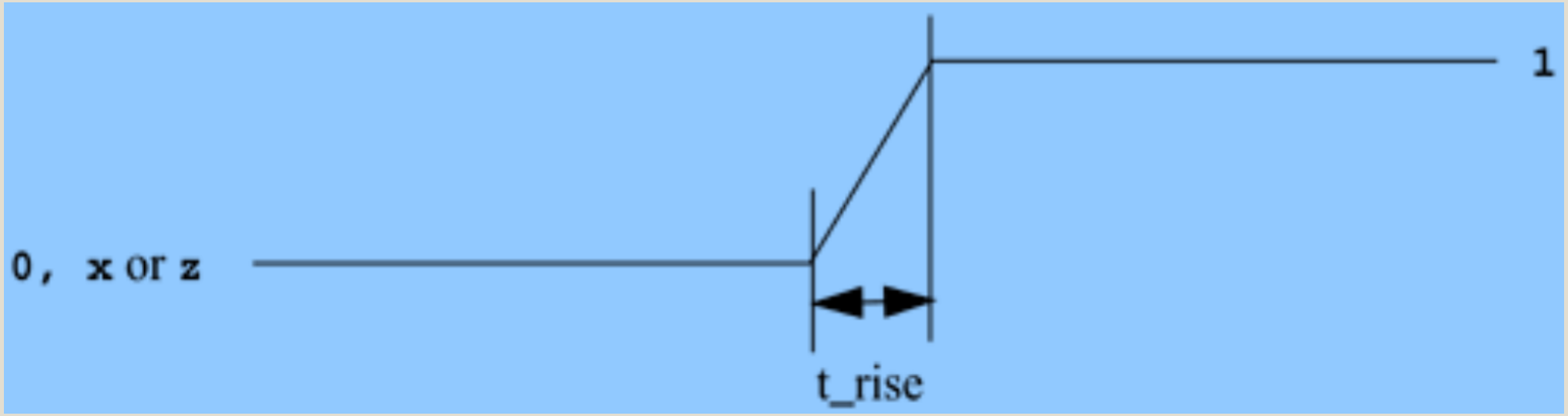
- In real circuits, logic gates have delays associated with them. Gate delays allow the Verilog user to specify delays through the logic circuits. Pin-to-pin delays can also be specified in Verilog.
- Verilog gate delays specify how values propagate through nets or gates.
- The gate delay declaration specifies a time needed to propagate a signal change from the gate input to its output.
- **The gate delay declaration can be used in gate instantiations.** The delays can also be used for delay control in procedural statements.
- Digital elements are binary entities and only hold either of the two values, 0 and 1.
- The transition from 0 to 1 and 1 to 0 has a **transitional delay**, and therefore each gate element propagates the value from input to its output.
- For example, a two-input AND gate has to switch the output to 1 if both inputs become 1 and back to 0 when inputs become 0.
- The net delay declaration specifies a time needed to propagate values from drivers through the net. It can be used in continuous assignments and net declarations.

Gate Delays: **Rise, Fall, and Turn-off Delays**

- The delays declaration can contain up to three values, such as **rise, fall, and turn-off delays**.
- **The time taken for the output of a gate to change from some value to 1 is called a rise delay.**
- **The time taken for the output of a gate to change form some value to 0 is called a fall delay.**
- **The time taken for the output of a gate to change from some value to high impedance is called turn-off delay.**
- If only one delay value is specified, then it is used for all signal changes. **The default delay is zero**.
- If **two delays** are specified, then the **first delay specifies the rise delay**, and the **second delay specifies the fall delay.**
- If the signal changes to high-impedance or unknown, then the smaller value will be used.

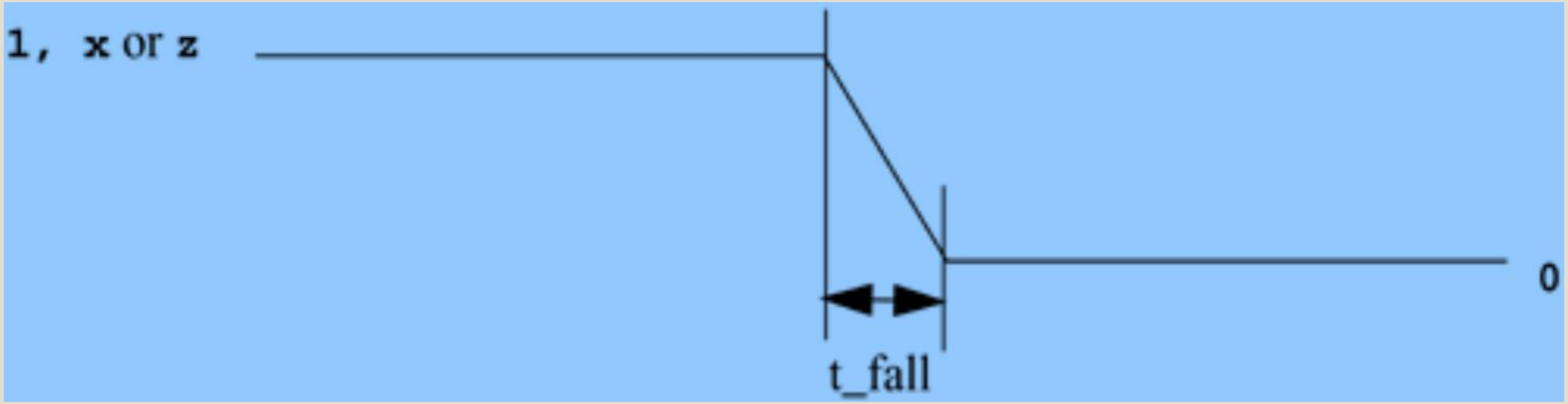
Rise delay

- The rise delay is associated with a gate output transition to a 1 from another value.



Fall delay

- The fall delay is associated with a gate output transition to a 0 from another value.



Turn-off delay

- The turn-off delay is associated with a gate output transition to the high impedance value(z) from another value.
- If the value changes to x, the minimum of the three delays is considered.
- Types of Delay Specification:
- `// Delay of delay_time for all transitions`

```
and #(delay_time) a1(out, i1, i2);
```

- `// Rise and Fall Delay Specification.`

```
and #(rise_val, fall_val) a2(out, i1, i2);
```

- `// Rise, Fall, and Turn-off Delay Specification`

```
bufif0 #(rise_val, fall_val, turnoff_val) b1 (out, in, control);
```

Gate Delays: Min/Typ/Max Values

- Verilog provides an additional level of control for each type of delay mentioned above.
- For each type of delay **rise, fall, and turn-off** three values, **min, typ, and max**, can be specified.
- Any one value can be chosen at the start of the simulation.
- **Min/typ/max** values are used to model devices whose delays vary within a minimum and maximum range because of the IC fabrication process variations.
- **Min value**
  - **The min value is the minimum delay value that the designer expects the gate to have.**
- **Typ val**
  - **The typ value is the typical delay value that the designer expects the gate to have.**
- **Max value**
  - **The max value is the maximum delay value that the designer expects the gate to have.**

Examples: min, typ, and max value specification

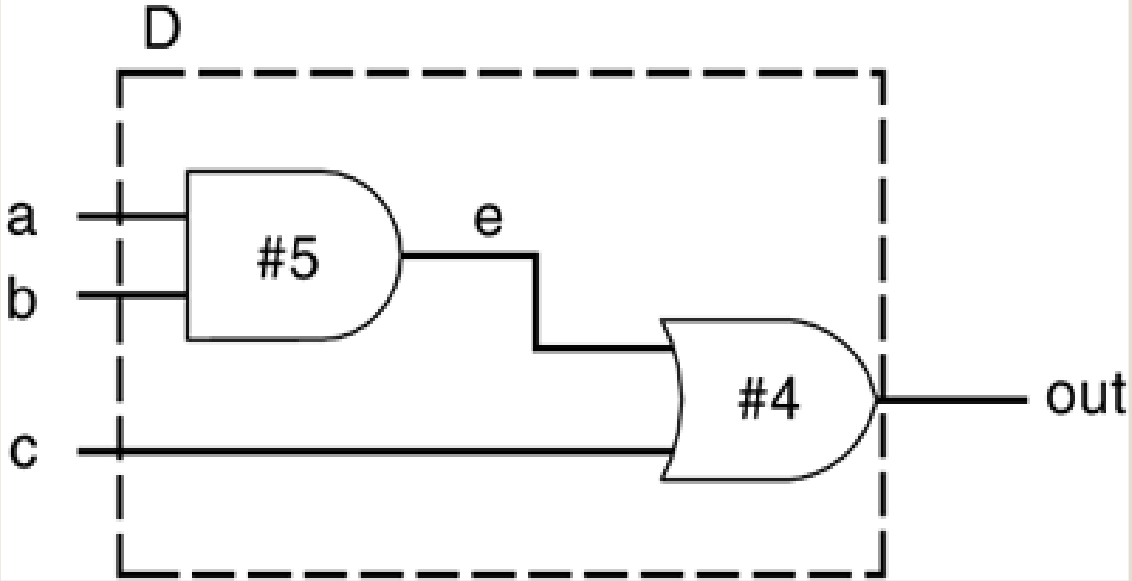
- Min, Max, and Typical Delay Values

```
// One delay
// if +mindelays, delay= 4
// if +typdelays, delay= 5
// if +maxdelays, delay= 6
and #(4:5:6) a1(out, i1, i2);
```

```
// Two delays
// if +mindelays, rise= 3, fall= 5, turn-off = min(3,5)
// if +typdelays, rise= 4, fall= 6, turn-off = min(4,6)
// if +maxdelays, rise= 5, fall= 7, turn-off = min(5,7)
and #(3:4:5, 5:6:7) a2(out, i1, i2);

// Three delays
// if +mindelays, rise= 2 fall= 3 turn-off = 4
// if +typdelays, rise= 3 fall= 4 turn-off = 5
// if +maxdelays, rise= 4 fall= 5 turn-off = 6
and #(2:3:4, 3:4:5, 4:5:6) a3(out, i1,i2);
```

Delay Example

- A simple module called D implements the following logic equations:
- **out = (a . b) + c**
- The gate-level implementation is shown in Module D.
- The module contains two gates with delays of 5 and 4 time units.



Eg1: Verilog Definition for Module D with Delay

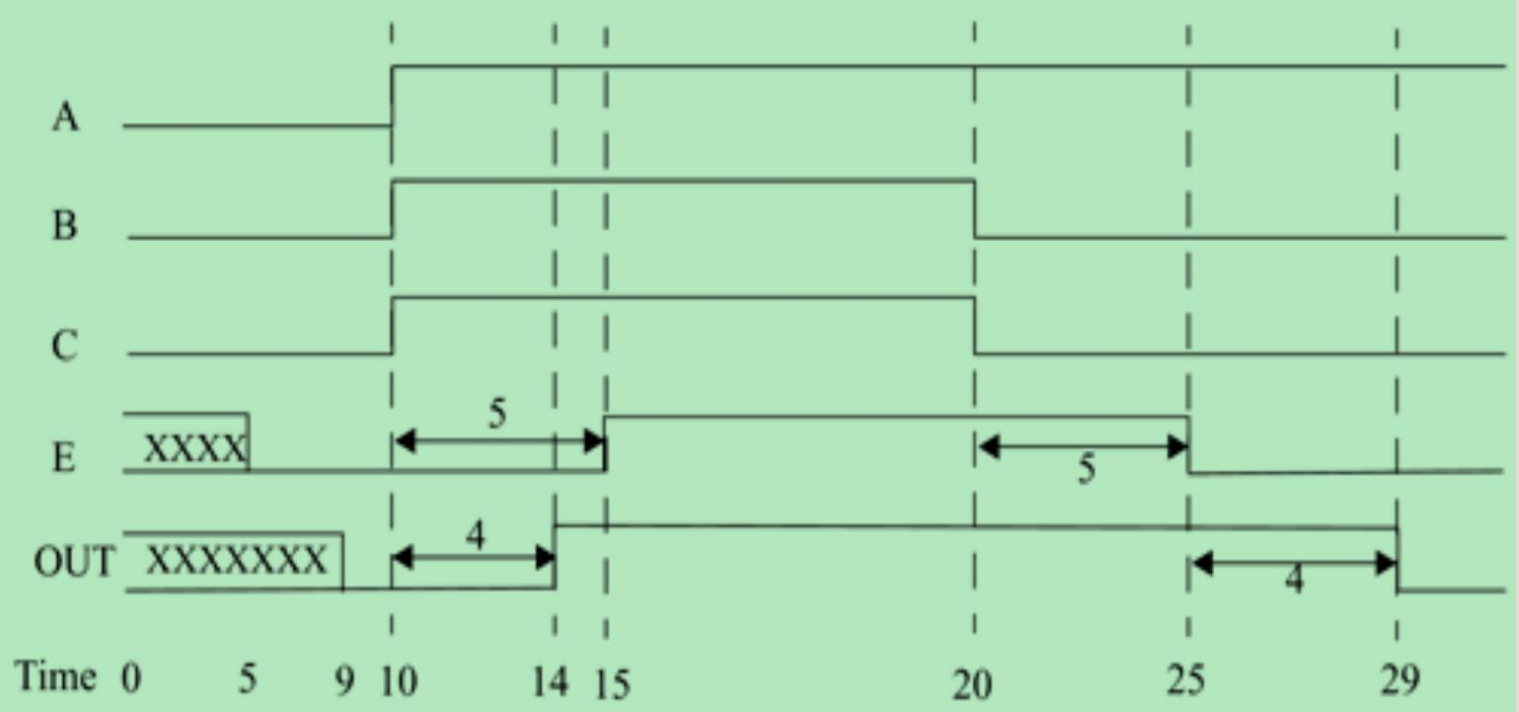
```
module D (out, a, b, c);
output out;//I/O port declarations
input a,b,c;
wire e;     // Internal nets
// Instantiate primitive gates
to build the circuit
and #(5) a1(e, a, b);
//Delay of 5 on gate a1
or #(4) o1(out, e,c);
//Delay of 4 on gate o1
endmodule
```

```
// Stimulus (top-level module)
module stimulus;
// Declare variables
reg A, B, C;
wire OUT;
// Instantiate the module D
D d1( OUT, A, B, C);
// Stimulate the inputs.
//Finish the simulation at 40 time
units.
initial
begin
A= 1'b0; B= 1'b0; C= 1'b0;
#10 A= 1'b1; B= 1'b1; C= 1'b1;
#10 A= 1'b1; B= 1'b0; C= 1'b0;
#20 $finish;
end
endmodule
```

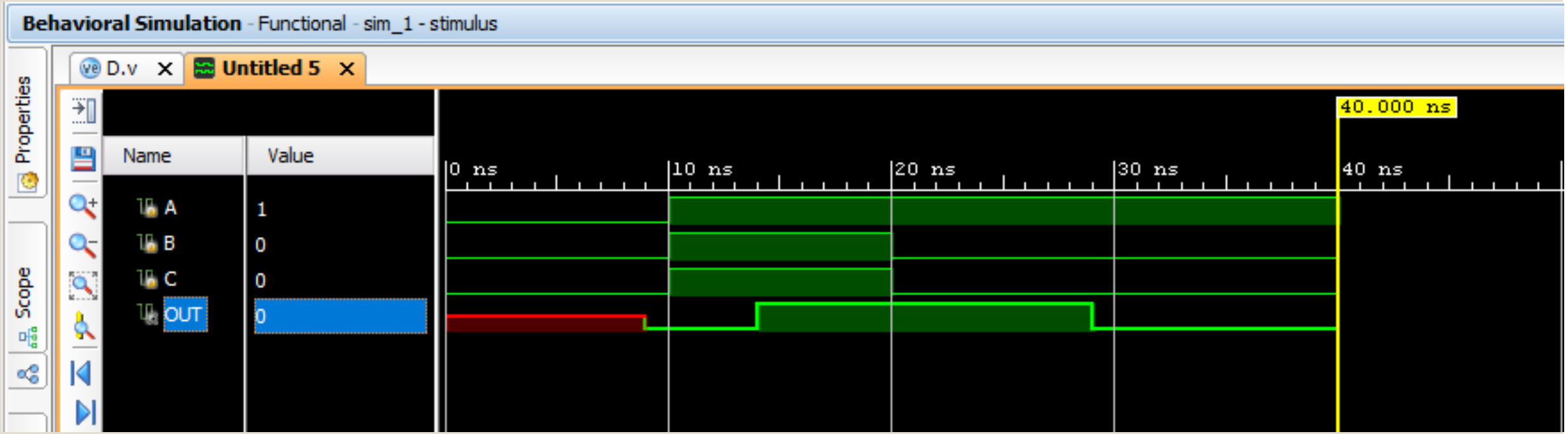
Waveforms for Delay Simulation

Simulation time at each transition is specified below the transition.

1. The outputs E and OUT are initially unknown.
2. At time 10, after A, B, and C all transition to 1, OUT transitions to 1 after a delay of 4 time units and E changes value to 1 after 5 time units.
3. At time 20, B and C transition to 0. E changes value to 0 after 5 time units, and OUT transitions to 0, 4 time units after E changes.



Waveforms for Delay Simulation



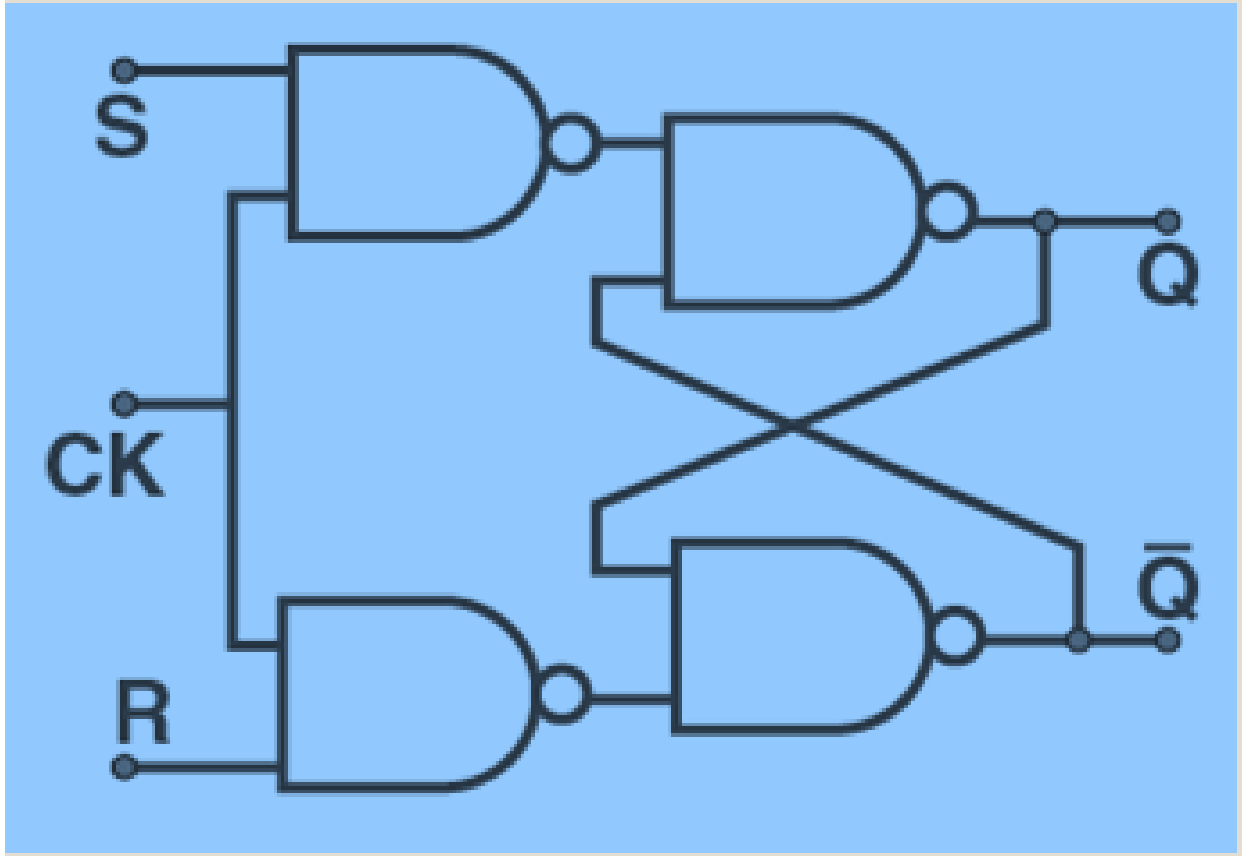
Example 2: Deduce the design block for the given Verilog code and draw the waveforms for the stimulus given:

```
module sr latch(q, qbar, s, r, ck); 10
output q, qbar;
input s, r, ck;
wire sbar, rbar;
nand #5 U0 (sbar, s, ck);
nand #5 U1 (rbar, r, ck);
nand #5 U2 (q, sbar, qbar);
nand #5 U3 (qbar, rbar, q);
endmodule
```

```
module stimulus;
reg s, r, ck=1'b0;
wire q, qbar;
sr latch uut (q, qbar, s, r, ck);
always #10 ck = ~ck;
initial
begin
s = 1'b1; r = 1'b0;
#25 s = 1'b0; r = 1'b0;
#20 s = 1'b1; r = 1'b1;
#20 s = 1'b0; r = 1'b1;
#20 $finish;
end
endmodule
```

Answer:

- The provided Verilog code implements an SR latch with clock using NAND gates.
- Let’s break down the design block and analyze its behavior.
- Design Analysis of SR Latch Module:
  - Inputs: s (Set), r (Reset), ck (Clock)
  - Outputs: q, qbar (complement of q)
  - Internal Wires: sbar, rbar
- Working:sbar and rbar are generated by nand gates (U0 and U1), where:
  - sbar is the NAND of s and ck
  - rbar is the NAND of r and ck
- q and qbar are determined by nand gates (U2 and U3), forming the cross-coupled latch:
  - q is the NAND of sbar and qbar
  - qbar is the NAND of rbar and q
- Timing:Each NAND gate has a delay of 5 time units.



Truth Table

S	R	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	-
1	1	1	-

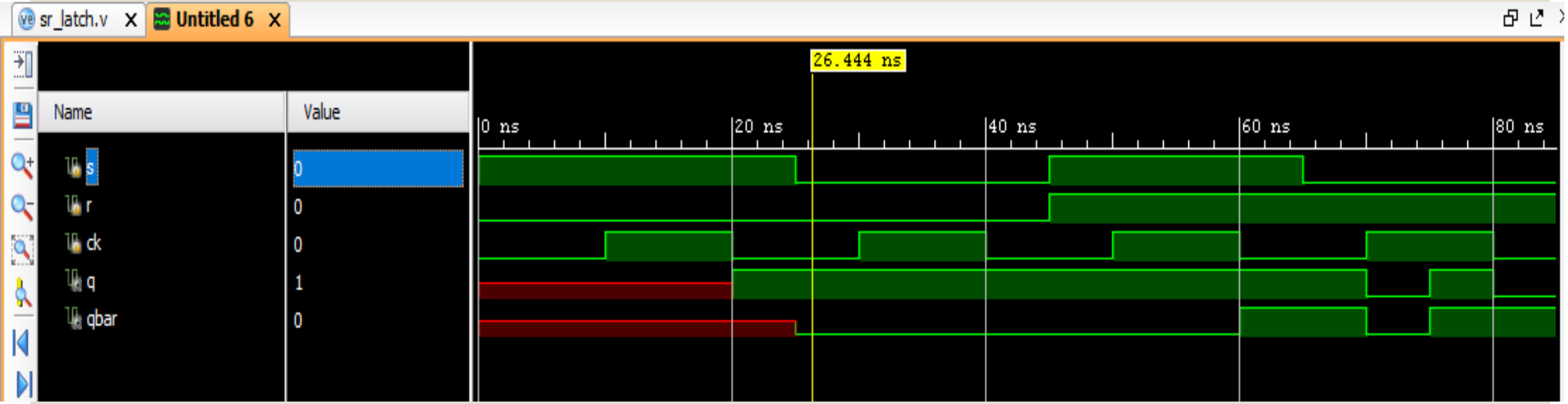
Analysis:

- Initial Conditions:
  - ck toggles every 10 time units (#10 ck = ~ck).
  - Initial Inputs: s = 1, r = 0
- Stimulus Sequence:
  - At t = 0: s = 1, r = 0 (Set condition)
  - At t = 25: s = 0, r = 0 (Hold condition)
  - At t = 45: s = 1, r = 1 (Invalid condition, both set and reset are 1)
  - At t = 65: s = 0, r = 1 (Reset condition)Simulation ends at t = 85.

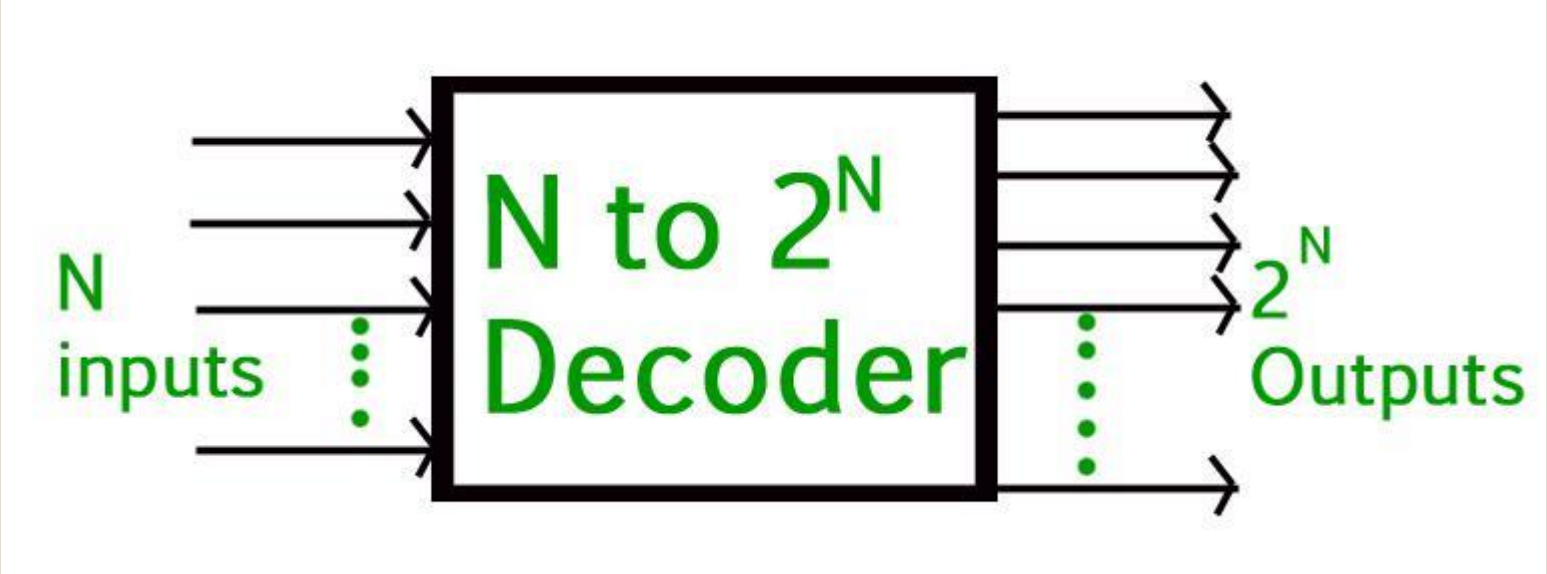
Analysis:

- **Expected Behavior of the SR Latch**
- The behavior can be summarized as follows:
  - Set Condition (s = 1, r = 0): q should become 1 and qbar should become 0.
  - Reset Condition (s = 0, r = 1): q should become 0 and qbar should become 1.
  - Hold Condition (s = 0, r = 0): The latch retains its previous state.
  - Invalid Condition (s = 1, r = 1): This condition is typically avoided as it leads to an undefined state.

Waveforms :


sr_latch.v
Untitled 6
Name
Value
s
0
r
0
ck
0
q
1
qbar
0
26.444 ns
0 ns
20 ns
40 ns
60 ns
80 ns

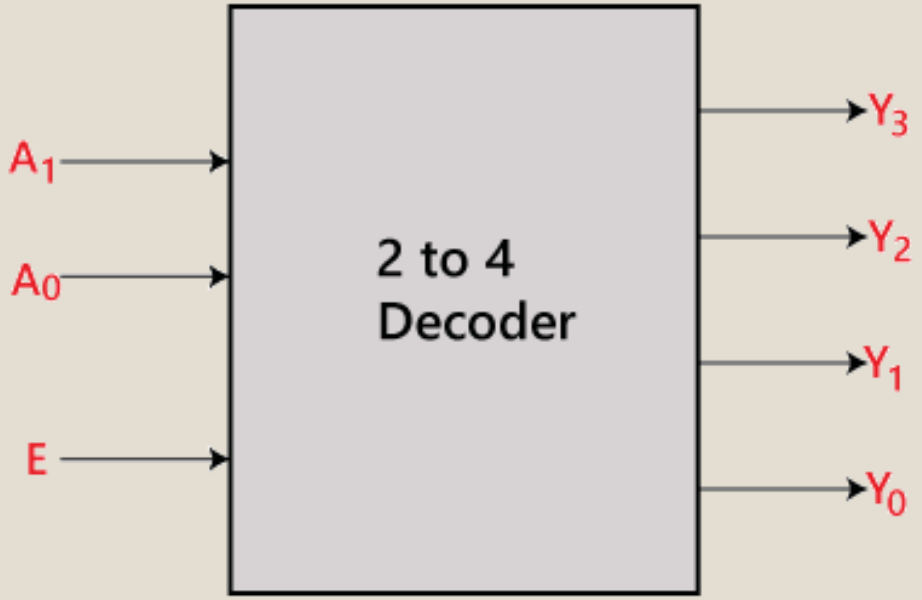
Eg:- Decoder gate level model



- **Decoder**
- This Decoder is a combinational logic circuit, and its purpose is to decode the data given to it. It is made of n number of input lines and 2^n output lines.
- For every probable input condition, there are various output signals and depending on the input.
- Need of Decoder: Memory Addressing, Seven-Segment Displays, Control Systems

Decoder - 2 to 4 line decoder:

- In the 2 to 4 line decoder, there is a total of three inputs, i.e., A0, and A1 and E and four outputs, i.e., Y0, Y1, Y2, and Y3.
- For each combination of inputs, when the enable 'E' is set to 1, one of these four outputs will be 1.



Truth Table:

Enable	INPUTS		OUTPUTS			
E	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

Decoder - 2 to 4 line decoder:

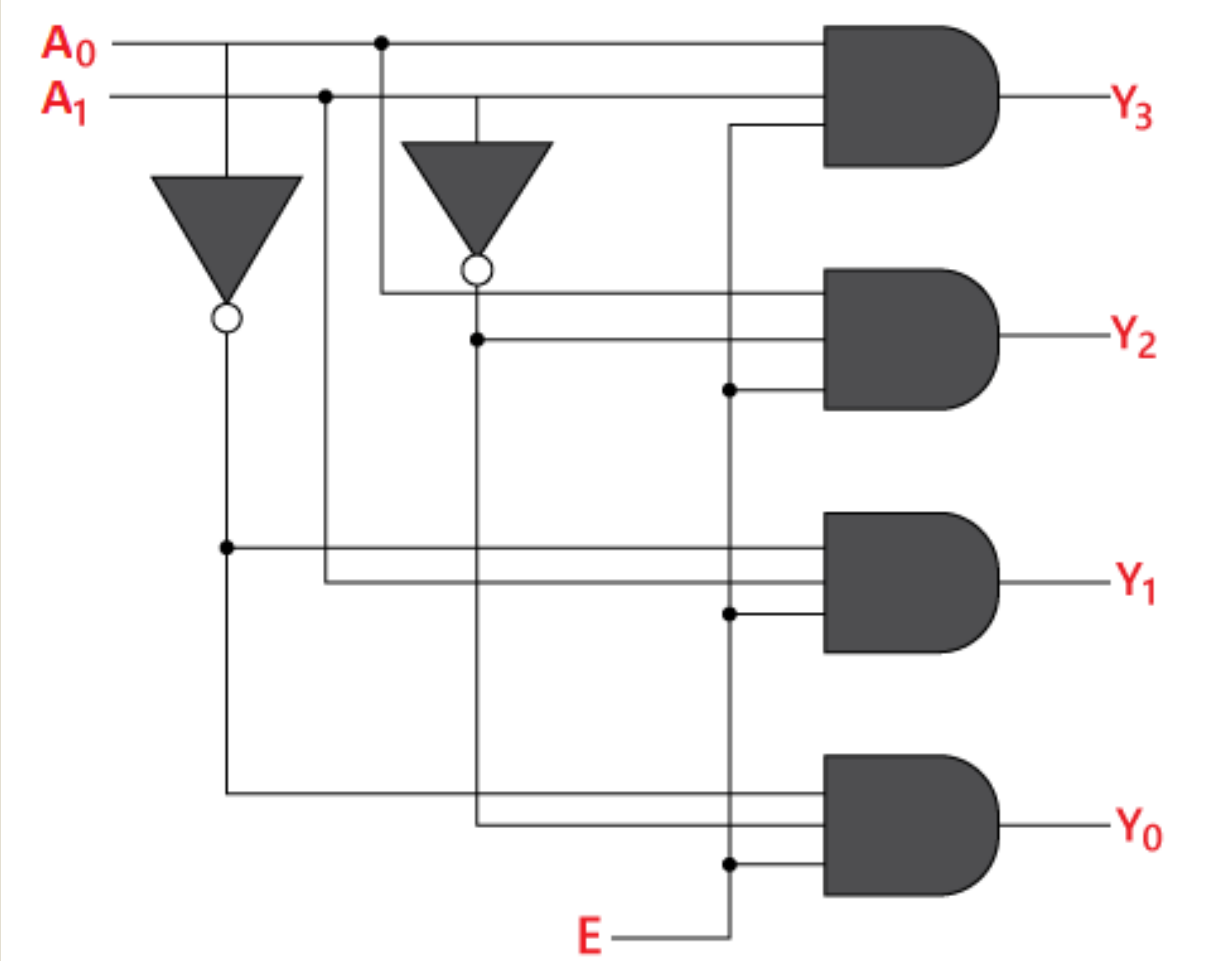
- **The logical expression and circuit of the term Y0, Y0, Y2, and Y3 is as follows:**

$$Y_3 = E.A_1.A_0$$

$$Y_2 = E.A_1.A_0'$$

$$Y_1 = E.A_1'.A_0$$

$$Y0 = E.A_1'.A_0'$$



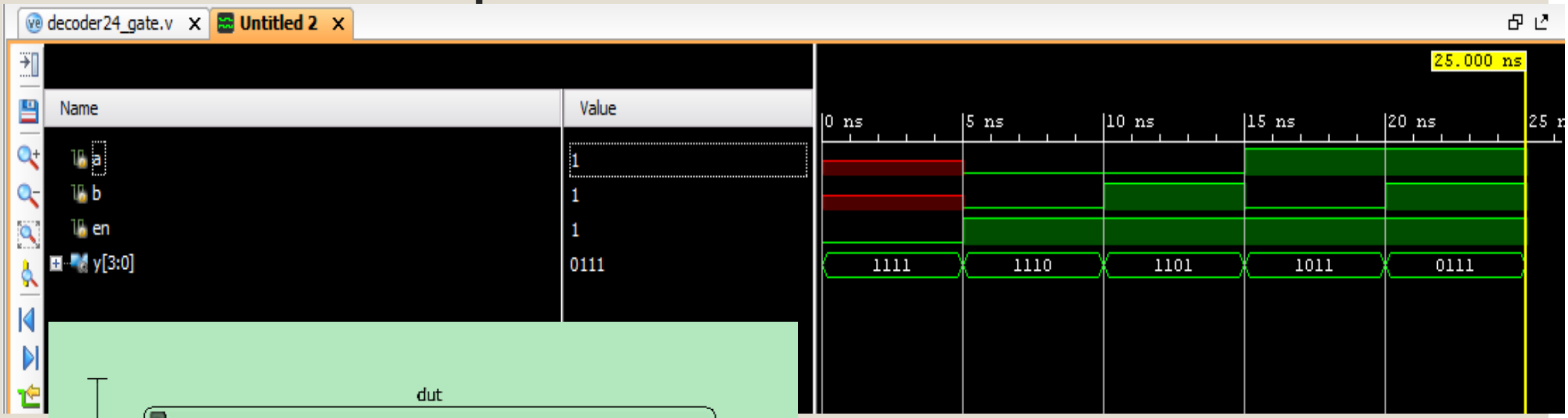
Decoder - 2 to 4 line decoder: gate level

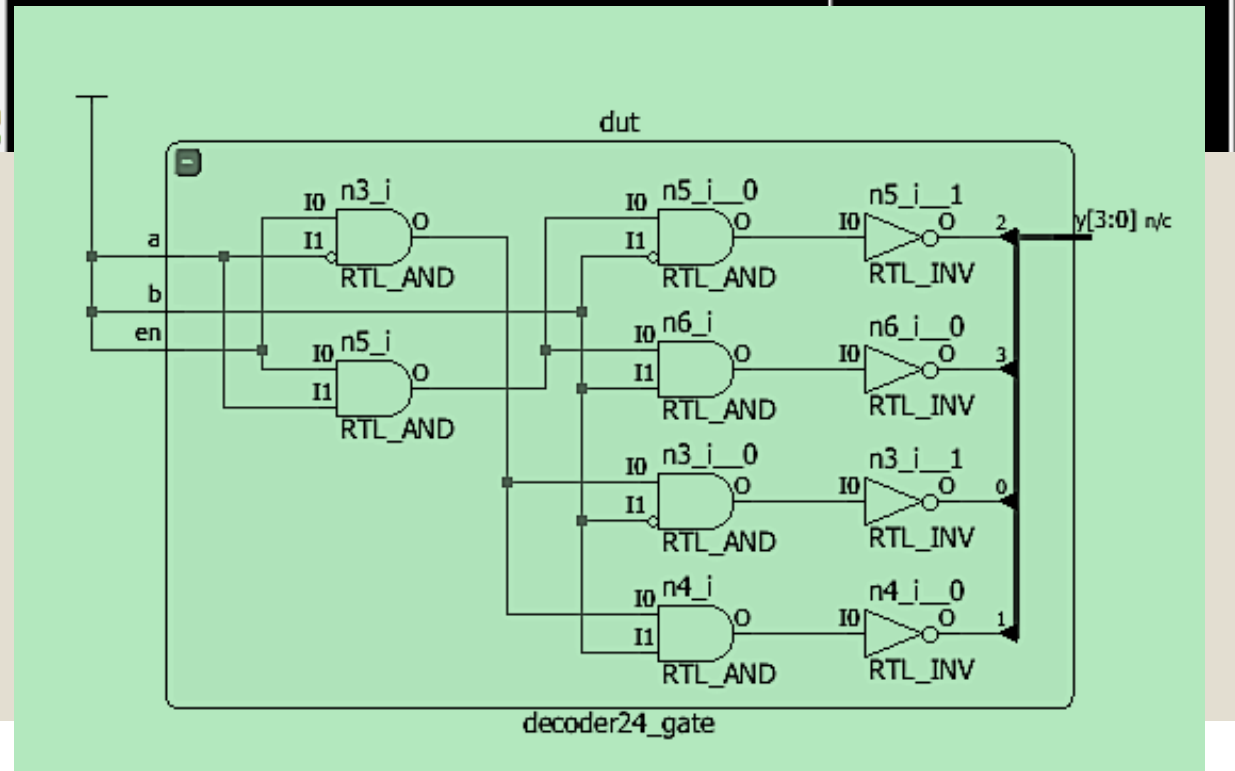
```
module decoder24_gate(en,a,b,y); // declare input and output ports
input en,a,b;
    output [3:0]y;
   wire na,nb; // supportive connections required to connect nand gates
    // instantiate 4 nand gates and 3 not gates
    not n1(na,a);
    not n2(nb,b);
    nand n3(y[0],en,na,nb);
    nand n4(y[1],en,na,b);
    nand n5(y[2],en,a,nb);
    nand n6(y[3],en,a,b);
endmodule
```

Decoder - 2 to 4 line decoder: TB

```
module tb;
  reg a,b,en;                          // input port are declared in reg(register)
  wire [3:0]y;                         // output port are declared in wire(net)
decoder24_gate dut(en,a,b,y);          // instantiate design block
  initial
    begin
      en=0;a=1'bx;b=1'bx;#5
      en=1;a=0;b=0;#5
      en=1;a=0;b=1;#5
      en=1;a=1;b=0;#5
      en=1;a=1;b=1;#5
     $finish;                          // terminate simulation using $finish system task
    end
endmodule
```

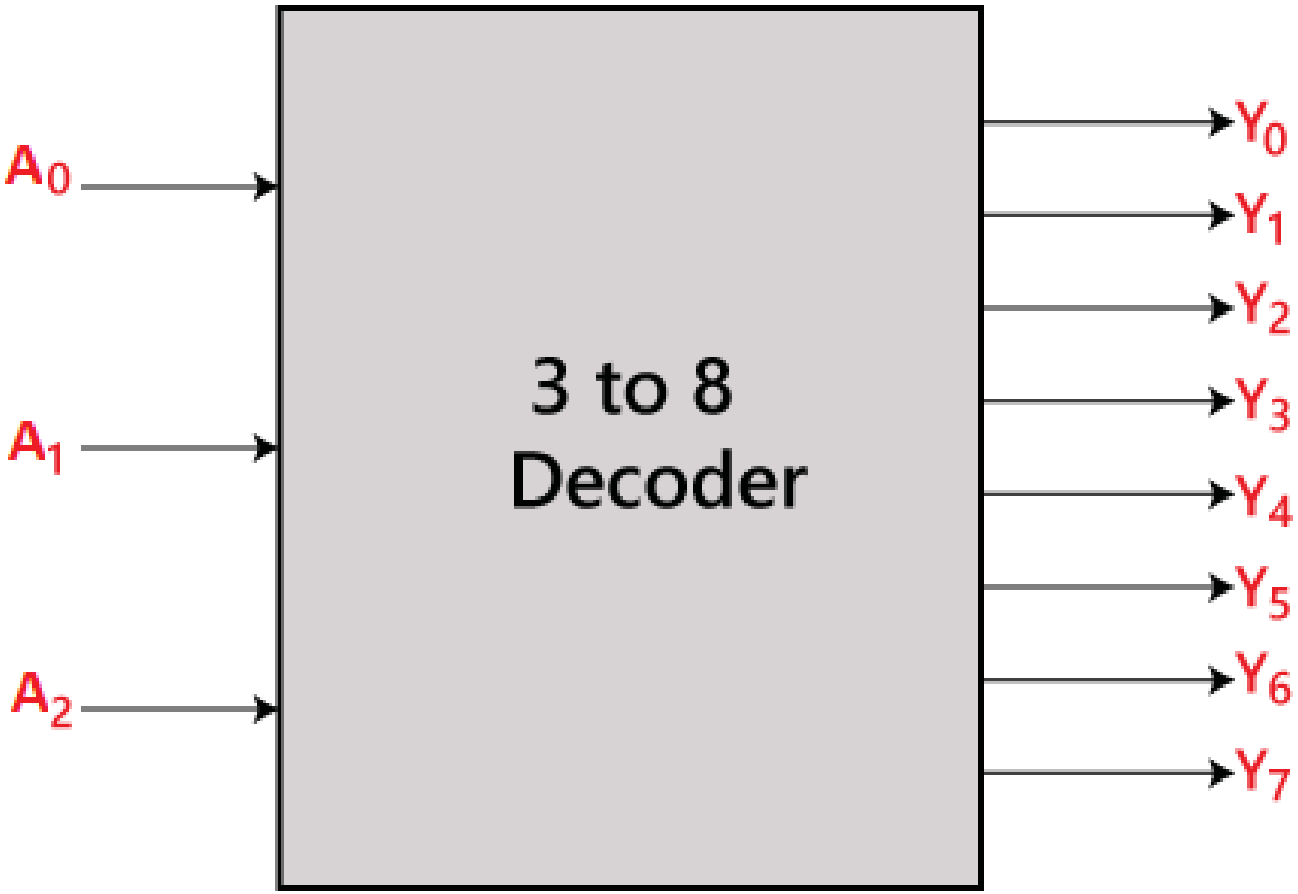
Simulation output & Schematic





Decoder - 3 to 8 line decoder:

- The 3 to 8 line decoder is also known as **Binary to Octal Decoder.**
- In a 3 to 8 line decoder, there is a total of **8 outputs**, i.e., **Y0, Y1, Y2, Y3, Y4, Y5, Y6, and Y7** and **3 inputs, i.e., A0, A1, and A2**.
- This circuit has an enable input 'E'.
- Just like 2 to 4 line decoder, when enable 'E' is set to 1, one of these 8 outputs will be 1.
- The block diagram and the truth table of the 3 to 8 line encoder are given below.



Decoder - 3 to 8 line decoder:

Truth Table:

Enable	INPUTS			Outputs							
E	A_2	A_1	A_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	x	x	x	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

The logical expression of the term Y_0, Y_1, Y_2, Y_3, Y_4, Y_5, Y_6, and Y_7 is as follows:

$$Y_0 = A_0'.A_1'.A_2'$$

$$Y_1 = A_0.A_1'.A_2'$$

$$Y_2 = A_0'.A_1.A_2'$$

$$Y_3 = A_0.A_1.A_2'$$

$$Y_4 = A_0'.A_1'.A_2$$

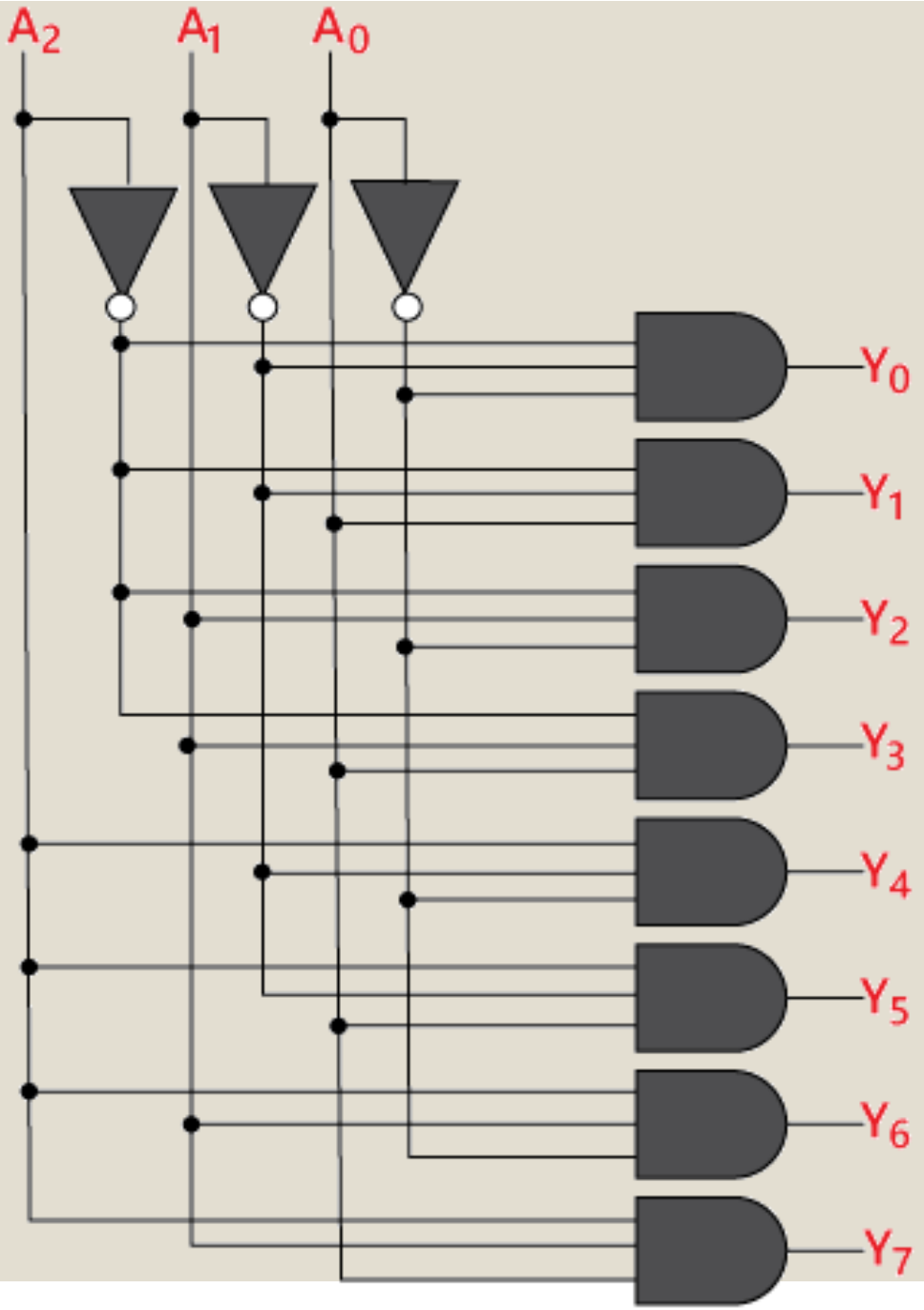
$$Y_5 = A_0.A_1'.A_2$$

$$Y_6 = A_0'.A_1.A_2$$

$$Y_7 = A_0.A_1.A_2$$

Decoder –
3 to 8 line decoder:

- Logical circuit of the above expressions is given :



a) Structural modelling (**gate level**): Decoder

```
module decoder( input E,a2,a1,a0,
output [7:0]Y );
not(a2n,a2);
not(a1n,a1);
not(a0n,a0);
and(Y[0],a2n,a1n,a0n,E);
and(Y[1],a2n,a1n,a0,E);
and(Y[2],a2n,a1,a0n,E);
and(Y[3],a2n,a1,a0,E);
and(Y[4],a2,a1n,a0n,E);
and(Y[5],a2,a1n,a0,E);
and(Y[6],a2,a1,a0n,E);
and(Y[7],a2,a1,a0,E);
endmodule
```

```
module tb;
wire [7:0]Y;
reg E,a2,a1,a0;
decoder A1(E,a2,a1,a0,Y);
initial begin
E=1;a2=0;a1=0;a0=0;
#5 E=1;a2=0;a1=0;a0=1;
#5 E=1;a2=0;a1=1;a0=0;
#5 E=1;a2=0;a1=1;a0=1;
#5 E=1;a2=1;a1=0;a0=0;
#5 E=1;a2=1;a1=0;a0=1;
#5 E=1;a2=1;a1=1;a0=0;
#5 E=1;a2=1;a1=1;a0=1;
#15 $finish;
end
endmodule
```

b) Data Flow modelling: Decoder

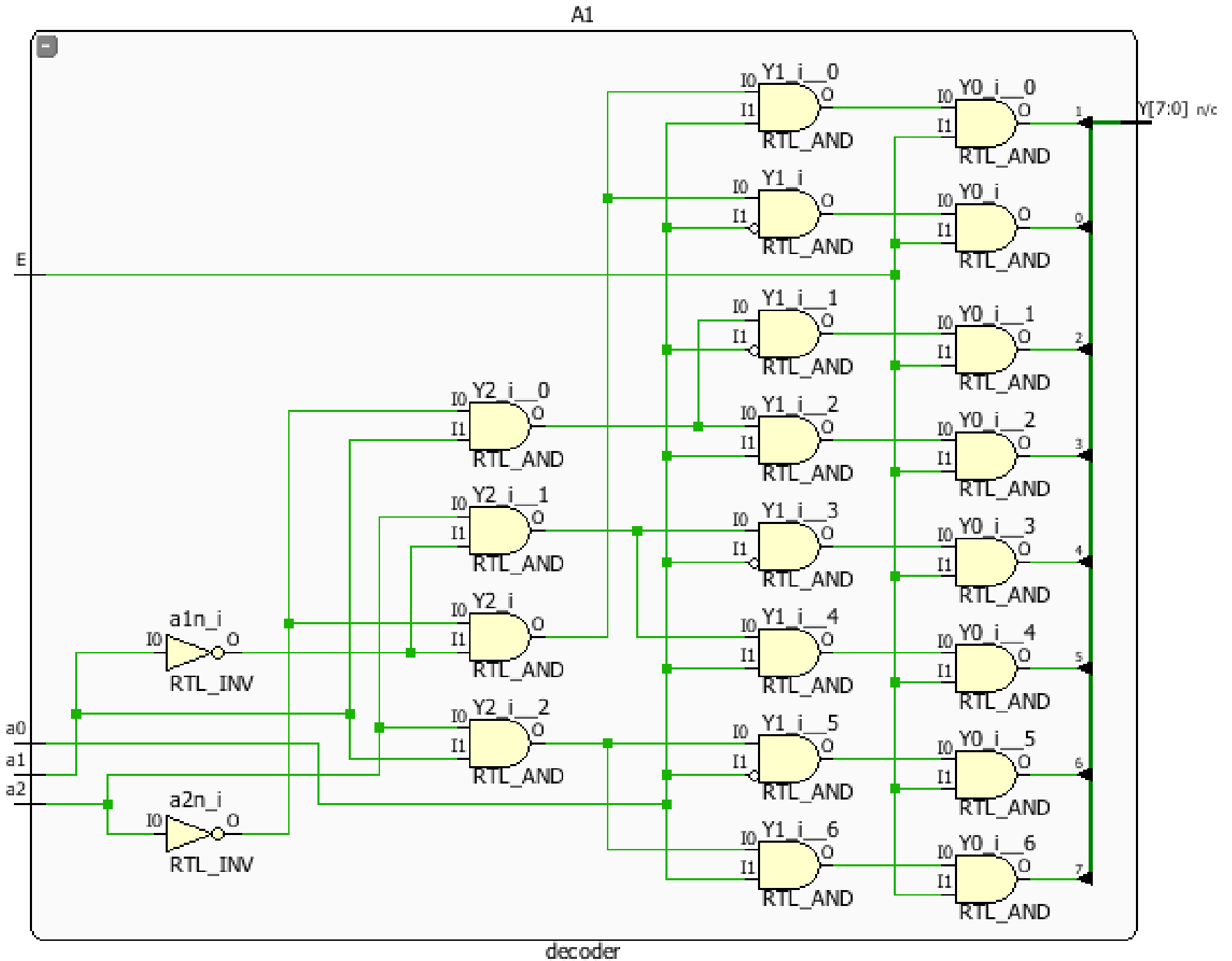
```
module decoder( input E,a2,a1,a0, output [7:0]Y);
assign Y[0]=((~a2)&(~a1)&(~a0)&E);
assign Y[1]=((~a2)&(~a1)&(a0)&E);
assign Y[2]=((~a2)&(a1)&(~a0)&E);
assign Y[3]=((~a2)&(a1)&(a0)&E);
assign Y[4]=((a2)&(~a1)&(~a0)&E);
assign Y[5]=((a2)&(~a1)&(a0)&E);
assign Y[6]=((a2)&(a1)&(~a0)&E);
assign Y[7]=((a2)&(a1)&(a0)&E);
endmodule
```

c) Behavioral Modelling: Decoder

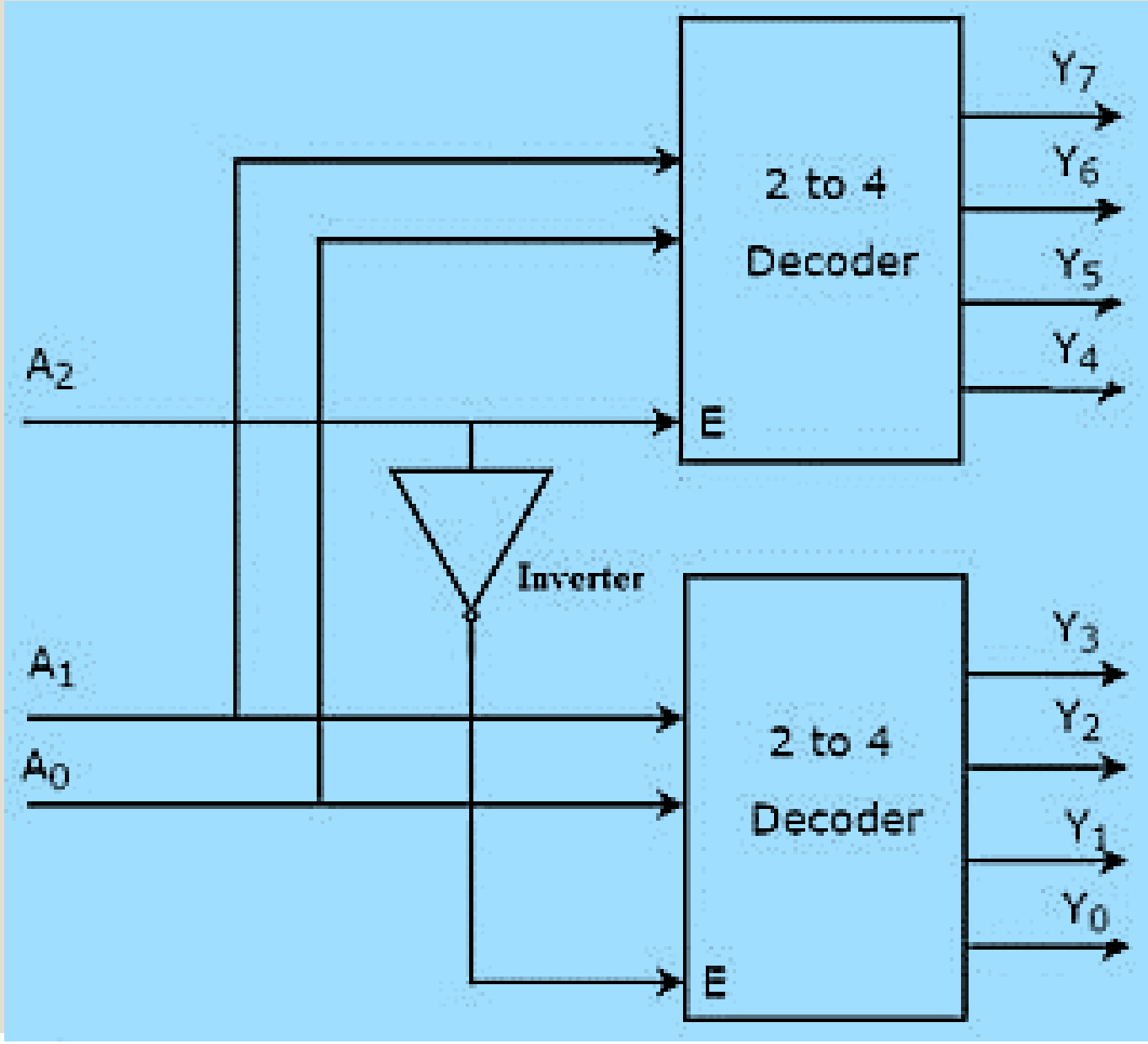
```
module decoder( input E,a2,a1,a0, output reg [7:0]Y);
always@(E,a2,a1,a0)
if(E==1)
case({a2,a1,a0})
3'b000:Y=8'd1;
3'b001:Y=8'd2;
3'b010:Y=8'd4;
3'b011:Y=8'd8;
3'b100:Y=8'd16;
3'b101:Y=8'd32;
3'b110:Y=8'd64;
3'b111:Y=8'd128;
default:Y=1'b0;
endcase
endmodule
```

Simulation Output

Schemat


A1
E
a0
a1
a2
a1n_i
a2n_i
RTL_INV
Y2_i__0
Y2_i__1
Y2_i__2
Y2_i
Y1_i__0
Y1_i__1
Y1_i__2
Y1_i__3
Y1_i__4
Y1_i__5
Y1_i__6
Y1_i
Y0_i__0
Y0_i__1
Y0_i__2
Y0_i__3
Y0_i__4
Y0_i__5
Y0_i__6
Y0_i
RTL_AND
Y[7:0]
n/c
decoder

3 to 8 line decoder using 2 to 4 decoder:



Verilog code

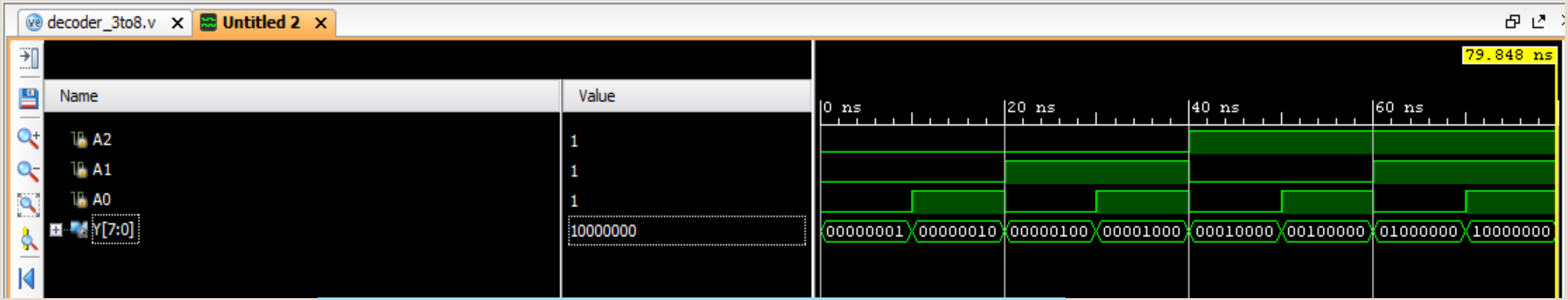
```
module decoder_3to8 ( input A2, A1, A0,  output [7:0] Y );
    wire E1, E2;

   assign E1 = A2;

   assign E2 = ~A2;

   // Instantiating two 2-to-4 decoders

decoder_2to4 dec1 (.A1(A1), .A0(A0), .E(E1), .Y(Y[7:4]));

decoder_2to4 dec2 (.A1(A1), .A0(A0), .E(E2), .Y(Y[3:0]));

endmodule
```

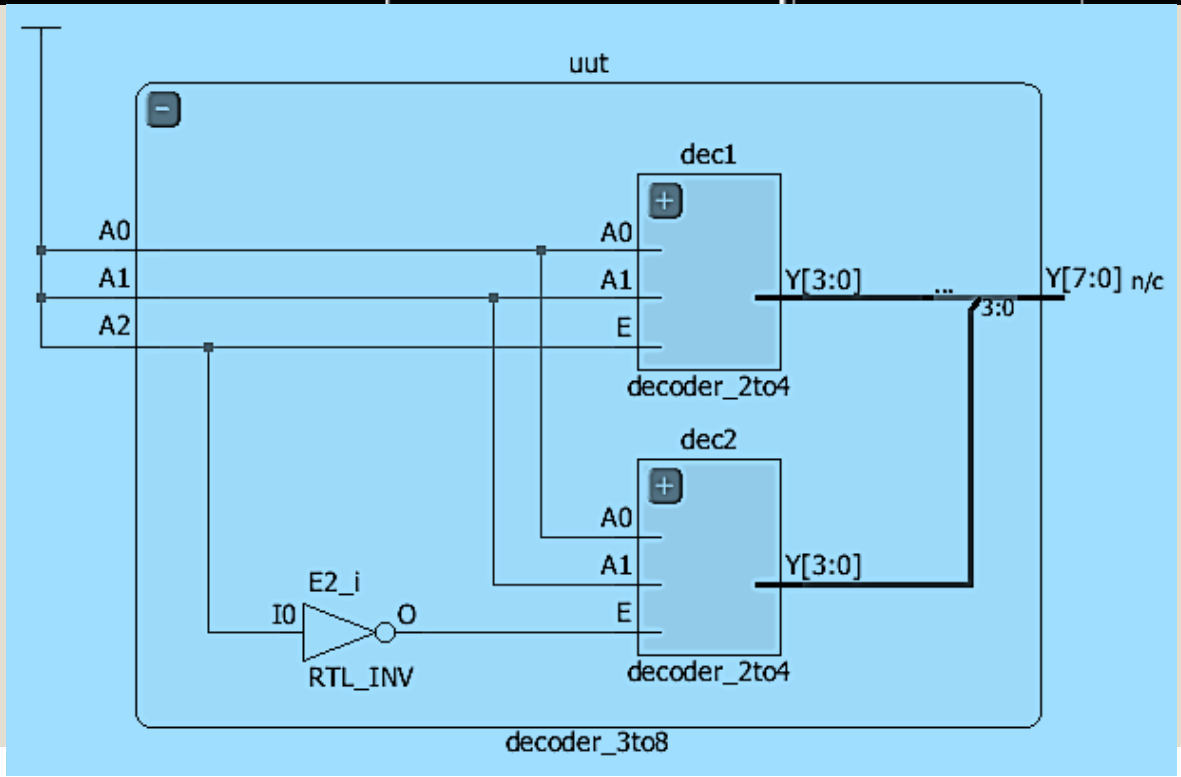
```
// 2-to-4 Decoder Module
module decoder_2to4 (
    input A1, A0, E,
    output [3:0] Y);

    assign Y[0] = E & ~A1 &~A0;
    assign Y[1] = E & ~A1 & A0;
    assign Y[2] = E & A1 & ~A0;
    assign Y[3] = E & A1 & A0;

endmodule
```

Verilog code

```
module testbench;
    reg A2, A1, A0;
    wire [7:0] Y;

    // Instantiate the 3-to-8 decoder
    decoder_3to8 uut ( .A2(A2), .A1(A1), .A0(A0),   .Y(Y)  );

   initial
   begin
        // Test each input combination
            A2 = 0; A1 = 0; A0 = 0; #10;
            A2 = 0; A1 = 0; A0 = 1; #10;
            A2 = 0; A1 = 1; A0 = 0; #10;
            A2 = 0; A1 = 1; A0 = 1; #10;
            A2 = 1; A1 = 0; A0 = 0; #10;
            A2 = 1; A1 = 0; A0 = 1; #10;
            A2 = 1; A1 = 1; A0 = 0; #10;
            A2 = 1; A1 = 1; A0 = 1; #10;
            $finish;
end
endmodule
```

Simulation output & Schematic





P-2

Dataflow Modelling: Continuous assignment, Delays, Expressions, Operators, Operands, Operator Types, and Examples.

Dataflow Modeling

- Dataflow modeling provides a powerful way to implement a design.
- Verilog allows a circuit to be designed in terms of the data flow between registers and how a design processes data rather than instantiation of individual gates.
- Automated tools are used to create a gate-level circuit from a dataflow design description.
- This process is called logic synthesis. Dataflow modeling has become a popular design approach as logic synthesis tools have become sophisticated.
- This approach allows the designer to concentrate on optimizing the circuit in terms of data flow.
- **For maximum flexibility in the design process, designers typically use a Verilog description style that combines the concepts of gate-level, data flow, and behavioural design.**
- In the digital design community, the term RTL (Register Transfer Level) design is commonly used for a combination of dataflow modeling and behavioural modeling.

Continuous Assignments

- A continuous assignment is the most basic statement in dataflow modeling, used to drive a value onto a net.
- This assignment replaces gates in the description of the circuit and describes the circuit at a higher level of abstraction.
- The assignment statement starts with the **keyword assign.**
- In Verilog, continuous assignments are used to create connections between signals where the value of a signal is always kept up-to-date based on an expression.
- This is a fundamental concept in digital design and hardware description, allowing you to create combinational logic where the output automatically reflects changes in the inputs.

Continuous Assignments

- **The syntax of an assign statement is : assign <signal_name> = <expression>;**

```
continuous_assign ::= assign [ drive_strength ][delay3]
list_of_net_assignments ;
```

- This is a general structure for a continuous assignment.
- **assign:** This keyword begins the continuous assignment.
- **[drive_strength]:** Optional. Specifies the strength of the signal driving the net. This can be useful when simulating real-world effects of signal strengths.
- **[delay3]:** Optional. Adds a delay to the assignment, simulating propagation delay in hardware.
- **list_of_net_assignments:** This is a list of assignments where each net on the left side of = is assigned a value (expression) on the right side.

Continuous Assignments

- **list_of_net_assignments ::= net_assignment { , net_assignment }**
- A list of net_assignment elements separated by commas.
- Each net_assignment represents one continuous assignment.
- **net_assignment ::= net_lvalue** = expression
- **net_lvalue:** The left-hand side of the assignment, representing the net (e.g., wire) that will receive the driven value.
- **=:** The assignment operator.
- expression: The right-hand side, representing the value or expression being assigned to the net.

Continuous Assignments

- **Example:**
- **assign (strong1, weak0) #10 a = b & c, d = e | f;**
- assign: Starts the continuous assignment.
- (strong1, weak0): Sets the drive strength (**strong1 for logic 1, weak0 for logic 0**).
- **#10**: Specifies a delay of 10 time units.
- **a = b & c, d = e | f;**
  - Multiple net_assignments, where a is driven by b & c, and d by e | f.

Continuous assignments have the following characteristics:

1. The left hand side of an assignment must always be a scalar or vector net or a concatenation of scalar and vector nets. It cannot be a scalar or vector register.
2. Continuous assignments are always active. The assignment expression is evaluated as soon as one of the right-hand-side operands changes and the value is assigned to the left-hand-side net.
3. The operands on the right-hand side can be **registers or nets or function calls**. Registers or nets can be scalars or vectors.
4. Delay values can be specified for assignments in terms of time units. Delay values are used to control the time when a net is assigned the evaluated value. This feature is similar to specifying delays for gates. It is very useful in modeling timing behaviour in real circuits.

Examples of Continuous Assignment

```
// Continuous assign. out is a net. i1 and i2 are nets.
assign out = i1 & i2;
// Continuous assign for vector nets. addr is a 16-bit
vector net and  addr1 and addr2 are 16-bit vector
registers.
assign addr[15:0] = addr1_bits[15:0]^ addr2_bits[15:0];
// Concatenation. Left-hand side is a concatenation of
a scalar net and a vector net.
assign {c_out, sum[3:0]} = a[3:0] + b[3:0] + c_in;
```

Implicit Continuous Assignment

- Instead of declaring a net and then writing a continuous assignment on the net, **Verilog provides a shortcut by which a continuous assignment can be placed on a net when it is declared.**
- There can be only one implicit declaration assignment per net because a net is declared only once.
- Eg: **`//Regular continuous assignment`**

```
wire out;
assign out = in1 & in2;
```

`//Same effect is achieved by an implicit continuous assignment`

```
wire out = in1 & in2;
```

Implicit Net Declaration

- If a signal name is used to the left of the continuous assignment, an implicit net declaration will be inferred for that signal name.
- If the net is connected to a module port, the width of the inferred net is equal to the width of the module port.
- Eg:

```
// Continuous assign. out is a net.
                    wire i1, i2;
                  assign out = i1 & i2;
//Note that out was not declared as a wire but an implicit wire
declaration for out is done by the simulator
```

Delays

- Delay values control the time between the change in a right-hand-side operand and when the new value is assigned to the left-hand side.
- Three ways of specifying delays in continuous assignment statements are:

1. Regular assignment delay,
2. Implicit continuous assignment delay,
3. Net declaration delay.

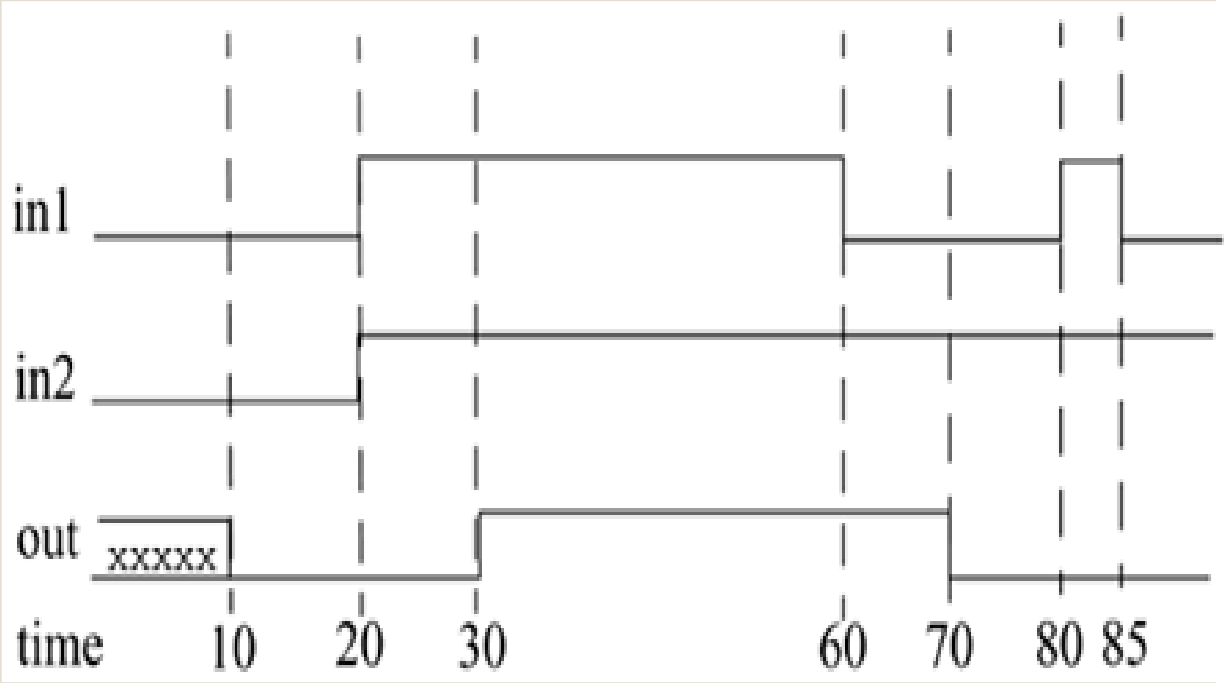
Regular Assignment Delay

- The first method is to assign a delay value in a continuous assignment statement.
- **The delay value is specified after the keyword assign.**
- Any change in values of in1 or in2 will result in a delay of 10 time units before re-computation of the expression in1 & in2, and the result will be assigned to out.
- If in1 or in2 changes value again before 10 time units when the result propagates to out, the values of in1 and in2 at the time of re-computation are considered.
- **This property is called inertial delay.**
- An input pulse that is shorter than the delay of the assignment statement does not propagate to the output.

Regular Assignment Delay

Eg: **assign #10 out = in1 & in2;** // Delay in a continuous assign

1. When signals in1 and in2 go high at time 20, out goes to a high 10 time units later (time = 30).
2. When in1 goes low at 60, out changes to low at 70.
3. However, in1 changes to high at 80, but it goes down to low before 10 time units have elapsed.
4. Hence, at the time of re-computation, 10 units after time 80, in1 is 0. Thus, out gets the value 0. **A pulse of width less than the specified assignment delay is not propagated to the output**.



Implicit Continuous Assignment Delay

- An equivalent method is to use an implicit continuous assignment to specify both a delay and an assignment on the net.

```
//implicit continuous assignment delay
wire #10 out = in1 & in2;
//same as
wire out;
assign #10 out = in1 & in2;
```

Net Declaration Delay

- A delay can be specified on a net when it is declared without putting a continuous assignment on the net.
- If a delay is specified on a net out, then any value change applied to the net out is delayed accordingly.
- Net declaration delays can also be used in gate-level modeling.
- **//Net Delays**

```
wire #10 out;
assign out = in1 & in2;
```

//The above statement has the same effect as the following.

```
wire out;
assign #10 out = in1 & in2;
```

Expressions, Operators, and Operands

- Dataflow modeling describes the design in terms of expressions instead of primitive gates. Expressions, operators, and operands form the basis of dataflow modeling.

Expressions

- Expressions are constructs that combine operators and operands to produce a result.

```
// Examples of expressions.
```

- Combines operands and operators

```
a ^ b
addr1[20:17] + addr2[20:17]
in1 | in2
```

Operands

- Operands can be constants, integers, real numbers, nets, registers, times, bit-select (one bit of vector net or a vector register), part-select (selected bits of the vector net or register vector), and memories or function calls (functions are discussed later).

- ```
  integer count, final_count;
  final_count = count + 1;//count is an integer operand
  ```
  ```

Operands

- ```
  real a, b, c;
  c = a - b; //a and b are real operands
  ```
- ```
  reg [15:0] reg1, reg2;
  reg [3:0] reg_out;
  reg_out = reg1[3:0] ^ reg2[3:0
  ```
- ```
  //part-select register operands
  reg ret_value;
  ret_value = calculate_parity(A, B);//calculate_parity
                                  is a function type operand
  ```
  ```

Operators and Operator Types

- Operators act on the operands to produce desired results.
- Verilog provides various types of operators.
- Verilog provides many different operator types. Operators can be arithmetic, logical, relational, equality, bitwise, reduction, shift, concatenation, or conditional.
- Some of these operators are similar to the operators used in the C programming language.
- Each operator type is denoted by a symbol.

Operator Types

Table 6-1. Operator Types and Symbols

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Arithmetic	*	multiply	two
	/	divide	two
	+	add	two
	-	subtract	two
	%	modulus	two
	**	power (exponent)	two

Operator Types

Logical	!	logical negation	one
	&&	logical and	two
	||	logical or	two
Relational	>	greater than	two
	<	less than	two
	>=	greater than or equal	two
	<=	less than or equal	two
Equality	==	equality	two
	!=	inequality	two
	===	case equality	two
	!==	case inequality	two

Operator Types

Bitwise	~	bitwise negation	one
	&	bitwise and	two
	|	bitwise or	two
	^	bitwise xor	two
	^~ or ~^	bitwise xnor	two
Reduction	&	reduction and	one
	~&	reduction nand	one
	|	reduction or	one
	~|	reduction nor	one
	^	reduction xor	one
	^~ or ~^	reduction xnor	one

Operator Types

Shift	>>	Right shift	Two
	<<	Left shift	Two
	>>>	Arithmetic right shift	Two
	<<<	Arithmetic left shift	Two
Concatenation	{ }	Concatenation	Any number
Replication	{ { } }	Replication	Any number
Conditional	?:	Conditional	Three

Operator Types: **Arithmetic Operators**

Examples:

```
– (b + c)
(a – b) + (c * d)
(a + b) / (a – b)
a % b
a ** 3
```

Arithmetic Operators:

+	unary (sign) plus
–	unary (sign) minus
+	binary plus (add)
–	binary minus (subtract)
*	multiply
/	divide
%	modulus
**	exponentiation

- There are two types of arithmetic operators: **binary and unary.**
- **Binary operators:**
  - Binary arithmetic operators are multiply (*), divide (/), add (+), subtract (-), power (**), and modulus (%). Binary operators take two operands.

Arithmetic Operators : Binary operators

```
1. A = 4'b0011; B = 4'b0100; // A and B are register vectors
2. D = 6; E = 4; F=2  // D and E are integers
3. A * B // Multiply A and B. Evaluates to 4'b1100
4. D / E // Divide D by E. Evaluates to 1.
          1. Truncates any fractional part.
5. A + B // Add A and B. Evaluates to 4'b0111
6. B - A // Subtract A from B. Evaluates to 4'b0001
7. F = E ** F; //E to the power F, yields 16
```

**** if an operand value is not known precisely, the result should be an unknown.**

**** Modulus operators produce the remainder from the division of two numbers.**

Arithmetic Operators : Unary operators

- The operators + and - can also work as unary operators.
- They are used to specify the positive or negative sign of the operand.
- Unary + or ? operators have higher precedence than the binary + or ? operators.
- -4 // Negative 4
- +5 // Positive 5
- Negative numbers are represented as 2's complement internally in Verilog.
- It is advisable to use negative numbers only of the type integer or real in expressions.

Arithmetic Operators : Unary operators

- **//Advisable to use integer or real numbers**
- **-10 / 5// Evaluates to -2**

//Do not use numbers of type <sss> '<base> <nnn>

- **-'d10 / 5//**
- Is equivalent (2's complement of 10)/5 = $(2^{32} - 10)/5$
- // where 32 is the default machine word width.
- // This evaluates to an incorrect and unexpected result

Arithmetic Operators : Unary operators

- To ensure correct signed arithmetic, you must explicitly declare the numbers as signed with a specific bit width.
- **$display("%0d", $signed(-10) / 5);**

  // Correctly evaluates to -2
- Alternatively, specify a fixed-width signed type for -'d10,

```
wire signed [31:0] num = -10;
$display("%0d", num / 5); // Correctly evaluates to -2
```

Logical Operators

- Logical operators are **logical-and (&&), logical-or (||) and logical-not (!).**
- Operators **&& and || are binary operators**. Operator **! is a unary operator.**
- **Logical operators follow these conditions:**

1. Logical operators always evaluate to a 1-bit value, 0 (false), 1 (true), or x (ambiguous).
2. If an operand is not equal to zero, it is equivalent to a logical 1 (true condition). If it is equal to zero, it is equivalent to a logical 0 (false condition). If any operand bit is x or z, it is equivalent to x (ambiguous condition) and is normally treated by simulators as a false condition.
3. Logical operators take variables or expressions as operands.

Logical Operators

- ***// Logical operations***

  A = 3; B = 0;

  1. A && B // Evaluates to 0. Equivalent to (logical-1 && logical-0)
  2. A || B // Evaluates to 1. Equivalent to (logical-1 || logical-0)
  3. !A // Evaluates to 0. Equivalent to not(logical-1)
  4. !B // Evaluates to 1. Equivalent to not(logical-0)

- ***// Unknowns***

  A = 2'b0x; B = 2'b10;

  A && B // Evaluates to x. Equivalent to (x && logical 1)

- ***// Expressions***

  (a == 2) && (b == 3)

  // Evaluates to 1 if both a == 2 and b == 3 are true. Evaluates to 0 if either is false.

Relational Operators

- Relational operators are **greater-than (>), less-than (<), greater-than-or-equal-to (>=), and less-than-or-equal-to (<=).**
- If relational operators are used in an expression, the expression returns a logical value of 1 if the expression is true and 0 if the expression is false.
- If there are any unknown or z bits in the operands, the expression takes a value x.
- These operators function exactly as the corresponding operators in the C programming language.

Relational Operators

// A = 4, B = 3

// X = 4'b1010, Y = 4'b1101, Z = 4'b1xxx

1. A <= B // Evaluates to a logical 0
2. A > B // Evaluates to a logical 1
3. Y >= X // Evaluates to a logical 1
4. Y < Z // Evaluates to an x

Equality Operators

- Equality operators are **logical equality (==), logical inequality (!=), case equality (===), and case inequality (!==).**
- When used in an expression, equality operators return logical value 1 if true, 0 if false.
- These operators compare the two operands bit by bit, with zero filling if the operands are of unequal length.

Expression	Description	Possible Logical Value
a == b	a equal to b, result unknown if x or z in a or b	0, 1, x
a != b	a not equal to b, result unknown if x or z in a or b	0, 1, x
a === b	a equal to b, including x and z	0, 1
a !== b	a not equal to b, including x and z	0, 1

Equality Operators

// A = 4 B = 3

// X = 4'b1010 Y = 4'b1101

// Z = 4'b1xxz M = 4'b1xxz N = 4'b1xxx

- **A == B // Results in logical 0**
- **X != Y // Results in logical 1**
- **X == Z // Results in x**
- **Z === M // Results in logical 1 (all bits match, including x and z)**
- **Z === N // Results in logical 0 (least significant bit does not match)**
- **M !== N // Results in logical 1**

Bitwise Operators

- Bitwise operators are negation (~), and(&), or (|), xor (^), xnor (^~, ~^).
- Bitwise operators perform a bit-by-bit operation on two operands.
- They take each bit in one operand and perform the operation with the corresponding bit in the other operand.
- If one operand is shorter than the other, it will be bit-extended with zeros to match the length of the longer operand.
- A z is treated as an x in a bitwise operation.
- The exception is the unary negation operator (~), which takes only one operand and operates on the bits of the single operand.

Bitwise Operators : Truth Tables

bitwise and	0	1	x
0	0	0	0
1	0	1	x
x	0	x	x

bitwise xor	0	1	x
0	0	1	x
1	1	0	x
x	x	x	x

bitwise or	0	1	x
0	0	1	x
1	1	1	1
x	x	1	x

bitwise xnor	0	1	x
0	1	0	x
1	0	1	x
x	x	x	x

bitwise negation	result
0	1
1	0
x	x

Bitwise Operators

- **// X = 4'b1010 Y = 4'b1101 Z = 4'b10x1**

```
~X        // Negation. Result is 4'b0101
X & Y     // Bitwise and. Result is 4'b1000
X | Y     // Bitwise or. Result is 4'b1111
X ^ Y     // Bitwise xor. Result is 4'b0111
X ^~ Y    // Bitwise xnor. Result is 4'b1000
X & Z     // Result is 4'b10x0
```

Bitwise Operators

- It is important to distinguish bitwise operators ~, &, and | from logical operators !, &&, ||.
- Logical operators always yield a logical value 0, 1, x, whereas bitwise operators yield a bit-by-bit value.
- Logical operators perform a logical operation, not a bit-by-bit operation.
- // X = 4'b1010, Y = 4'b0000

X | Y // bitwise operation. Result is 4'b1010

X || Y // logical operation. Equivalent to 1 || 0. Result is 1.

Operators: Logical vs. Bitwise

```
1'b1 && 1'b1        = 1        1'b1 & 1'b1        = 1
1'b1 && 1'b0        = 0        1'b1 & 1'b0        = 0
! 1'b1              = 0        ~ 1'b1             = 0
! 3'b000            = 1        ~ 3'b000           = 111
3'b111 && 3'b111    = 1        3'b111 & 3'b111    = 111
3'b100 && 3'b111    = 0        3'b000 & 3'b111    = 000
```

Reduction Operators

- **Reduction operators are and (&), nand (~&), or (|), nor (~|), xor (^), and xnor (~^, ^~).**
- Reduction operators take only one operand. Reduction operators perform a bitwise operation on a single vector operand and yield a 1-bit result.
- The difference is that bitwise operations are on bits from two different operands, whereas reduction operations are on the bits of the same operand.
- Reduction operators work bit by bit from right to left. Reduction nand, reduction nor, and reduction xnor are computed by inverting the result of the reduction and, reduction or, and reduction xor, respectively.

Reduction Operators

```
// X = 4'b1010
&X          //Equivalent to 1 & 0 & 1 & 0. Results in 1'b0
|X          //Equivalent to 1 | 0 | 1 | 0. Results in 1'b1
^X          //Equivalent to 1 ^ 0 ^ 1 ^ 0. Results in 1’b0
```

A reduction xor or xnor can be used for even or odd parity generation of a vector.

Shift Operators

- Shift operators are **right shift (>>), left shift (<<), arithmetic right shift (>>>), and arithmetic left shift (<<<).**
- Regular shift operators shift a vector operand to the right or the left by a specified number of bits.
- The operands are the vector and the number of bits to shift. When the bits are shifted, the **vacant bit positions are filled with zeros**.
- Shift operations do not wrap around.
- Arithmetic shift operators use the context of the expression to determine the value with which to fill the vacated bits.

Shift Operators

// X = 4'b1100

Y = X >> 1; //Y is 4'b0110. Shift right 1 bit. 0 filled in MSB position.

Y = X << 1; //Y is 4'b1000. Shift left 1 bit. 0 filled in LSB position.

Y = X << 2; //Y is 4'b0000. Shift left 2 bits.

- **integer a, b, c;** //Signed data types

a = 0;

b = -10; // 00111...10110 binary

c = a + (b >>> 3); // Results in -2 decimal, due to arithmetic shift

Shift Operators

integer a, b, c; //Signed data types

a = 0; b = -10; // 00111...10110 binary

c = a + (b >>> 3); // Results in -2 decimal, due to arithmetic shift

- Represent 10 in binary:
- 10 = 0000 0000 0000 0000 0000 0000 0000 1010 (32 bits)
- Take the **2's complement (invert all bits and add 1):**
- -10 = 1111 1111 1111 1111 1111 1111 1111 0110 (32 bits)
- **Thus, b = -10 is stored as 1111 1111 1111 1111 1111 1111 1111 0110**

Shift Operators

- **Arithmetic Right Shift (b >>> 3):**
- The arithmetic right shift operator (>>>) preserves the sign of the number during shifting. For signed numbers, it replicates the sign bit (most significant bit, MSB) during the shift.
- **b = -10 in binary: 1111 1111 1111 1111 1111 1111 1111 0110**
- Perform a 3-bit arithmetic right shift:

After >>> 3: 1111 1111 1111 1111 1111 1111 1111 1110

This new value represents -2 in decimal.

Addition (a + (b >>> 3)): a = 0 b >>> 3 = -2

Perform the addition: $c = 0+(-2) = -2$

Examples:

Find the value of Y = ((&A) | (B>>2) | (A ˆ B)); if A = 1011 and B = 1111

Compute &A (Bitwise AND Reduction of A):
The reduction AND operator (&) performs a logical AND operation across all bits of A.
For A=1011:

1∧0∧1∧1=0

So, &A=0.

Compute B >> 2 (Right Shift of B by 2 bits):

The right shift operator (>>) shifts the bits of B to the right by 2 positions, filling with zeros on the left.
For, B=1111:
B>>2=0011

Examples:

Compute A ˆ B (Bitwise XOR of A and B):
The bitwise XOR operator (ˆ) compares each bit of A and B, returning 1 if the bits are different, and 0 if they are the same.
For A=1011 and B=1111:
A⊕B=0100

Combine Using Bitwise OR (|):Now substitute the values into the expression: Y=(&A)|(B>>2)|(A⊕B)
Substitute the values:
Y=(0)|(0011)|(0100)
Perform the bitwise OR operation:
0 | 0011 | 0100 **=0111**
Y=0111
Decimal Value of Y:
In decimal, **0111=7**

Examples: (HW)

Find the value of :

Z = ((|B) & (A << 2) ^ (A + B));

If, A=1101 and B=1010.

Answer

Explanation:

1. `|B`: Bitwise OR reduction of B.
2. `A << 2`: Left shift A by 2 bits.
3. `A + B`: Bitwise addition of A and B.
4. `^`: Bitwise XOR operation.
5. `&`: Bitwise AND operation.

**** Answer: *Z*=111011Z=111011 (binary) or *Z*=59Z=59 (decimal).**

1. Compute |B (Bitwise OR Reduction of B):

The **bitwise OR** reduction (|) computes the logical OR of all bits in B.

For B=1010:

1|0|1|0=1

So, **|B=1**

2. Compute `A << 2` (Left Shift of A by 2 Bits):

The **left shift operator** (<<) shifts the bits of A to the left by 2 positions, filling with zeros on the right.

For A=1101:

A<<2=110100

So, **A<<2=110100**

(6 bits).

3. Compute `A + B` (Bitwise Addition of A and B):

The **addition** is performed bit-by-bit as binary addition.

For A=1101 and B=1010

1101+1010 =10111

So,

A+B=10111

(5 bits).

4. Compute (A << 2) ^ (A + B) (Bitwise XOR):

The bitwise XOR operator (^) compares each bit of the operands, returning 1 if the bits are different, and 0 if they are the same.

For A<<2=110100 and A+B=10111 (align bits):

A<<2=110100(6 bits)

A+B=0010111

(add leading zero to make it 6 bits)

Now perform the XOR operation:

110100⊕0010111 =100011.

5. Compute

`(|B) & ((A << 2) ^ (A + B))`:

The **bitwise AND operator** (`&`) compares each bit of the operands, returning `1` if both bits are `1`, and `0` otherwise.

For |B= 1 and 100011:

- Since |B=1, the result of the AND operation retains all bits of the second operand.

So, Z=100011

The value of Z is 100011 in binary, or ---- in decimal.

Examples:

1) Find the value of Y = ((&A) | (B<<2) | (A | B)), if A = 1101 and B = 1001.

2) Find the value of y, if y = ((A+B) && (|C)) + (A<<<2); A = 1101, B = 1010 and C = 0111.

3) Find the value of y, if y = ((A ˆ B) && (ˆ C))+((A>>>3)|(C<<2));

   A = 1101, B = 1010 and C = 0111.

Concatenation Operator

- The concatenation operator ({, }) provides a mechanism to append multiple operands.
- The operands must be sized.
- Unsized operands are not allowed because the size of each operand must be known for computation of the size of the result.
- Concatenations are expressed as operands within braces, with commas separating the operands.
- Operands can be scalar nets or registers, vector nets or registers, bit-select, part-select, or sized constants.

Concatination Operator

- Curly braces is the concatenation operator in Verilog we can use this to concatenate inputs and outputs.

Example :

```
module gate(a,b,cin,sum,carry);
input a,b;
output sum,carry;

assign {carry,sum} = a + b + cin;
//Concatination operator used

endmodule
```

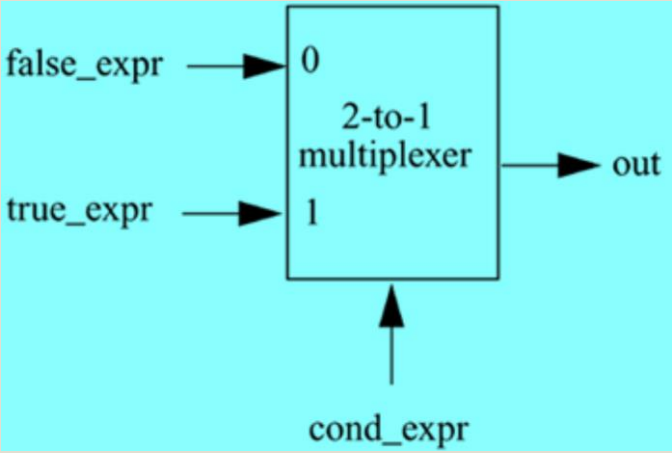
Replication Operator

- We can replicate any input and output using same curly braces.

```
module repli(a,b,y);
input a,b;
output [2:0]y;

wire x;
assign x = a & b;

assign y = {3{x}};
//replicating x three times

endmodule
```

Conditional Operator

- Question mark operator is the conditional operator, and it is used as follows
- assign y = condition?(if_true_output):(if_false_output);



2X1 MUX using conditional operation as given below:

```
module gate(a,b,s,y);
input a,b,s;
output y;

assign y = s?a:b;
//conditional operator used
//If s is 1, a is selected else b is selected, which is 2X1 MUX

endmodule
```

Conditional Operator

- Conditional operations can be nested.
- Each true_expr or false_expr can itself be a conditional operation.
- `assign out = (A == 3) ? ( control ? x : y ):( control ? m : n);`

Operator Precedence

- Having discussed the operators, it is now important to discuss operator precedence.
- If no parentheses are used to separate parts of expressions, Verilog enforces the followin precedence.
- It is recommended that parentheses be used to separate expressions except in case of unary operators or when there is no ambiguity.

Operator Precedence

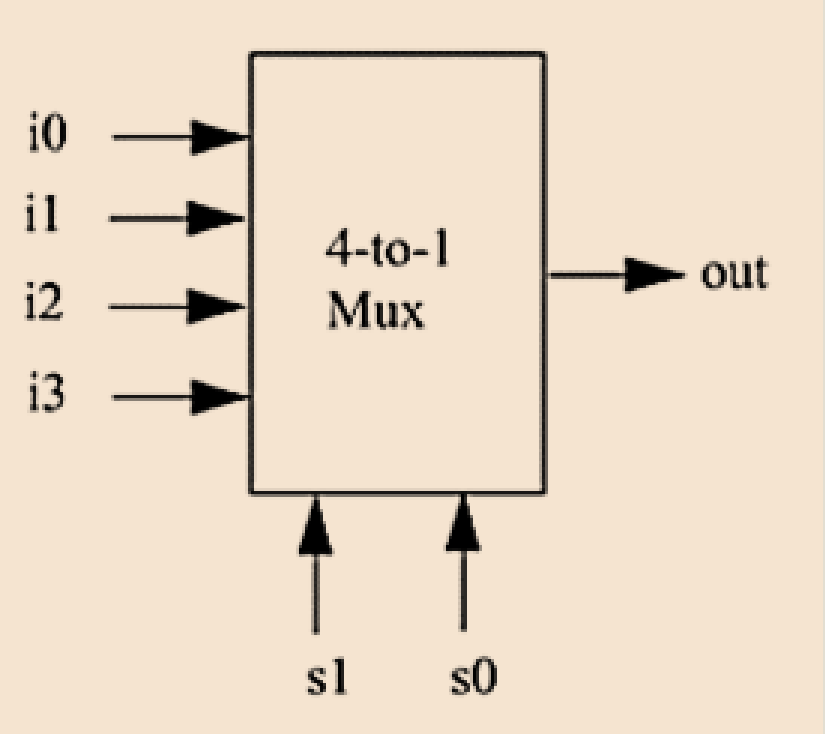
Operators	Operator Symbols	Precedence
Unary	+ - ! ~	Highest precedence
Multiply, Divide, Modulus	* / %	
Add, Subtract	+ -	
Shift	<< >>	
Relational	< <= > >=	
Equality	== != === !==	
Reduction	&, ~& ^ ^~ |, ~|	
Logical	&& ||	
Conditional	?:	Lowest precedence

Examples: 1 - 4-to-1 Multiplexer

- We can use assignment statements instead of gates to model the logic equations of the multiplexer.
- Notice that everything is same as the gate-level Verilog description except that computation of out is done by specifying one logic equation by using operators instead of individual gate instantiations. I/O ports remain the same.
- This is important so that the interface with the environment does not change. Only the internals of the module change.

Examples: 1 - 4-to-1 Multiplexer, a)Using Logic Equations

```
// Module 4-to-1 multiplexer using data flow. logic equation
// Compare to gate-level model
module mux4_to_1 (out, i0, i1, i2, i3, s1, s0);
// Port declarations from the I/O diagram
output out;
input i0, i1, i2, i3;
input s1, s0;
//Logic equation for out
assign out = (~s1 & ~s0 & i0)|
(~s1 & s0 & i1) |
(s1 & ~s0 & i2) |
(s1 & s0 & i3) ;
endmodule
```



Examples: 1 - 4-to-1 Multiplexer, b) using conditional operator

```
// Module 4-to-1 multiplexer using data flow. Conditional operator.
// Compare to gate-level model
module multiplexer4_to_1 (out, i0, i1, i2, i3, s1, s0);
// Port declarations from the I/O diagram
output out;
input i0, i1, i2, i3;
input s1, s0;
// Use nested conditional operator
assign out = s1 ? ( s0 ? i3 : i2) : (s0 ? i1 : i0) ;
endmodule
```

Examples: 1 - 4-to-1 Multiplexer, b) using conditional operator

```
// Module 4-to-1 multiplexer using data flow. Conditional operator.
// Compare to gate-level model
module multiplexer4_to_1 (out, i0, i1, i2, i3, s1, s0);
// Port declarations from the I/O diagram
output out;
input i0, i1, i2, i3;
input s1, s0;
// Use nested conditional operator
assign out = s1 ? ( s0 ? i3 : i2) : (s0 ? i1 : i0) ;
endmodule
```

Examples: 1 - 4-to-1 Multiplexer, b) using conditional operator

1. Outer Ternary:

If s1 is true: The value of out is determined by the inner ternary (s0 ? i3 : i2).

If s1 is false: The value of out is determined by the inner ternary (s0 ? i1 : i0).

2. Inner Ternary:

Each inner ternary decides between two inputs based on the value of s0:

- **If s0 is true:**
  - **s0 ? i3 : i2 evaluates to i3.**
  - **s0 ? i1 : i0 evaluates to i1.**
- **If s0 is false:**
  - **s0 ? i3 : i2 evaluates to i2.**
  - **s0 ? i1 : i0 evaluates to i0.**

Examples: 2 - 4-bit Full Adder a) Using dataflow operators

```
// Define a 4-bit full adder by using dataflow statements.
module fulladd4(sum, c_out, a, b, c_in);
// I/O port declarations
output [3:0] sum;
output c_out;
input[3:0] a, b;
input c_in;
// Specify the function of a full adder
assign {c_out, sum} = a + b + c_in;
endmodule
```

Examples: 3 - 4-bit Full Adder with Carry Lookahead

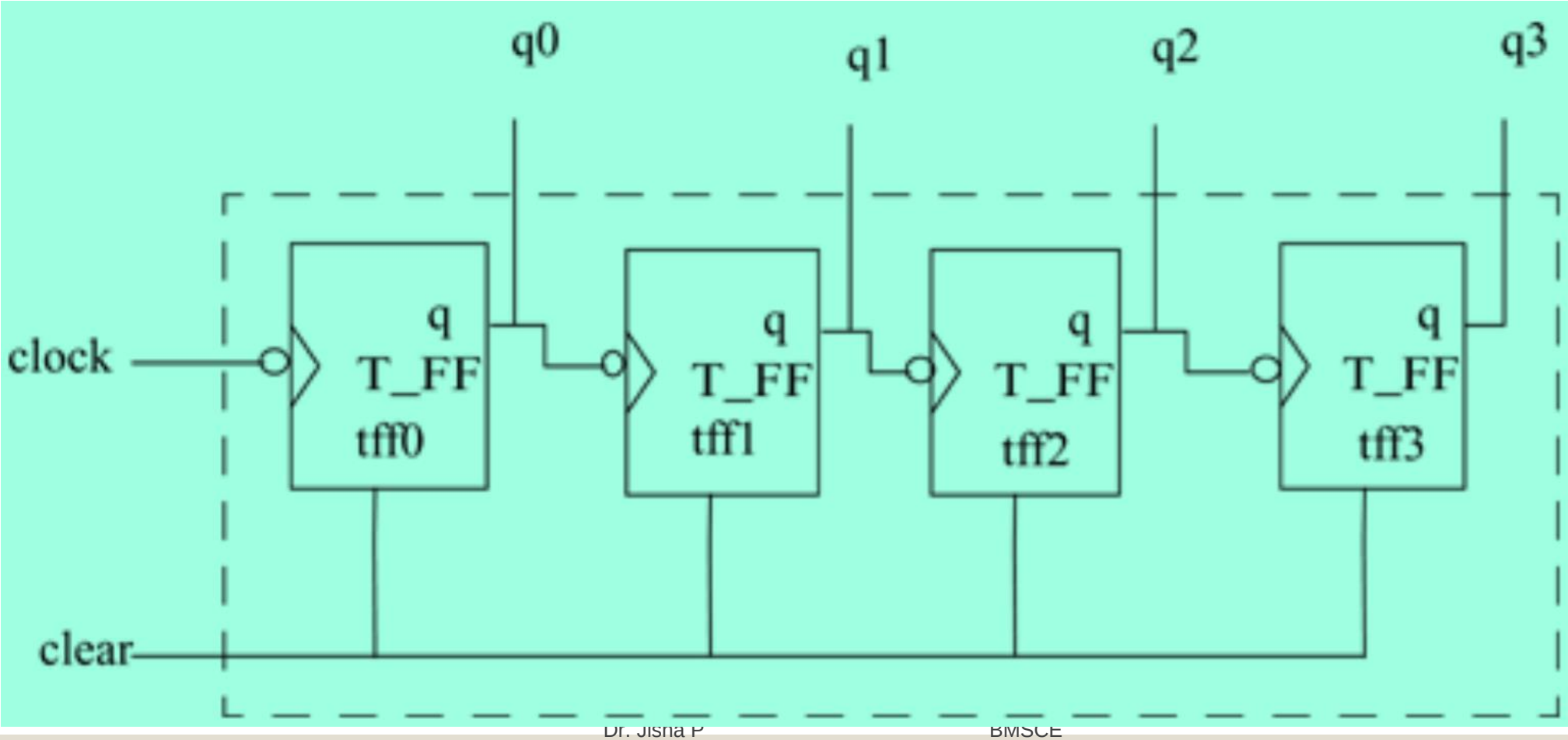
```
module fulladd4(sum, c_out, a, b, c_in);
// Inputs and outputs
output [3:0] sum;
output c_out;
input [3:0] a,b;
input c_in;
// Internal wires
wire p0,g0, p1,g1, p2,g2, p3,g3;
wire c4, c3, c2, c1;
// compute the p for each stage
assign p0 = a[0] ^ b[0], p1 = a[1] ^ b[1], p2 =
a[2] ^ b[2],  p3 = a[3] ^ b[3];
// compute the g for each stage
assign g0 = a[0] & b[0], g1 = a[1] & b[1],
g2 = a[2] & b[2], g3 = a[3] & b[3];
```

```
// carry lookahead computation
assign c1 = g0 | (p0 & c_in),
c2 = g1 | (p1 & g0) | (p1 & p0 & c_in),
c3 = g2 | (p2 & g1) | (p2 & p1 & g0) | (p2 & p1 & p0 &
c_in),
c4 = g3 | (p3 & g2) | (p3 & p2 & g1) | (p3 & p2 & p1 &
g0) |
(p3 & p2 & p1 & p0 & c_in);
// Compute Sum
assign sum[0] = p0 ^ c_in,
sum[1] = p1 ^ c1,
sum[2] = p2 ^ c2,
sum[3] = p3 ^ c3;
// Assign carry output
assign c_out = c4;
endmodule
```

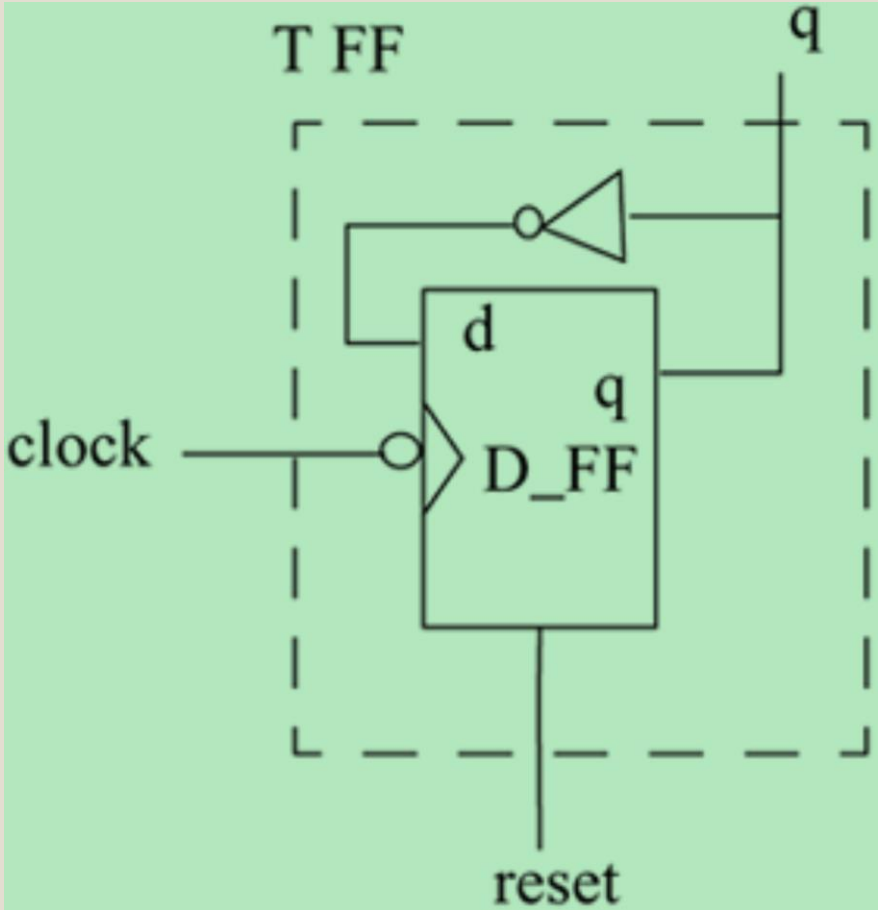
Examples: 4 - Ripple Counter

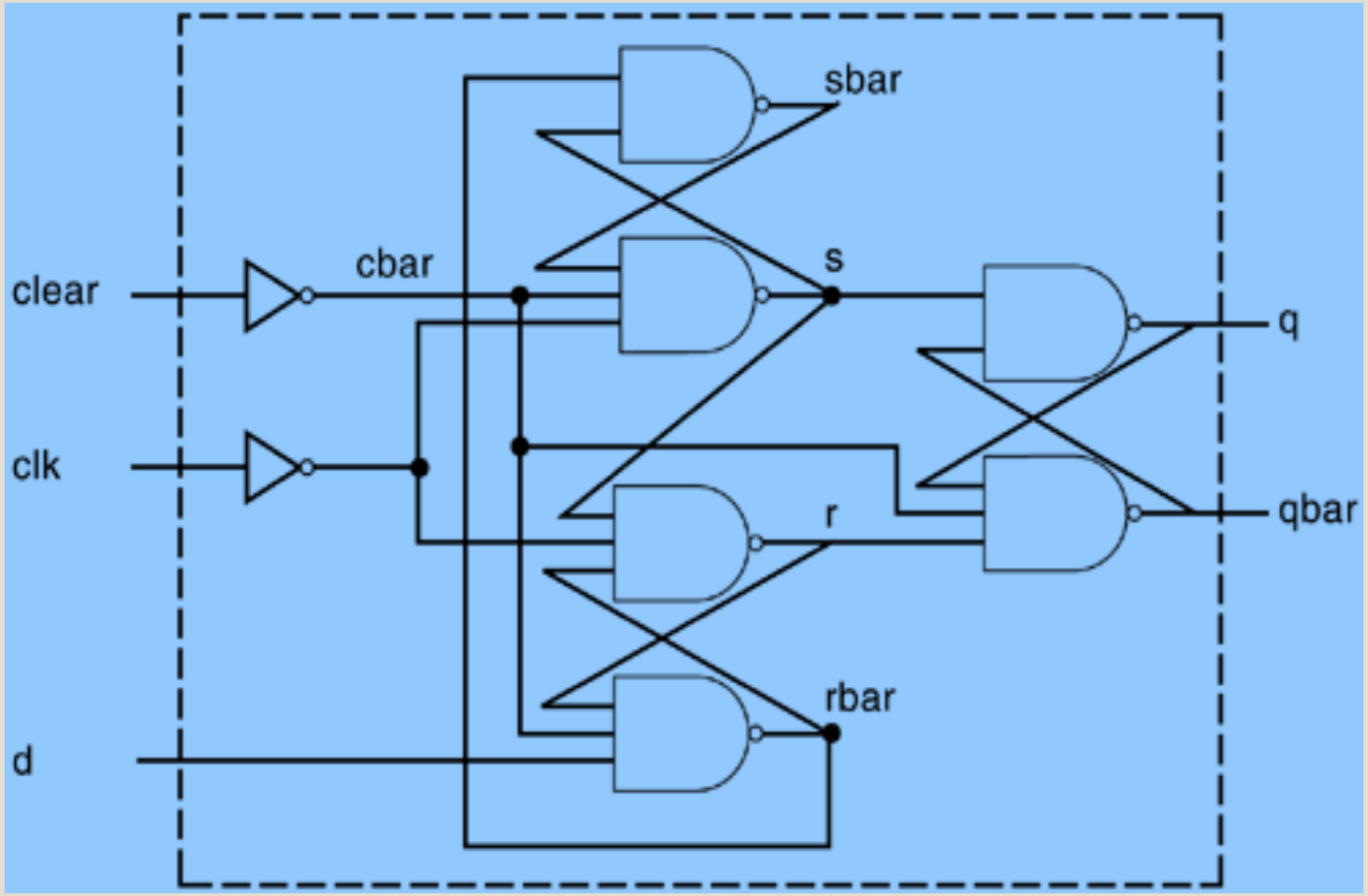
Design a 4-bit ripple counter by using negative edge-triggered T flipflops.

The diagrams for the 4-bit ripple carry counter modules are shown below.



Examples: 4 - Ripple Counter





Negative Edge-Triggered D-flipflop with Clear

T-flipflop is built with one D-flipflop and an inverter gate.

Examples: 4 - Ripple Counter

```
// Ripple counter
module counter(Q , clock, clear);
// I/O ports
output [3:0] Q;
input clock, clear;
// Instantiate the T flipflops
T_FF tff0(Q[0], clock, clear);
T_FF tff1(Q[1], Q[0], clear);
T_FF tff2(Q[2], Q[1], clear);
T_FF tff3(Q[3], Q[2], clear);
endmodule
```

Examples: 4 - Ripple Counter

Verilog Code for T-flipflop

```
// Edge-triggered T-flipflop. Toggles every clock cycle.
module T_FF(q, clk, clear);
// I/O ports
output q;
input clk, clear;
// Instantiate the edge-triggered DFF Complement of output q is fed back.
// Notice qbar not needed. Unconnected port.
edge_dff ff1(q, ,~q, clk, clear);
endmodule
```

Examples: 4 - Ripple Counter

Verilog Code for Edge-Triggered D-flipflop:

```
// Edge-triggered D flipflop
module edge_dff(q, qbar, d, clk, clear);
// Inputs and outputs
output q,qbar;
input d, clk, clear;
// Internal variables
wire s, sbar, r, rbar,cbar;
// dataflow statements Create a complement of signal clear
assign cbar = ~clear;
// Input latches; A latch is level sensitive. An edge-sensitive
// flip-flop is implemented by using 3 SR latches.
assign sbar = ~(rbar & s),
s = ~(sbar & cbar & ~clk),
r = ~(rbar & ~clk & s),
rbar = ~(r & cbar & d);
// Output latch
assign q = ~(s & qbar),
qbar = ~(q & r & cbar);
endmodule
```

Examples: 4 - Ripple Counter - Stimulus Module

```
module stimulus;
// Declare variables for stimulating input
reg CLOCK, CLEAR;
wire [3:0] Q;
initial
$monitor($time, " Count Q = %b Clear=
%b", Q[3:0],CLEAR);
// Instantiate the design block counter
counter c1(Q, CLOCK, CLEAR);
// Stimulate the Clear Signal
initial
begin
CLEAR = 1'b1;
#34 CLEAR = 1'b0;
#200 CLEAR = 1'b1;
#50 CLEAR = 1’b0;
end
```

```
// Set up the clock to toggle every 10
time units
initial
begin
CLOCK = 1'b0;
forever #10 CLOCK = ~CLOCK;
end
// Finish the simulation at time 400
initial
begin
#400 $finish;
end
endmodule
```

Examples: 4 - Ripple Counter - Stimulus Module

The output of the simulation is shown below. Note that the clear signal resets the count to zero.

```
0 Count Q = 0000 Clear= 1
34 Count Q = 0000 Clear= 0
40 Count Q = 0001 Clear= 0
60 Count Q = 0010 Clear= 0
80 Count Q = 0011 Clear= 0
100 Count Q = 0100 Clear= 0
120 Count Q = 0101 Clear= 0
140 Count Q = 0110 Clear= 0
160 Count Q = 0111 Clear= 0
180 Count Q = 1000 Clear= 0
200 Count Q = 1001 Clear= 0
```

```
200 Count Q = 1001 Clear= 0
220 Count Q = 1010 Clear= 0
234 Count Q = 0000 Clear= 1
284 Count Q = 0000 Clear= 0
300 Count Q = 0001 Clear= 0
320 Count Q = 0010 Clear= 0
340 Count Q = 0011 Clear= 0
360 Count Q = 0100 Clear= 0
380 Count Q = 0101 Clear= 0
```

thank you